Section 1 — I/O Mapping

Part A: What Is I/O and Why Do We Even Need This?

A.1 The Basic Problem

Your computer has a **CPU** — the brain. The CPU needs to get data from two kinds of places:

Place Type 1: Memory (RAM)

- This is where your programs live while running.
- The CPU talks to RAM all the time — fetch instruction, fetch data, store result.
- RAM is fast, predictable, and "inside" the computer's core.

Place Type 2: I/O Devices

- Keyboard, mouse, hard disk, USB drive, monitor, printer, network card.
- These are "outside" the core. They are slower, weirder, and each one behaves differently.
- A keyboard sends one byte when you press a key. A hard disk sends millions of bytes in a burst. A monitor needs pixels painted constantly.

> **The Big Engineering Problem:** The CPU only has one set of wires (the system bus) to talk to EVERYTHING. How does it know whether the address it puts on those wires means "go to RAM location 5000" or "go ask the keyboard what key was pressed"?

This is the entire problem of **I/O Mapping**. Every solution to this problem is either:

- **Isolated I/O** — two separate worlds.
- **Memory-Mapped I/O** — one world, smart addressing.

Part B: The System Bus — Three Wires That Run Everything

Before we solve the I/O problem, we need to understand what the CPU actually uses to talk. It uses three groups of wires called **buses**.

B.1 What Is a Bus?

A **bus** is just a bundle of parallel wires that carries information. Think of it as a highway with multiple lanes. Each lane carries one bit (0 or 1).

Bus 1: The Address Bus

- **What it carries:** A number — the "location" the CPU wants to access.
- **Direction:** CPU → Everyone else (ONE-WAY only).
- **How many wires:** Depends on the CPU. 16 wires = can address $2^{16} = 65{,}536$ locations. 32 wires = $2^{32} = 4{,}294{,}967{,}296$ locations. 64 wires = $2^{64} = 18{,}446{,}744{,}073{,}709{,}551{,}616$ locations.
- **Example:** CPU puts `00000000000000000000001001110001000` (binary for 5000) on the address bus. Every device sees this number. But only the device that "owns" address 5000 responds.
- **Analogy:** The address bus is like shouting a house number on a street. Every house hears it, but only the house with that number opens the door.

Bus 2: The Data Bus

- **What it carries:** The actual information being moved — bytes, words, instructions.
- **Direction:** BOTH WAYS. CPU can send data OUT (write) or receive data IN (read).
- **How many wires:** Usually 8, 16, 32, or 64 bits wide. A 64-bit data bus moves 8 bytes at once.
- **Example:** CPU reads from address 5000. The device at 5000 puts `10110101` on the data bus. CPU grabs it.
- **Analogy:** The data bus is the delivery truck. It carries the package to or from the house.

Bus 3: The Control Bus

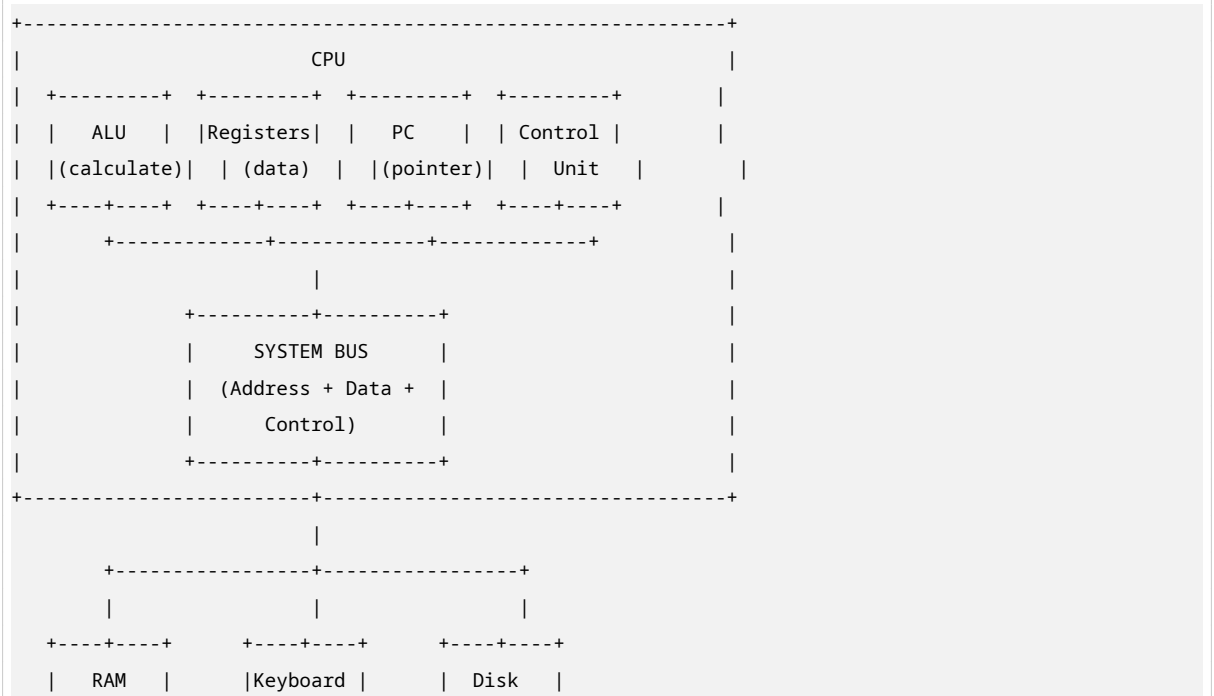
- **What it carries:** Command signals — "Are we reading? Writing? Is this memory or I/O? Is this valid?"
- **Direction:** Mostly CPU → Devices, but some signals come back (like "I'm ready" or "Error").

Key signals:

Signal	Description
$\overline{M/IO}$	Memory or I/O selector (only in Isolated I/O)
$\overline{RD}$	Read command (active LOW)
$\overline{WR}$	Write command (active LOW)
CLK	Clock signal (timing)
READY	Device says "I' m done, you can continue"

- **Analogy:** The control bus is the traffic cop. It tells everyone when to move, when to stop, and which direction to go.

B.2 Visual Diagram: How the Buses Connect



```
| (Memory)|      |  (I/O)  |      |  (I/O)  |
+---------+      +---------+      +---------+
```

> [!IMPORTANT] RAM and I/O devices sit on **the same wires**. The only way to tell them apart is through the **Control Bus signals** (Isolated I/O) or **Address decoding** (MMIO).

Part C: The Control Unit — The CPU's Traffic Controller

C.1 Where Does the Control Bus Get Its Signals?

Inside the CPU, there is a piece of hardware called the **Control Unit (CU)**.

What the CU does:

1. It looks at the instruction currently being executed (fetched from RAM).
2. It figures out what that instruction needs — read from memory? Write to I/O? Add two numbers?
3. It generates the correct pattern of 0s and 1s on the Control Bus.
4. It coordinates **timing** — it makes sure signals happen in the right order and at the right clock tick.

Example: If the instruction is `IN AL, 60H` (read from I/O port 60H, which is the keyboard):

- CU sets $\overline{M/IO} = 0$ (I/O mode).
- CU sets $\overline{RD} = 0$ (read mode).
- CU puts `0000000001100000` (60H in binary) on Address Bus.
- Keyboard sees: "Oh, address 60H, I/O mode, read command — that's me!" and puts the key code on Data Bus.

C.2 Combinational Logic — What the CU Is Made Of

The Control Unit is built from **combinational logic circuits**.

What does "combinational" mean?

- The output depends **ONLY** on the current inputs.
- There is **NO memory**. No "history." No "state."
- If input A and B are both 1, output is always 1. Every time. Forever.

Analogy: A calculator. You press 2 + 2, you get 4. The calculator doesn't remember you pressed 2 + 2 yesterday. It only cares about what you pressed RIGHT NOW.

The CU uses combinational logic to generate control signals because:

- It needs to produce the correct $\overline{M/IO}$, $\overline{RD}$, $\overline{WR}$ pattern instantly based on the current instruction.
- There is no "maybe" — the instruction `MOV AX, [5000]` **ALWAYS** means "read from memory," so the CU **ALWAYS** sets $\overline{M/IO} = 1$ and $\overline{RD} = 0$.

> [!NOTE] Some parts of the CPU (like registers, program counter) **DO** have memory — those are **sequential circuits**. But the control signal generation itself is combinational.

Part D: Isolated I/O — Two Completely Separate Worlds

D.1 The Core Idea

In **Isolated I/O** (also called **Port-Mapped I/O** or **PMIO**), the CPU has a special physical pin called $\overline{M/IO}$ that acts like a switch.

How it works:

- Before every read or write operation, the CPU sets $\overline{M/IO}$ to either 1 or 0.
- This tells EVERY DEVICE on the bus: "The next operation is for MEMORY" or "The next operation is for I/O."

```
M/IO̅ = 1  →  "Hey everyone, I'm talking to RAM now."
M/IO̅ = 0  →  "Hey everyone, I'm talking to an I/O device now."
```

Think of it like a train track switch:

- The train (data) wants to go to either Memory Station or I/O Station.
- Before the train moves, someone flips the switch ($\overline{M/IO}$).
- The train follows the track to the correct station.

D.2 The Four Control Signals

Now we add two more signals:

- $\overline{RD}$ (Read Bar) — goes LOW (0) when CPU wants to **RECEIVE** data.
- $\overline{WR}$ (Write Bar) — goes LOW (0) when CPU wants to **SEND** data.

Why the Bar on Top? What Is "Active Low"? This is **CRITICAL** to understand. In digital electronics:

- **Active HIGH:** Signal is "ON" when it is 1 (5V or 3.3V), "OFF" when it is 0 (0V).
- **Active LOW:** Signal is "ON" when it is 0 (0V), "OFF" when it is 1 (5V or 3.3V).

Why do we use active LOW for control signals?

Reason 1: Electrical reliability

- When a wire is at HIGH voltage (1), it can pick up electrical noise from nearby wires and accidentally flip to LOW.
- When a wire is at LOW voltage (0), it is grounded — very stable, hard to accidentally flip to HIGH.
- So "doing nothing" = HIGH (safe, stable). "Doing something" = LOW (deliberate, strong).
- This means: **Idle state = 1. Active state = 0.**

Reason 2: Historical standard

- Early transistor-transistor logic (TTL) chips were designed this way. The convention stuck.

> [!WARNING] When you see $\overline{RD} = 0$, it means **"READ IS ACTIVE."** When $\overline{RD} = 1$, it means **"READ IS INACTIVE."**

D.3 The Four Possible Commands

By combining $\overline{M/IO}$ with $\overline{RD}$ and $\overline{WR}$, we get four distinct operations:

Command 1: Memory Read

Signal	Value	Meaning
$\overline{M/IO}$	1	Memory mode
$\overline{RD}$	0	Read active
$\overline{WR}$	1	Write inactive

What happens: CPU reads data from RAM at the address on the Address Bus.

Command 2: Memory Write

Signal	Value	Meaning
$\overline{M/IO}$	1	Memory mode
$\overline{RD}$	1	Read inactive
$\overline{WR}$	0	Write active

What happens: CPU writes data from Data Bus to RAM at the address on the Address Bus.

Command 3: I/O Read

Signal	Value	Meaning
$\overline{M/IO}$	0	I/O mode
$\overline{RD}$	0	Read active
$\overline{WR}$	1	Write inactive

What happens: CPU reads data from an I/O device at the port address on the Address Bus.

Command 4: I/O Write

Signal	Value	Meaning
$\overline{M/IO}$	0	I/O mode
$\overline{RD}$	1	Read inactive
$\overline{WR}$	0	Write active

What happens: CPU writes data from Data Bus to an I/O device at the port address on the Address Bus.

D.4 The Boolean Logic Behind $\overline{IOR}$ — Explained Step by Step

The formula for generating the **I/O Read** signal:

$$\overline{IOR} = \overline{M/IO} + \overline{RD}$$

First, let's understand what this formula is trying to do:

We want a signal called $\overline{IOR}$ (I/O Read Bar) that goes LOW (activates) **ONLY** when:

1. We are in I/O mode ($\overline{M/IO} = 0$).
2. **AND** we want to read ($\overline{RD} = 0$).

So logically: $\overline{IOR}$ should be 0 when ($\overline{M/IO} = 0$) **AND** ($\overline{RD} = 0$).

But the formula uses OR (+), not AND (·). Why?

D.4.1 De Morgan's Law — The Magic Trick **De Morgan's Law** is a fundamental rule in Boolean algebra. It says:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

And the reverse:

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Why does this matter?

Because in hardware, we often build circuits using NAND and NOR gates (which are cheaper and faster than AND/OR gates). De Morgan's Law lets us convert between them.

Now look at our $\overline{IOR}$ formula again:

$$\overline{IOR} = \overline{M/IO} + \overline{RD}$$

Let:

- $A = \overline{M/IO}$ (this is the $\overline{M/IO}$ signal).
- $B = \overline{RD}$ (this is the $\overline{RD}$ signal).

So: $\overline{IOR} = A + B$

But wait — we want IOR to be active (LOW) when BOTH A = 0 and B = 0.

If A = 0 and B = 0, then A + B = 0 + 0 = 0.

So $\overline{IOR} = 0$, which means $\overline{IOR}$ **= 0 → I/O Read fires!**

D.4.2 Complete Truth Table Case 1: I/O Read (Should FIRE)

- $\overline{M/IO} = 0$ (I/O mode).
- $\overline{RD} = 0$ (read active).
- Formula: $\overline{IOR} = 0 + 0 = \mathbf{0}$.
- Result: $\overline{IOR} = \mathbf{0} \rightarrow$ **I/O Read ACTIVATES**

Case 2: Memory Read (Should NOT fire)

- $\overline{M/IO} = 1$ (memory mode).
- $\overline{RD} = 0$ (read active).
- Formula: $\overline{IOR} = 1 + 0 = \mathbf{1}$.
- Result: $\overline{IOR} = \mathbf{1} \rightarrow$ **I/O Read INACTIVE** (Correct — this is a memory read, not I/O.)

Case 3: I/O Write (Should NOT fire)

- $\overline{M/IO} = 0$ (I/O mode).
- $\overline{RD} = 1$ (write active, read inactive).
- Formula: $\overline{IOR} = 0 + 1 = \mathbf{1}$.
- Result: $\overline{IOR} = \mathbf{1} \rightarrow$ **I/O Read INACTIVE** (Correct — this is a write, not a read.)

Case 4: Memory Write (Should NOT fire)

- $\overline{M/IO} = 1$ (memory mode).
- $\overline{RD} = 1$ (write active).
- Formula: $\overline{IOR} = 1 + 1 = \mathbf{1}$.
- Result: $\overline{IOR} = \mathbf{1} \rightarrow$ **I/O Read INACTIVE** (Correct — neither I/O nor read.)

D.4.3 Why This Works (The Deep Explanation) The formula $\overline{IOR} = \overline{M/IO} + \overline{RD}$ is designed so that:

> $\overline{IOR} = \mathbf{0}$ **ONLY when both inputs are 0.**

This is because of how an OR gate works with active-low signals:

Input A	Input B	A OR B
0	0	**0** ← Only case where output is 0
0	1	1
1	0	1
1	1	1

So the OR gate naturally produces 0 only when BOTH inputs are 0.

But here's the clever part: the signals $\overline{M/IO}$ and $\overline{RD}$ are **ALREADY inverted** (they are active-low). So when we say "I/O mode," $\overline{M/IO} = 0$. When we say "read mode," $\overline{RD} = 0$.

The OR gate sees two 0s and outputs 0. Perfect.

> [!IMPORTANT] This is a **"negative-logic AND"** disguised as an OR gate. In positive logic, this would be an AND. In negative logic (where 0 means "yes"), it's an OR. This is exactly what De Morgan's Law describes.

D.4.4 The Other Three Signals (Same Logic) The same pattern generates all four control signals:

Memory Read Signal ($\overline{MEMR}$):

- Should fire when $\overline{M/IO} = 1$ **AND** $\overline{RD} = 0$.
- Formula: $\overline{MEMR} = (\overline{M/IO}) + \overline{RD}$
- Verify: $\overline{M/IO} = 1 \rightarrow \overline{1} = 0$. $\overline{RD} = 0$. So $0 + 0 = 0$. $\overline{MEMR} = 0$. Fires!

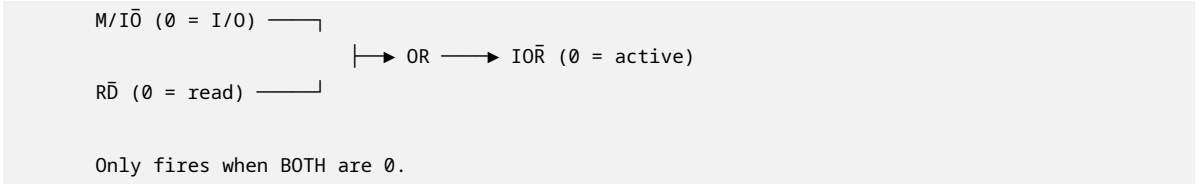
Memory Write Signal ($\overline{MEMW}$):

- Should fire when $\overline{M/IO} = 1$ **AND** $\overline{WR} = 0$.
- Formula: $\overline{MEMW} = (\overline{M/IO}) + \overline{WR}$

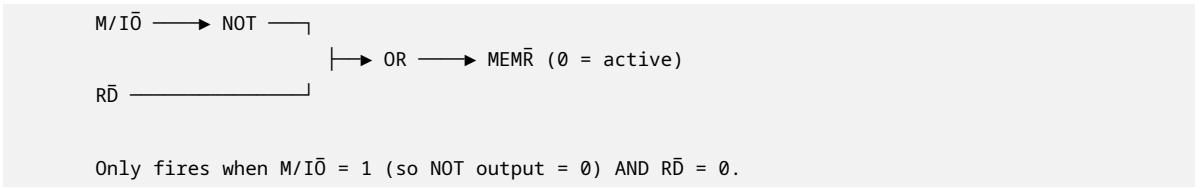
I/O Write Signal ($\overline{IOW}$):

- Should fire when $\overline{M/IO} = 0$ **AND** $\overline{WR} = 0$.
- Formula: $\overline{IOW} = \overline{M/IO} + \overline{WR}$

D.4.5 Visual Circuit Diagram For $\overline{IOR}$ (I/O Read):



For $\overline{MEMR}$ (Memory Read):



Part E: Memory-Mapped I/O (MMIO) — One World, Smart Addressing

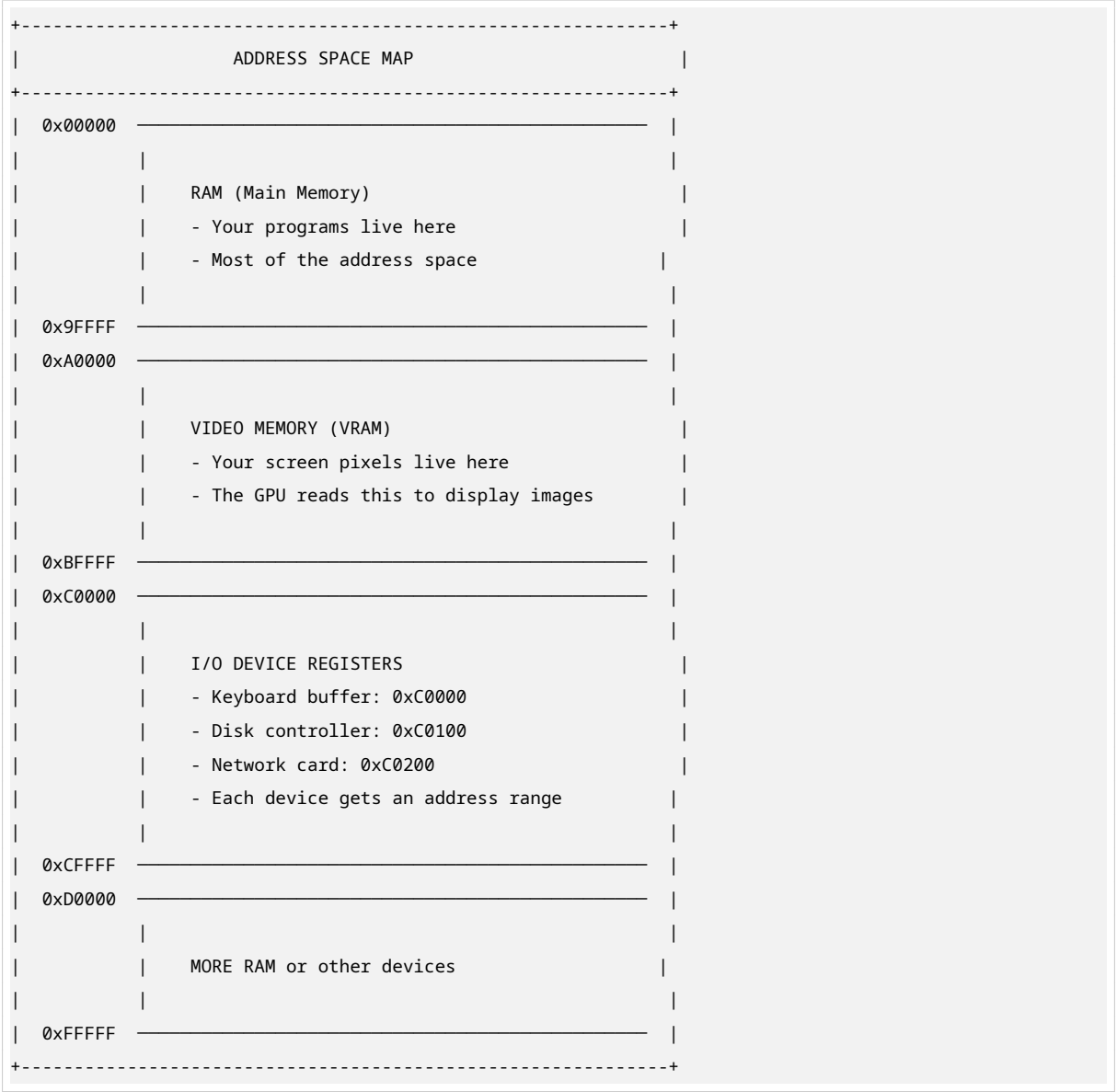
E.1 The Core Idea

In **Memory-Mapped I/O**, there is **NO $\overline{M/IO}$ pin at all.** The CPU only has $\overline{RD}$ and $\overline{WR}$.

So how does the CPU tell memory from I/O?

Answer: BY ADDRESS.

The entire address space (all possible numbers the CPU can put on the Address Bus) is divided into zones:



How it works:

1. CPU wants to read from the keyboard.
2. CPU puts address `0xC0000` on the Address Bus.
3. CPU sets $\overline{RD} = 0$ (read active).
4. The **Memory Controller Hub (MCH)** sees address `0xC0000`.
5. MCH checks: "Is 0xC0000 in the RAM range? No. Is it in the keyboard range? Yes!"
6. MCH activates the keyboard's chip-select line.
7. Keyboard puts the key code on the Data Bus.
8. CPU reads it — just like reading from RAM!

> [!IMPORTANT] **The CPU has NO IDEA it's talking to I/O.** It thinks it's just reading from memory. The MCH handles the routing secretly.

E.2 The Memory Controller Hub (MCH) — The Secret Router

The **MCH** (also called **Northbridge** in older systems, or just **memory controller** in modern CPUs) is a chip that sits between the CPU and everything else.

What the MCH does:

1. Receives the address from the Address Bus.
2. Compares it against pre-programmed address ranges.
3. Decides: RAM? GPU? Keyboard? Disk?
4. Activates the correct device and deactivates all others.

E.3 Base Address Registers (BARs) — How the MCH Knows

A **Base Address Register (BAR)** is a special storage location (usually in the device's own hardware or in the MCH) that stores the starting address of a device's memory range.

Example BAR setup:

BAR for Keyboard:

- Base Address: `0xC0000`
- Size: `0x0100` (256 bytes)
- Range: `0xC0000` to `0xC00FF`
- Meaning: Any address from 0xC0000 to 0xC00FF goes to the keyboard.

BAR for Disk Controller:

- Base Address: `0xC0100`
- Size: `0x0100` (256 bytes)
- Range: `0xC0100` to `0xC01FF`

BAR for Network Card:

- Base Address: `0xC0200`
- Size: `0x0100` (256 bytes)
- Range: `0xC0200` to `0xC02FF`

How are BARs set?

- During system boot, the BIOS/UEFI probes each device and asks: "How much address space do you need?"
- The device responds with its size requirement.
- The BIOS assigns a base address and writes it into the device's BAR.
- From then on, the MCH uses these BARs to route traffic.

E.4 The Comparator Circuit — How the MCH Actually Checks

Inside the MCH, there is a **comparator circuit** for each device. This circuit answers one question: **"Does this address fall within this device's range?"**

How a comparator works (step by step):

Let's say we want to check if address `0xC0050` is in the keyboard's range (`0xC0000` to `0xC00FF`).

Step 1: The MCH has the keyboard's BAR stored: Base = `0xC0000`, Size = `0x0100`.

Step 2: Calculate the end address:

$$\text{End} = \text{Base} + \text{Size} - 1$$

$$\text{End} = 0xC0000 + 0x0100 - 1 = 0xC00FF$$

Step 3: Check two conditions:

- Is Address $\geq$ Base? `0xC0050` $\geq$ `0xC0000`? **YES.**
- Is Address $\leq$ End? `0xC0050` $\leq$ `0xC00FF`? **YES.**

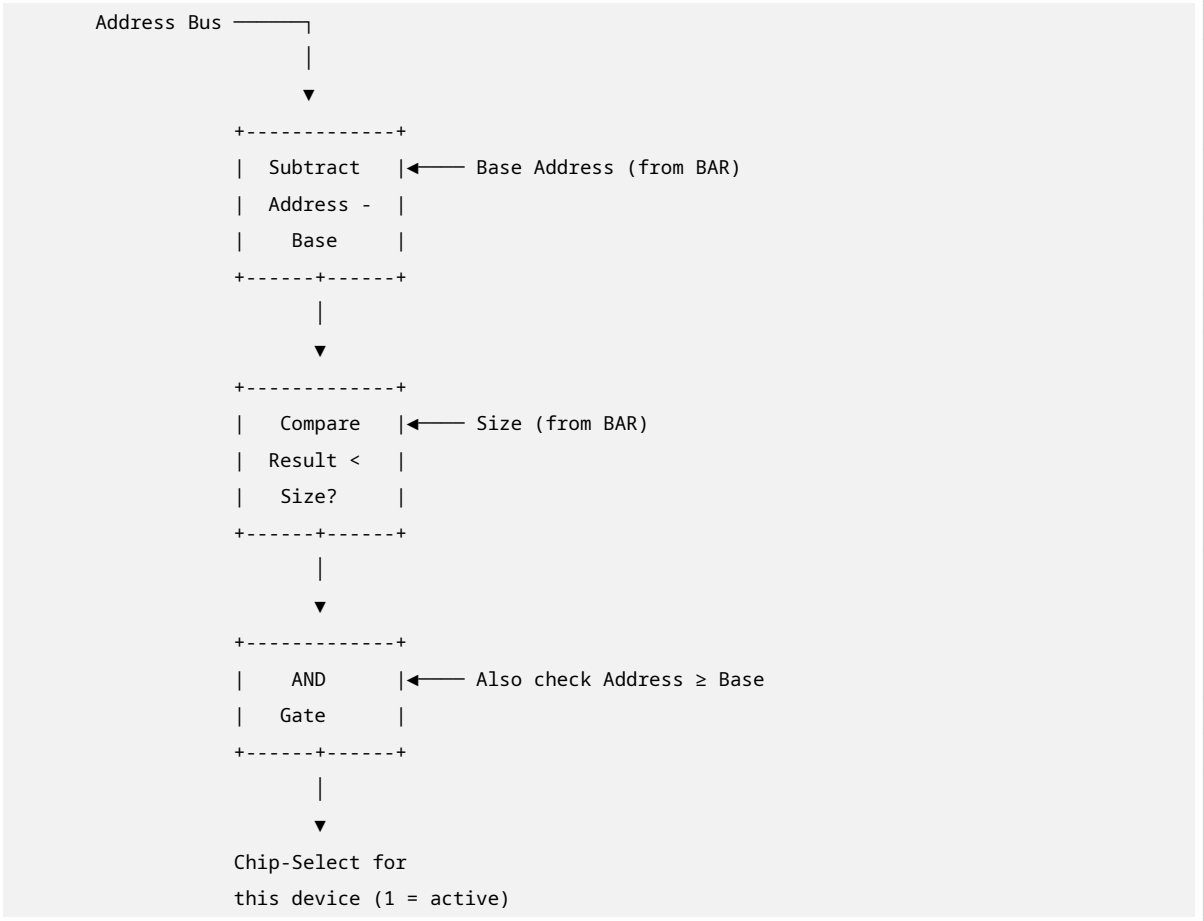
Step 4: If **BOTH** conditions are true, the comparator outputs 1 (match found).

- This 1 activates the keyboard's chip-select line.
- All other devices see 0 on their chip-select and stay quiet.

In hardware, this is done with:

- **Subtractors** to compute Address - Base.
- **Comparators** to check if result < Size.
- **AND gates** to combine the conditions.

Visual diagram of one comparator:



Part F: Isolated I/O vs Memory-Mapped I/O — Complete Comparison

F.1 Side-by-Side Comparison

ISOLATED I/O (Port-Mapped I/O)	MEMORY-MAPPED I/O (MMIO)
• Special CPU pin? YES — M/IO̅ pin exists	• Special CPU pin? NO — no M/IO̅ pin at all
• Separate address space? YES — I/O ports are completely separate from memory addresses	• Separate address space? NO — shares address space with RAM
• Instructions needed? SPECIAL: IN, OUT (e.g., IN AL, 60H)	• Instructions needed? NORMAL: MOV, LOAD, STORE (same as memory ops)
• Who decodes the address? Control Unit via M/IO̅ pin	• Who decodes the address? MCH / Memory Controller with BARs
• Max I/O addresses SMALL — limited by architecture (e.g., 65536 ports on x86)	• Max I/O addresses HUGE — limited only by address bus width (64-bit = 18 quintillion)
• Used in Classic x86 (Intel 8086, 8088, early PCs)	• Used in Modern systems, ARM, RISC-V, modern x86-64, GPU VRAM access
• Speed Fast — dedicated pins, no address decoding	• Speed Fast — but slightly more overhead for decoding
• Programming complexity HARDER — must use special I/O instructions	• Programming complexity EASIER — use normal memory instructions, compiler can optimize automatically

F.2 When to Use Which? (Practical Scenarios)

[!TIP] **Use Isolated I/O when:**

- You have a simple 8-bit or 16-bit microcontroller.
- You need guaranteed fast I/O with no decoding delay.
- You have a small, fixed set of I/O devices.
- Example: Intel 8086 systems, some embedded systems.

[!TIP] **Use Memory-Mapped I/O when:**

- You have a modern 32-bit or 64-bit system.
- You have many devices with large address ranges (like GPUs with hundreds of MB of VRAM).
- You want to use the same instructions and caching for I/O as for memory.
- You want the compiler to optimize I/O access.
- Example: ARM processors, modern x86-64, smartphones, game consoles.

Part G: What Your Tutorial Was Missing — Added Here

G.1 Why "Active Low" Is Physically Necessary

The electrical reason:

In transistor circuits (TTL logic):

- A HIGH signal (logic 1) is created by pulling the wire UP to +5V (or +3.3V) through a resistor.
- A LOW signal (logic 0) is created by pulling the wire DOWN to 0V (ground) through a transistor.

The problem with HIGH as active:

- If the wire is floating (not connected to anything), electrical noise can make it look like HIGH.
- A floating wire might accidentally trigger a "read" command — disaster.
- Also, pulling UP requires current through a resistor, which consumes power.

The advantage of LOW as active:

- Ground (0V) is very stable. It's hard for noise to make ground look like +5V.
- A floating wire naturally drifts toward HIGH (through the pull-up resistor), which means "inactive" — safe.
- When we want to activate, we use a transistor to **STRONGLY** pull to ground. This is reliable.
- **Result:** Idle = HIGH (safe, no power wasted). Active = LOW (strong, reliable).

[!IMPORTANT] **This is why every control signal in hardware has a bar on top.** $\overline{RD}$, $\overline{WR}$, $\overline{IOR}$, $\overline{MEMR}$ — all active LOW.

G.2 What the MCH Comparator Actually Does (Gate-Level)

To check if Address is in range [Base, Base + Size):

The hardware uses this logic:

1. Compute `Address - Base`.
2. Check if `Address - Base < Size`.
3. Also check `Address $\geq$ Base` (handles wrap-around).

At the gate level:

```
Address bits: A15 A14 A13 ... A0
Base bits:    B15 B14 B13 ... B0 (from BAR)

For each bit position i:

Xi = Ai XNOR Bi  (XNOR = 1 when both bits are equal)

Match = X15 AND X14 AND X13 AND ... AND X0

If Match = 1, then Address == Base (exact match)

For range checking, we use a subtractor and comparator:

Difference = Address - Base

In_range = (Difference < Size) AND (Address ≥ Base)
```

XNOR Gate Truth Table:

A	B	A XNOR B
0	0	**1** (match!)
0	1	0
1	0	0
1	1	**1** (match!)

> [!NOTE] The MCH has a chain of XNOR gates comparing each address bit to the stored BAR bits. If ALL bits match, the chip-select fires.

G.3 Why MMIO Won in Modern Systems

Historical context:

- Intel 8086 (1978) used Isolated I/O with a 16-bit address bus.
- I/O ports were limited to 0–65535 (2^{16}).
- This was fine when you had a keyboard, a serial port, and a floppy disk.

Modern reality:

- A modern GPU has 8 GB, 16 GB, or 24 GB of VRAM.
- A modern NVMe SSD controller needs megabytes of register space.
- USB controllers, Thunderbolt, network cards — all need large address ranges.
- 64-bit address space = 18,446,744,073,709,551,616 possible addresses.
- Giving a GPU 16 GB out of 18 quintillion is like giving someone a drop from the ocean.

Technical advantages of MMIO:

1. **Unified instructions:** Use `MOV`, `LOAD`, `STORE` for everything. No special `IN/OUT` instructions needed.
2. **Compiler optimization:** The compiler treats I/O like memory. It can reorder, cache, and optimize I/O accesses using the same rules as memory.

3. **Caching:** MMIO regions can be cached (carefully) to improve performance.
4. **Large address space:** No artificial limit on I/O device size.
5. **Simpler CPU design:** No need for $\overline{M/IO}$ pin, no need for separate I/O instruction decode logic.

> [!IMPORTANT] **Result:** Modern ARM, RISC-V, and x86-64 systems use MMIO exclusively. Isolated I/O exists only for backward compatibility in x86 systems (the `IN/OUT` instructions still work but are rarely used).

Part H: Complete Example Walkthrough

H.1 Example: Reading a Key Press in Isolated I/O

System: Intel 8086, Isolated I/O
Task: Read which key was pressed from the keyboard
Keyboard I/O Port: `60H` (hexadecimal 60)

Step 1 → CPU decodes instruction `IN AL, 60H`

- This is an I/O read instruction.
- Port number = 60H = 96 in decimal.
- CPU knows: "I need to read from I/O port 96."

Step 2 → CPU sets $\overline{M/IO} = 0$

- Control Unit generates this signal.
- All devices on bus see: "This is an I/O operation, not memory."

Step 3 → CPU puts port address on Address Bus

- Address Bus = `0000000001100000` (binary for 60H).
- Note: In Isolated I/O, the address bus carries the PORT NUMBER, not a memory address.

Step 4 → CPU sets $\overline{RD} = 0$

- Read command is active.
- $\overline{WR}$ stays at 1 (inactive).

Step 5 → $\overline{IOR}$ signal is generated

- $\overline{M/IO} = 0$, $\overline{RD} = 0$.
- $\overline{IOR} = 0 + 0 = 0 \rightarrow$ I/O Read fires!
- Keyboard sees $\overline{IOR} = 0$ and recognizes: "CPU wants to read from me!"

Step 6 → Keyboard puts scan code on Data Bus

- Keyboard has been storing the last key pressed.
- It puts the scan code (e.g., `1C` for 'A' key) on the 8-bit Data Bus.

Step 7 → CPU reads Data Bus into AL register

- CPU latches the value from Data Bus.
- AL register now contains `1C`.
- CPU can now process this key press.

Step 8 → Control signals return to idle

- $\overline{RD}$ goes back to 1.

- $\overline{IOR}$ goes back to 1.
- Bus is free for next operation.

H.2 Example: Reading a Key Press in Memory-Mapped I/O

System: ARM processor, MMIO
Task: Read which key was pressed from the keyboard
Keyboard Memory Address: `0xFFFF0000` (assigned by BIOS)

Step 1 → CPU executes `LDRB R0, [R1]`

- R1 contains the address `0xFFFF0000`.
- `LDRB` = Load Register Byte (read one byte from memory).
- CPU thinks: "I need to read from memory address 0xFFFF0000."

Step 2 → CPU puts address on Address Bus

- Address Bus = `11111111111111110000000000000000` (binary for 0xFFFF0000).
- No $\overline{M/IO}$ pin exists. CPU doesn't know or care if this is RAM or I/O.

Step 3 → CPU sets $\overline{RD} = 0$

- Read command is active.

Step 4 → MCH receives the address

- MCH checks: "Is 0xFFFF0000 in RAM range?"
- RAM ends at 0x9FFFFF. No.
- MCH checks BARs: "Keyboard BAR = 0xFFFF0000 to 0xFFFF00FF."
- Match! 0xFFFF0000 is in keyboard range.

Step 5 → MCH activates keyboard chip-select

- Keyboard receives: "You are selected. CPU wants to read."
- All other devices are deactivated.

Step 6 → Keyboard puts scan code on Data Bus

- Same as before: scan code `1C` appears on Data Bus.

Step 7 → CPU reads Data Bus into R0

- R0 register now contains `1C`.
- CPU is happy — it got its "memory read" result.

Step 8 → MCH deactivates keyboard

- Operation complete. Bus free.

> [!IMPORTANT] **Key difference:** The CPU had **NO IDEA** it was talking to I/O. It executed a normal memory read. The MCH did all the magic routing invisibly.

Part I: Key Points to Remember

- **I/O Problem:** CPU has one bus for everything. Must distinguish RAM from I/O devices.
- **Three Buses:** Address (where), Data (what), Control (how).

- **Isolated I/O:** Uses $\overline{M/IO}$ pin to switch between two worlds. Needs special `IN/OUT` instructions. Limited address space.
- **MMIO:** No $\overline{M/IO}$ pin. Uses address ranges to route traffic. Normal memory instructions work. MCH with BARs does the routing.
- **Active Low:** Control signals fire when 0, not 1. More reliable electrically. Bars on top ($\overline{RD}$, $\overline{WR}$) mean active low.
- **De Morgan's Law:** $\overline{A \cdot B} = \overline{A} + \overline{B}$. Explains why OR gates can implement AND logic in active-low systems.
- **BAR:** Base Address Register. Stores where a device's memory range starts. Set at boot time by BIOS.
- **MCH:** Memory Controller Hub. The chip that compares addresses against BARs and routes signals to the right device.
- **Modern systems use MMIO exclusively.** Isolated I/O is legacy.

Part J: Math Practice (Complete Solutions)

J.1 Convert I/O Port 60H to Binary

Given: Port number = 60H (hexadecimal)

Step 1: Write each hex digit as 4 bits.

- 6 in hex = `0110` in binary.
- 0 in hex = `0000` in binary.

Step 2: Combine them.

- 60H = `0110 0000` in binary.

Step 3: For a 16-bit address bus, pad with leading zeros.

- 60H on 16-bit bus = `0000000001100000`.

Answer: `0000000001100000` (binary) = 96 (decimal).

J.2 Calculate Address Range from BAR

Given:

- Base Address = `0xC0000`
- Size = `0x0100` (256 bytes)

Step 1: End Address = Base + Size − 1.

$$\text{End} = 0xC0000 + 0x0100 - 1 = 0xC0100 - 1 = 0xC00FF$$

Step 2: Verify in decimal.

- 0xC0000 = 786,432
- 0x0100 = 256
- 0xC00FF = 786,432 + 256 − 1 = 786,687

Answer: Range is `0xC0000` to `0xC00FF` (786,432 to 786,687 in decimal).

J.3 Verify $\overline{IOR}$ Formula with Different Values

Given: $\overline{IOR} = \overline{M/IO} + \overline{RD}$

Test Case: $\overline{M/IO} = 0$, $\overline{RD} = 1$

Step 1: Substitute values.

- $\overline{IOR} = 0 + 1$

Step 2: OR gate truth table.

- 0 OR 1 = 1

Step 3: Interpret result.

- $\overline{IOR} = 1 \rightarrow$ Inactive $\rightarrow$ I/O Read does **NOT** fire.

Conclusion: Correct! $\overline{RD} = 1$ means "not reading," so I/O Read should not happen.

Part K: Common Mistakes to Avoid

[!WARNING] **Mistake 1:** Thinking $\overline{M/IO} = 0$ means "memory."

- **Reality:** $\overline{M/IO}$ = 0 means I/O. $\overline{M/IO}$ = 1 means memory. The bar is only over the IO part, but the whole signal is active-low for I/O.

[!WARNING] **Mistake 2:** Confusing I/O port numbers with memory addresses.

- **Reality:** In Isolated I/O, port 60H and memory address 60H are completely different places. The $\overline{M/IO}$ pin distinguishes them.

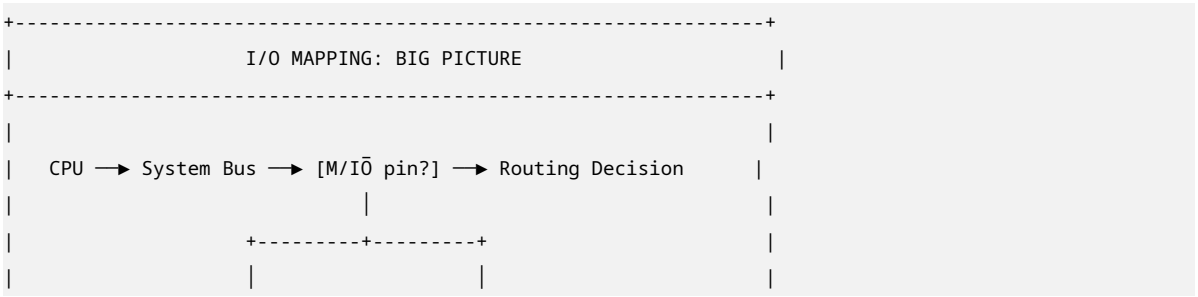
[!WARNING] **Mistake 3:** Thinking MMIO uses special I/O instructions.

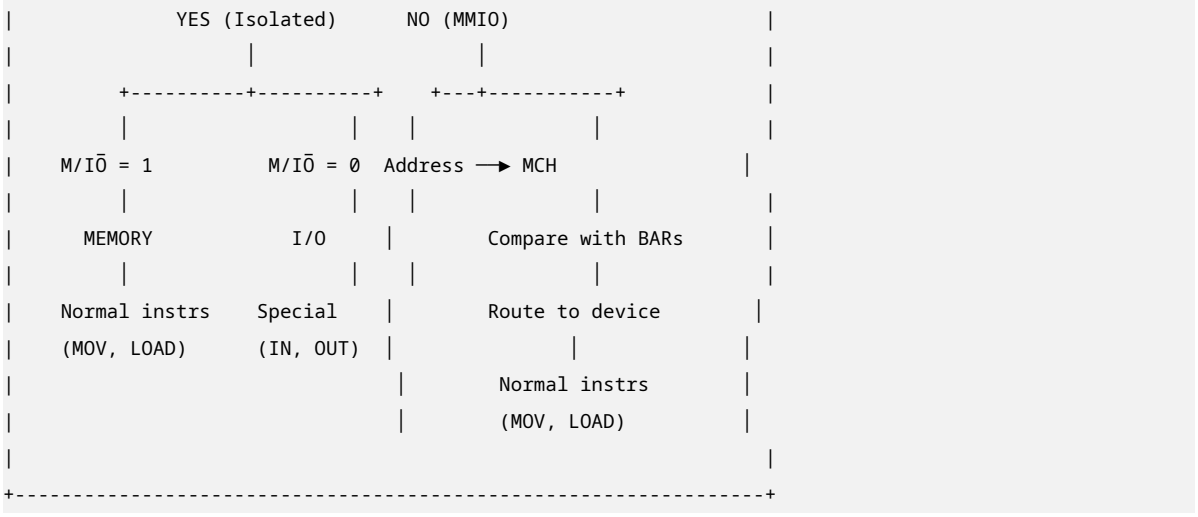
- **Reality:** MMIO uses **NORMAL** memory instructions (`MOV`, `LDR`, etc.). The hardware routing is invisible to the CPU.

[!WARNING] **Mistake 4:** Forgetting that active-low means 0 = ON.

- **Reality:** $\overline{RD}$ = 0 means "read is happening." $\overline{RD}$ = 1 means "read is NOT happening."

Part L: Diagram Summary (One-Page Review)





This completes the Section 1 research package on I/O Mapping. Every concept is explained from first principles, every formula is solved step-by-step, and every diagram is preserved for reference.

Section 2 — Register Transfer Level (RTL) & Micro-Operations

Part A: What Even Is RTL?

A.1 The Problem RTL Solves

You have a CPU. It executes instructions like `MOV AX, BX` or `ADD R1, R2`. But the CPU does NOT do these in one magical step. It breaks every instruction into **tiny, atomic steps** that happen one after another, each on a clock tick.

RTL (Register Transfer Level) is the language we use to write down exactly what moves where during each of those tiny steps.

The arrow symbol ← is EVERYTHING:

```
Destination ← Source
```

This means: **"Copy the value from Source into Destination. Leave Source unchanged."**

It is NOT:

- An equals sign (we are not solving an equation).
- A "move" that deletes the source (the source keeps its value).
- A suggestion (this is a physical wire transfer happening in hardware).

Analogy: Imagine you have two buckets, A and B. Bucket A has 5 liters of water. You pour bucket A into bucket B. Now both buckets have 5 liters. That is what ← means. A still has 5. B now has 5.

A.2 What Is a Micro-Operation (µ-op)?

A **micro-operation** is one single, indivisible action that happens inside the CPU in one clock cycle.

Examples of micro-operations:

- Copy PC into MAR.
- Read memory and put result in MBR.
- Add two numbers in the ALU.
- Increment PC by 1.

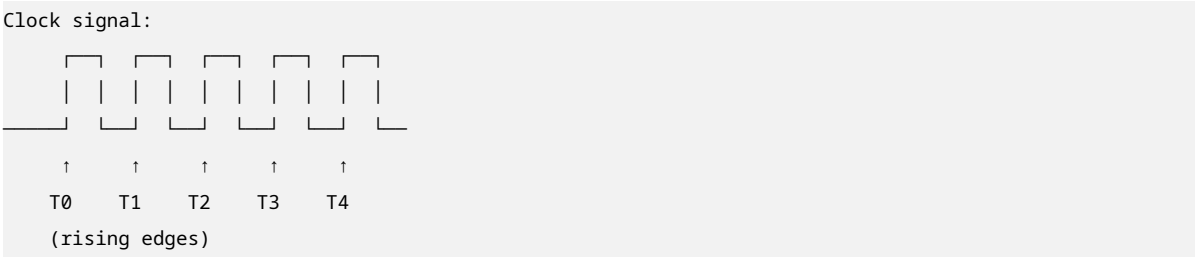
Every CPU instruction (like `IN AL, 60H`) is broken into 4–6 micro-operations. The Control Unit has a tiny ROM inside it called the **microcode memory** that stores the sequence of µ-ops for each instruction.

Why do we need µ-ops?

- The CPU's internal wiring is fixed. You cannot magically "execute an IN instruction" in one step.
- You must: fetch the instruction, decode it, set up the address, assert the right control signals, read the data, store the result.
- Each of these needs its own clock cycle because wires take time to settle, and you cannot do everything at once.

A.3 The Clock — Why Everything Happens in Steps

The **clock** is a square wave signal that oscillates between 0 and 1 at a fixed frequency.



Key rule: All register transfers happen on the **rising edge** of the clock (when the signal goes from 0 to 1). Between rising edges, the wires settle and signals propagate.

At 3 GHz (modern CPU):

- One clock cycle = 1 / 3,000,000,000 seconds = **0.333 nanoseconds**.
- That is 333 picoseconds. Light travels about 10 centimeters in that time.

Why does this matter? Because when we say "T4 can stall for thousands of cycles," we mean the CPU is frozen for **microseconds** — an eternity at this speed.

Part B: The Key Registers

Before we trace any instruction, you must know these registers exist inside the CPU. They are physical hardware — flip-flop circuits etched into silicon.

B.1 PC — Program Counter

- **What it holds:** The memory address of the NEXT instruction to fetch.
- **Size:** Same width as the address bus (16-bit on 8086, 32-bit on x86, 64-bit on modern CPUs).
- **What it does:** After every instruction fetch, it increments to point to the next instruction.
- **Analogy:** PC is like your finger pointing at the next line in a recipe book. It always stays one line ahead of what you are currently doing.
- **Important:** PC does NOT hold the instruction itself. It holds the ADDRESS where the instruction lives in RAM.

B.2 MAR — Memory Address Register

- **What it holds:** The address of the memory location (or I/O port) the CPU wants to access RIGHT NOW.
- **What it does:** The memory system watches this register. Whatever address appears here, memory will try to read from or write to that location.
- **Analogy:** MAR is like telling the librarian: "I want the book at shelf 5, row 3." The librarian goes and gets it. MAR is your request slip.

- **Key point:** MAR is loaded BEFORE any memory operation. It is the setup step.

B.3 MBR — Memory Buffer Register

- **What it holds:** The actual data bytes being transferred to or from memory.
- **What it does:** It is the "holding area" between CPU and memory.
  - When READING: Data from memory lands here first, then moves to its final destination.
  - When WRITING: Data is placed here first, then memory copies it from here.
- **Analogy:** MBR is like a loading dock. Trucks (data) come in and out through here. Nothing goes directly from the warehouse (RAM) to the office (CPU) without passing through the dock.
- **Key point:** MBR is always involved in memory transfers. Always.

B.4 IR — Instruction Register

- **What it holds:** The actual instruction bytes that were just fetched from memory.
- **What it does:** Once an instruction is in IR, the Control Unit reads the **opcode** (the first few bits) and figures out what instruction this is and what µ-ops to execute.
- **Analogy:** IR is like the chef reading the recipe card. The recipe was fetched from the book (RAM), but now it is in the chef's hands (IR) being studied.
- **Key point:** No instruction is ever executed directly from RAM. It MUST pass through IR first.

B.5 AL — Accumulator Low (x86-Specific)

- **What it holds:** An 8-bit general-purpose register in x86 architecture.
- **What it does:** Stores results of operations. In the `IN` instruction, it receives the byte read from the I/O port.
- **Analogy:** AL is like a notepad the CPU uses for quick calculations and temporary storage.
- **Note:** In 16-bit mode, AX = AH (high byte) + AL (low byte). In 32-bit, EAX extends AX. In 64-bit, RAX extends EAX.

B.6 R0, R1 — General-Purpose Registers

- **What they hold:** Whatever the programmer or operating system puts in them.
- **What they do:** Hold addresses, data, counters, pointers — anything.
- **In our MMIO example:** R0 holds the MMIO address of a device. R1 will hold the result after the read.
- **Analogy:** These are like blank index cards you can write anything on.

B.7 Register Relationships (Visual Diagram)

```
+-------------------------------------------------------------+
|                          CPU                                |
|  +----------+  +----------+  +----------+  +----------+     |
```

```
|  |   PC    |  |   MAR   |  |   MBR   |  |   IR    |         |
|  | (points |  | (address|  | (data   |  | (current|         |
|  |  ahead) |  |  to use)|  |  buffer)|  |  instr) |         |
|  +----+----+  +----+----+  +----+----+  +----+----+         |
|       |           |           |           |              |
|       +-----------+-----------+-----------+              |
|                   Internal Bus                            |
|                        |                                  |
|              +----------+----------+                       |
|              |    Control Unit    |                      |
|              |  (decodes IR, makes |                       |
|              |   control signals)  |                       |
|              +--------------------+                      |
|                        |                                  |
|              +----------+----------+                       |
|              |      System Bus      |                       |
|              |  (Address + Data +   |                       |
|              |      Control)        |                       |
|              +--------------------+                      |
+-----------------------------------------------------------+
```

Part C: Isolated I/O — `IN AL, PORT`

C.1 What the Instruction Means

```
IN AL, PORT
```

Plain English: "Read one byte from I/O port number PORT, and store that byte in the AL register."

PORT is an 8-bit number that is **hardcoded into the instruction itself.** It is not a variable. It is not in a register. It is literally part of the instruction bytes.

Example: `IN AL, 60H`

- Opcode byte tells CPU: "This is an IN instruction."
- Next byte is `60H` = 96 in decimal = the keyboard port.
- Result: The byte from keyboard port 60H goes into AL.

C.2 The Five Clock Cycles (T0 through T4)

Cycle T0 — Fetch Phase: Set Up the Instruction Address

```
T0:  MAR ← PC
```

What just happened:

- The CPU copies the value in PC into MAR.
- PC holds the address of the next instruction (say, `0x0100`).

- MAR now holds `0x0100`.

What this means physically:

- The address `0x0100` appears on the Address Bus.
- Memory sees this address and prepares to respond.

What has NOT happened yet:

- The instruction has NOT been read.
- No data has moved.
- This is purely setup.

Analogy: You walk into a library and tell the librarian the call number of the book you want. The librarian has not fetched the book yet. You have just made the request.

Why PC goes into MAR and not directly to memory:

- PC is inside the CPU. Memory is outside. The CPU cannot "show" PC to memory directly.
- MAR is the official "window" through which the CPU shows addresses to the outside world.
- Every memory access starts with MAR being loaded.

Cycle T1 — Fetch Phase: Read the Instruction from Memory

```
T1:  MBR ← Memory[MAR]
     PC  ← PC + 1
```

What just happened (two things at once):

Thing 1: `MBR ← Memory[MAR]`

- Memory looks at the address in MAR (`0x0100`).
- Memory reads the bytes stored at that address.
- Those bytes (the instruction `IN AL, 60H`) travel across the Data Bus into MBR.
- MBR now holds the raw instruction bytes.

Thing 2: `PC ← PC + 1`

- PC increments by 1 (or by the instruction length, depending on architecture).
- PC now points to the NEXT instruction (`0x0101` or `0x0102`).
- This happens in parallel with the memory read.

Why can two things happen at once?

- These are independent operations.
- The memory read uses the Address Bus and Data Bus.
- The PC increment happens entirely inside the CPU (no bus needed).
- The same clock edge triggers both.

Control Bus signals during T1:

- $\overline{RD} = \mathbf{0}$ (read active — we want data FROM memory).
- $\overline{M/IO} = \mathbf{1}$ (memory mode — we are reading from RAM, not I/O).
- $\overline{WR} = \mathbf{1}$ (write inactive).

Analogy: The librarian fetches your book (MBR gets the instruction) while simultaneously, you look up the next book you want (PC increments). Both happen at the same time.

Cycle T2 — Decode Phase: Figure Out What the Instruction Means

```
T2:  IR ← MBR
```

What just happened:

- The instruction bytes move from MBR into IR.
- IR now holds the complete instruction: `IN AL, 60H`.

What the Control Unit does:

- The Control Unit reads the **opcode** (the first byte of the instruction).
- It looks up in its internal ROM: "Opcode = E4H means IN instruction."
- It finds the micro-operation sequence for IN.
- It prepares the control signals that will be needed for T3 and T4.

What does NOT happen:

- No data moves to or from memory or I/O.
- No addresses go on the bus.
- This is purely internal CPU housekeeping.

Analogy: You got the recipe book (MBR → IR). Now you are reading the recipe to see what ingredients you need and what steps to follow. You are not cooking yet. You are planning.

The Control Unit as a Look-Up Table:

```
+----------------------------------------+
|         CONTROL UNIT INTERNAL ROM        |
|                                          |
|  Opcode Bits ──► |  μ-op Sequence      |
|                  |  for this instr     |
|                  |                     |
|  E4H (IN)    ──► |  T3: MAR←IR[port]   |
|                  |  T4: AL←Bus_Data    |
|                  |                     |
|  A1H (MOV)   ──► |  T3: ...            |
|                  |  T4: ...            |
|                  |                     |
|  05H (ADD)   ──► |  T3: ...            |
|                  |  T4: ...            |
|                  |                     |
+----------------------------------------+
```

Cycle T3 — Execute Phase: Set Up the I/O Port Address

```
T3:  MAR ← IR[port_address]
```

What just happened:

- The Control Unit extracts the port number from the instruction in IR.
- In `IN AL, 60H`, the port number is `60H`.
- This port number is loaded into MAR.

Critical difference from T0:

- T0 loaded the **instruction address** into MAR (to fetch the instruction from RAM).
- T3 loads the **I/O port number** into MAR (to read from the I/O device).

Simultaneously, the Control Unit asserts two physical pins:

$\overline{M/IO} = \mathbf{0}$

- This is a REAL voltage change on a REAL wire leaving the CPU chip.
- Pin 28 on the Intel 8086 package, to be exact.
- Every device on the system bus sees this pin go LOW.
- They all know: "The CPU is in I/O mode now. Memory should ignore this."

$\overline{RD} = \mathbf{0}$

- The read command is active.
- Combined with $\overline{M/IO} = 0$, this generates the $\overline{IOR}$ signal (from Section 1).

The OR gate logic from Section 1:

```
IOR̄ = M/IO̅ + R̄D̄ = 0 + 0 = 0 → I/O Read ACTIVE!
```

What happens on the bus:

- Address Bus shows `0000000001100000` (60H in binary, 16-bit).
- Control Bus shows $\overline{M/IO} = 0$, $\overline{RD} = 0$, $\overline{IOR} = 0$.
- Every I/O device checks: "Is my address 60H?"
- Only the keyboard controller (at port 60H) responds. All others ignore.

Analogy: You tell the librarian: "I want book number 60 from the I/O section, not the regular shelves." The librarian flips a sign saying "I/O section" ($\overline{M/IO} = 0$) and asks for book 60 ($\overline{RD} = 0$). Only the I/O shelf responds.

Cycle T4 — Execute Phase: Read the Data from the Device

```
T4:  AL ← Bus_Data
```

What just happened:

- The keyboard controller sees $\overline{IOR} = 0$ and address 60H on the bus.
- It drives 8 bits of data onto the Data Bus.
- Example: If you pressed the 'A' key, it puts `1CH` (scan code for 'A') on the bus.
- The CPU reads these 8 bits from the Data Bus.
- The CPU stores them in the AL register.

Who is driving the bus?

- **NOT the CPU.** The CPU is passive here. It is LISTENING.

- The I/O device (keyboard controller) is the one putting data on the Data Bus.
- The CPU just captures it.

What happens to the control signals after T4:

- $\overline{M/IO}$ returns to 1 (back to memory mode).
- $\overline{RD}$ returns to 1 (read inactive).
- $\overline{IOR}$ returns to 1 (inactive).
- The bus is now free for the next operation.

Instruction complete! AL now holds the key scan code.

C.3 Complete T0–T4 Summary for `IN AL, 60H`

Cycle	Micro-Operation	What It Means
T0	`MAR ← PC`	Set up instruction address
T1	`MBR ← Memory[MAR]`	Fetch instruction bytes
	`PC ← PC + 1`	Point to next instruction
T2	`IR ← MBR`	Decode the instruction
T3	`MAR ← IR[port_address]`	Set up I/O port address
	$\overline{M/IO} = 0$, $\overline{RD} = 0$	Switch to I/O mode, read
T4	`AL ← Bus_Data`	Read data from device
	$\overline{M/IO} = 1$, $\overline{RD} = 1$	Return to idle state

---

Part D: Memory-Mapped I/O — `LOAD R1, [R0]`

D.1 What the Instruction Means

```
LOAD R1, [R0]
```

Plain English: "Read the value at the memory address stored in register R0, and put that value into register R1."

The brackets `[ ]` mean "the contents of the memory location whose address is in R0."

Example scenario:

- R0 contains `0xFFFF0000` (the MMIO address of a network card's receive buffer).
- The CPU does NOT know this is a device address. It thinks R0 contains a normal RAM address.
- The MCH will intercept and route to the network card.

D.2 The Six Clock Cycles (T0 through T5)

Cycle T0 — Fetch Phase: Set Up the Instruction Address

```
T0:  MAR ← PC
```

Identical to Isolated I/O T0.

- PC holds the address of the `LOAD` instruction.
- PC is copied into MAR.
- Address appears on Address Bus.
- Memory prepares to respond.

Why identical? Because the CPU cannot possibly know what instruction is coming until it fetches it. Every instruction, regardless of type, starts with `MAR ← PC`.

Cycle T1 — Fetch Phase: Read the Instruction from Memory

```
T1:  MBR ← Memory[MAR]
     PC  ← PC + 1
```

Identical to Isolated I/O T1.

- The `LOAD R1, [R0]` instruction bytes arrive from RAM into MBR.
- PC increments to point to the next instruction.

Still no difference from Isolated I/O. The CPU is just fetching an instruction from RAM. It has no idea what type of instruction this is yet.

Control Bus signals:

- $\overline{RD} = 0$, $\overline{M/IO} = 1$ (memory read, standard fetch).

Cycle T2 — Decode Phase: Figure Out What the Instruction Means

```
T2:  IR ← MBR
```

Identical pattern to Isolated I/O T2.

- Instruction bytes move into IR.
- Control Unit decodes the opcode.
- It recognizes: "This is a LOAD instruction. Source address is in R0. Destination is R1."
- It prepares control signals for execute.

Still no difference. The decode phase is the same for all instructions.

Cycle T3 — Execute Phase: Set Up the Memory Address

```
T3:  MAR ← R0
```

What just happened:

- The 32-bit value in R0 (`0xFFFF0000`) is copied into MAR.

- MAR now holds the MMIO address of the network card.

Control Unit asserts:

- $\overline{RD} = \mathbf{0}$ (read active).
- **NO $\overline{M/IO}$ signal!** There is no $\overline{M/IO}$ pin in MMIO systems.

This is the KEY DIFFERENCE from Isolated I/O:

- In Isolated I/O, the CPU explicitly says "I/O mode" with $\overline{M/IO} = 0$.
- In MMIO, the CPU fires a completely standard memory read.
- The CPU is entirely unaware that R0 contains a device address.

Analogy: You tell the librarian: "I want the book at address 0xFFFF0000." You do not say "this is a special I/O book." You treat it like any other book request. The librarian (MCH) will figure out where it actually is.

Cycle T4 — Execute Phase: The Hidden Stall (The Trap)

```
T4:  MCH intercepts address in MAR → reroutes to PCIe bus
```

What happens:

Step 1: The MCH watches the Address Bus.

- It sees `0xFFFF0000` in MAR.
- It checks its internal BAR table.

Step 2: MCH finds a match.

- BAR for Network Card: Base = `0xFFFF0000`, Size = `0x1000`.
- `0xFFFF0000` falls within this range.
- MCH decides: "This is NOT RAM. This is the network card."

Step 3: MCH reroutes the request.

- Instead of sending the read to DDR5 RAM, it sends it over the PCIe bus to the network card.
- The CPU is completely unaware of this rerouting.

Step 4: The CPU stalls.

- The CPU asserted $\overline{RD} = 0$ and is now waiting for data on the Data Bus.
- It has no idea whether it is waiting for RAM or a PCIe device.
- **It just waits.**

The latency problem:

Source	Response Time
DDR5 RAM	~70–100 nanoseconds
PCIe device round-trip	~1,000–10,000 nanoseconds
CPU clock at 3 GHz	~0.333 nanoseconds per cycle

How many cycles does T4 take?

- Fast case (PCIe): 1,000 ns / 0.333 ns = ~**3,000 cycles**.
- Slow case (PCIe): 10,000 ns / 0.333 ns = ~**30,000 cycles**.

The CPU is frozen for THOUSANDS of cycles doing NOTHING.

Analogy: You ask the librarian for a book. The librarian realizes it is not in this library — it is in a library across the country. The librarian sends a fax (PCIe request) and waits for the book to be mailed back. You stand at the counter frozen, unable to do anything else, while the librarian waits.

Cycle T5 — Execute Phase: Receive the Data

```
T5:  R1 ← Bus_Data
```

What just happened:

- The network card finally responds.
- It puts the data onto the Data Bus.
- The CPU reads the data and stores it in R1.
- The instruction is complete.

Total cycles: T0 through T5 = **6 cycles minimum** (plus the thousands of stall cycles hidden inside T4).

D.3 Complete T0–T5 Summary for `LOAD R1, [R0]`

Cycle	Micro-Operation	What It Means
T0	`MAR ← PC`	Set up instruction address
T1	`MBR ← Memory[MAR]`	Fetch instruction bytes
	`PC ← PC + 1`	Point to next instruction
T2	`IR ← MBR`	Decode the instruction
T3	`MAR ← R0`	Set up device address
	$\overline{RD} = 0$	Standard memory read
T4	MCH intercepts & reroutes	Hidden stall to PCIe
	(thousands of cycles)	CPU frozen waiting
T5	`R1 ← Bus_Data`	Read data from device
	$\overline{RD} = 1$	Return to idle state

Part E: Why the "CPU Is Blind" Matters

E.1 The Core Insight

In MMIO, the CPU generates **the exact same bus signals** whether it is reading RAM or a device register.

The routing decision is made entirely by the MCH hardware, invisible to the CPU's Control Unit.

This creates three major engineering problems:

E.2 Problem 1: Unpredictable Stall Duration

What happens:

- The CPU cannot predict how long T4 will take.
- It does not know if it is waiting for RAM (fast, ~100 ns) or a PCIe device (slow, ~10,000 ns).
- The CPU's pipeline has no way to plan for this.

The solution: Out-of-Order Execution

Modern CPUs use **out-of-order execution (OOE)** to solve this:

Normal (in-order):

```
Instr 1: LOAD R1, [R0]     ⟶ STALL ⟶ complete
Instr 2: ADD R2, R3, R4    ⟶ waits for Instr 1
Instr 3: SUB R5, R6, R7    ⟶ waits for Instr 1
Instr 4: MUL R8, R9, R10   ⟶ waits for Instr 1
```

Out-of-order:

```
Instr 1: LOAD R1, [R0]     ⟶ STALL (set aside)
Instr 2: ADD R2, R3, R4    ⟶ execute now (doesn't need R1)
Instr 3: SUB R5, R6, R7    ⟶ execute now (doesn't need R1)
Instr 4: MUL R8, R9, R10   ⟶ execute now (doesn't need R1)
... later, when LOAD completes:
Instr 1: LOAD R1, [R0]     ⟶ complete, write R1
```

How it works:

- The CPU has a **reservation station** that holds instructions waiting for operands.
- It scans ahead for instructions that do not depend on the stalled result.
- It executes those instructions early.
- When the stalled instruction finally completes, it writes its result and any dependent instructions resume.

Analogy: You are cooking dinner. The oven is preheating (stall). Instead of standing still, you chop vegetables and make salad. When the oven is ready, you put the food in. You did not waste time waiting.

E.3 Problem 2: Caching Must Be Disabled for MMIO

What happens if we cache MMIO addresses?

Imagine a network card with a "packets received" register at MMIO address `0xFFFF0000`.

Without cache disable:

- T5: CPU reads `0xFFFF0000`, gets value `5` (5 packets received).
- CPU caches this value: "Address `0xFFFF0000` has value `5`."
- Network card receives 10 more packets. Register now holds `15`.
- CPU reads `0xFFFF0000` again.

- CPU checks cache: "Oh, I already know this address has `5`."
- **CPU never sees the new packets!** It uses stale data.

This is catastrophic. Device registers change constantly. Caching them is wrong.

The solution: Page Cache Disable (PCD)

The OS uses **page tables** to manage memory. Each page (usually 4 KB) has flags:

```
Page Table Entry Flags (simplified):
+----------------------------------------+
|  Physical Address | Flags              |
|  (where it maps)  |                    |
+----------------------------------------+
|                   | P = Present        |
|                   | R/W = Read/Write    |
|                   | U/S = User/Supervisor |
|                   | PCD = Page Cache Disable ← THIS ONE |
|                   | PWT = Page Write Through |
|                   | A = Accessed |
|                   | D = Dirty |
+----------------------------------------+
```

PCD = 1 means: "Never cache this page. Always go to the actual memory/device for every access."

For MMIO regions, the OS sets PCD = 1 on all page table entries covering device addresses.

Consequence: Every MMIO read is slow. There is no caching shortcut. The CPU must always wait for the full round-trip.

E.4 Problem 3: Write Ordering Must Be Preserved

What happens:

When programming a GPU, you might write multiple registers in sequence:

```
STORE R1, [GPU_REGISTER_1]   // Set resolution
STORE R2, [GPU_REGISTER_2]   // Set color depth
STORE R3, [GPU_REGISTER_3]   // Enable display
```

The GPU expects these in order. If the CPU reorders them:

```
STORE R3, [GPU_REGISTER_3]   // Enable display (WRONG! Not configured yet!)
STORE R1, [GPU_REGISTER_1]   // Set resolution
STORE R2, [GPU_REGISTER_2]   // Set color depth
```

The GPU tries to enable display before it is configured → CRASH or garbage output.

Why would the CPU reorder?

- Out-of-order execution can reorder independent memory writes.
- The CPU sees no dependency between the three STOREs (they write to different addresses).
- It might execute them in whatever order is fastest.

The solution: Memory Barriers (Fences)

A **memory barrier** is a special instruction that says: "Do NOT reorder memory operations across this line."

```
STORE R1, [GPU_REGISTER_1]
STORE R2, [GPU_REGISTER_2]
FENCE                          ← Memory barrier
STORE R3, [GPU_REGISTER_3]
```

What the FENCE does:

- All writes before the fence must complete before any writes after the fence begin.
- The CPU stalls until the first two writes are confirmed done.
- Only then does it execute the third write.

Common barrier instructions:

Architecture	Instruction	Meaning
x86	`MFENCE`	Memory fence
x86	`SFENCE`	Store fence
x86	`LFENCE`	Load fence
ARM	`DMB`	Data memory barrier
ARM	`DSB`	Data synchronization barrier

Part F: Side-by-Side Comparison

F.1 Isolated I/O vs. Memory-Mapped I/O

```
+------------------------------+     +------------------------------+
|   ISOLATED I/O               |     |   MEMORY-MAPPED I/O (MMIO)   |
|   (IN AL, PORT)              |     |   (LOAD R1, [R0])            |
+------------------------------+     +------------------------------+
|                              |     |                              |
| • Total cycles               |     | • Total cycles               |
|   T0 → T4 = 5 cycles         |     |   T0 → T5 = 6 cycles         |
|   (no hidden stall)          |     |   (plus thousands of stall   |
|                              |     |   cycles inside T4)          |
|                              |     |                              |
| • Special CPU signal at      |     | • Special CPU signal at      |
|   execute phase?             |     |   execute phase?               |
|   YES — M/IO̅ = 0           |     |   NO — plain R̅D̅ only        |
|                              |     |                              |
| • Who decides this is I/O?   |     | • Who decides this is I/O?     |
|   The CPU's Control Unit     |     |   The MCH, by checking BARs  |
|   (explicitly via M/IO̅ pin) |     |   (invisible to CPU)         |
|                              |     |                              |
| • Does CPU know it's         |     | • Does CPU know it's           |
|   accessing a device?        |     |   accessing a device?          |
|   YES — explicitly           |     |   NO — completely unaware      |
```

```
|                              |     |                              |
| • Latency penalty?           |     | • Latency penalty?             |
|   NONE — port I/O is fast    |     |   YES — T4 can stall for       |
|   (predictable, one cycle)   |     |   thousands of cycles          |
|                              |     |                              |
| • Can result be cached?      |     | • Can result be cached?        |
|   NO — I/O ports are never   |     |   NO — MMIO regions are        |
|   cached                     |     |   marked PCD=1 (uncacheable)   |
|                              |     |                              |
| • Special instruction        |     | • Special instruction          |
|   required?                  |     |   required?                    |
|   YES — IN/OUT only          |     |   NO — any LOAD/STORE works    |
|                              |     |                              |
| • Used in                    |     | • Used in                      |
|   Classic x86 (8086, 8088)   |     |   Modern x86-64, ARM,          |
|                              |     |   RISC-V, GPUs, smartphones    |
|                              |     |                              |
| • Bus complexity             |     | • Bus complexity               |
|   Simpler — M/IO̅ pin makes   |     |   More complex — MCH must      |
|   routing obvious              |     |   decode every address         |
|                              |     |                              |
+------------------------------+     +------------------------------+
```

Part G: What Your Tutorial Did Not Explicitly Cover

G.1 Why T0, T1, T2 Are Always Identical

The deep reason:

The CPU is a **finite state machine** — a hardware device with a fixed number of states and transitions.

State 1: FETCH

- Always starts with `MAR ← PC`.
- Why? Because the CPU cannot possibly know what instruction is coming until it fetches it.
- There is no "skip fetch" option. The hardware is wired to always do this first.

State 2: FETCH COMPLETE

- Always does `MBR ← Memory[MAR]` and `PC ← PC + 1`.
- Why? Because the instruction bytes must come into the CPU, and PC must advance.

State 3: DECODE

- Always does `IR ← MBR`.
- Why? Because the Control Unit can only read instructions from IR. It cannot read directly from MBR.

This uniformity reduces transistor count and power consumption. If every instruction had a completely different fetch/decode sequence, the CPU would need massively more logic. By making the first three steps identical, the hardware is simpler, cheaper, and more reliable.

Analogy: Every time you enter a restaurant, you always: (1) walk to the host stand, (2) get a menu,

(3) read the menu. Only THEN do you order. You never skip straight to ordering. The restaurant is designed this way for efficiency.

G.2 What "Simultaneous" Means at T1

When RTL shows:

```
T1:  MBR ← Memory[MAR]
     PC  ← PC + 1
```

These are NOT sequential. They do NOT happen one after the other in the same cycle. They happen **in parallel at the exact same moment.**

How is this possible physically?

The clock edge triggers everything at once:

```
Clock rising edge at T1:
    |
    ▼
+-----------------------------------------+
|  At the EXACT moment the clock goes     |
|  from 0 to 1:                           |
|                                         |
|  • Memory starts driving data onto bus  |
|    → MBR will capture it by end of cycle|
|                                         |
|  • PC increment circuit has already     |
|    computed PC+1 during previous cycle   |
|    → PC register updates to new value    |
|                                         |
|  Both happen simultaneously because    |
|  the wires were already settled before   |
|  the clock edge. The edge is just the   |
|  "commit" signal.                        |
+-----------------------------------------+
```

This is fundamental to synchronous digital circuits:

- During the cycle, combinational logic computes the next values.
- At the clock edge, all registers update simultaneously.
- The new values then propagate through combinational logic for the next cycle.

Analogy: At a race start, all runners begin moving at the exact same moment when the starting gun fires. They do not take turns. The gun is the clock edge.

G.3 The Physical Reality of the $\overline{M/IO}$ Pin

In the **Intel 8086 processor** (1978), the $\overline{M/IO}$ pin is **pin 28** on the 40-pin DIP package.

```
Intel 8086 Pin Diagram (simplified):
```

```
+----------------------------------------+
|  1  — VCC (+5V)                        |
|  2  — A14 (Address bit 14)             |
|  ...                                   |
|  28 — M/IO̅ (Memory or I/O)             |
|  ...                                   |
|  40 — GND (Ground)                     |
+----------------------------------------+
```

Every device on the system bus watches this pin on every cycle.

Why this is elegant:

- One pin tells the entire system: "Memory mode" or "I/O mode."
- No address decoding needed for this decision.
- Fast and simple.

Why this was abandoned:

- Modern CPUs have thousands of pins. But pin 28 was a **dedicated function pin.**
- Modern systems have so many devices (GPU, USB, NVMe, Thunderbolt) that a single pin cannot express the routing complexity.
- MMIO with BAR-based routing is more flexible and scalable.

G.4 What a Pipeline Bubble Actually Is

The assembly line analogy:

Imagine a 5-stage pipeline:

```
Stage 1: Fetch      Stage 2: Decode    Stage 3: Execute   Stage 4: Memory    Stage 5: Writeback
   |                   |                  |                  |                  |
   ▼                   ▼                  ▼                  ▼                  ▼
+-----+             +-----+            +-----+            +-----+            +-----+
|Instr|             |Instr|            |Instr|            |Instr|            |Instr|
|  5  |             |  4  |            |  3  |            |  2  |            |  1  |
+-----+             +-----+            +-----+            +-----+            +-----+
```

Normal operation: Every clock cycle, each instruction moves one stage to the right. Five instructions are being worked on simultaneously.

When Stage 3 (Execute) stalls:

```
Cycle N:
   |                   |                  |                  |                  |
   ▼                   ▼                  ▼                  ▼                  ▼
+-----+             +-----+            +-----+            +-----+            +-----+
|Instr|             |Instr|            |STALL|            |Instr|            |Instr|
|  5  |             |  4  |            |  3  |            |  2  |            |  1  |
+-----+             +-----+            +-----+            +-----+            +-----+

Cycle N+1:
   |                   |                  |                  |                  |
```

```
  ▼           ▼           ▼           ▼           ▼
+-----+     +-----+     +-----+     +-----+     +-----+
|Instr|     |Instr|     | NOP |     |Instr|     |Instr|
|  6  |     |  5  |     |(bubble)   |  3  |     |  2  |
+-----+     +-----+     +-----+     +-----+     +-----+
```

What is a NOP (No-Operation)?

- An instruction that does nothing.
- It occupies a pipeline slot but produces no useful result.
- It is inserted automatically by the pipeline control logic to fill the gap.

Why bubbles matter:

- Every bubble is a wasted clock cycle.
- A CPU with thousands of bubbles is running at a fraction of its potential speed.
- This is why MMIO stalls are so expensive — they create massive bubbles.

Part H: Complete Math Practice

H.1 Calculate Clock Cycles for a 5 µs PCIe Delay

Given:

- CPU frequency = 3 GHz.
- PCIe device delay = 5 microseconds (µs) = 5,000 nanoseconds (ns).

Step 1: Convert frequency to cycle time.

$$\text{Cycle time} = \frac{1}{\text{frequency}} = \frac{1}{3{,}000{,}000{,}000 \text{ Hz}} = 0.333... \text{ ns}$$

Step 2: Calculate number of cycles.

$$\text{Cycles} = \frac{\text{Total delay}}{\text{Cycle time}} = \frac{5{,}000 \text{ ns}}{0.333 \text{ ns}} = \mathbf{15{,}000 \text{ cycles}}$$

Answer: The CPU stalls for approximately **15,000 clock cycles** waiting for the PCIe device.

H.2 Convert MMIO Address to Binary

Given: MMIO address = `0xFFFF0000`

Step 1: Convert each hex digit to 4 bits.

Hex	Binary
F	1111
F	1111
F	1111
F	1111

Table 5 – continued

Hex	Binary
0	0000
0	0000
0	0000
0	0000

Step 2: Combine.

- `0xFFFF0000` = `11111111111111110000000000000000` (32 bits).

Verification in decimal:

$$0xFFFF0000 = (15 \times 16^7) + (15 \times 16^6) + (15 \times 16^5) + (15 \times 16^4) = 4{,}294{,}901{,}760$$

Answer: `11111111111111110000000000000000` in binary.

H.3 Calculate Pipeline Efficiency with Bubbles

Given:

- Pipeline has 5 stages.
- Normal execution: 1 instruction per cycle (after pipeline fill).
- MMIO stall: 1,000 cycles of bubbles.
- Total instructions in program: 10,000.

Step 1: Calculate ideal cycles.

$$\text{Ideal} = \text{Pipeline fill} + \text{Instructions} = 5 + 10{,}000 = 10{,}005 \text{ cycles}$$

Step 2: Calculate actual cycles with stall.

$$\text{Actual} = \text{Ideal} + \text{Stall cycles} = 10{,}005 + 1{,}000 = 11{,}005 \text{ cycles}$$

Step 3: Calculate efficiency.

$$\text{Efficiency} = \frac{\text{Ideal}}{\text{Actual}} = \frac{10{,}005}{11{,}005} = 0.909 = \mathbf{90.9\%}$$

Answer: The pipeline runs at **90.9% efficiency** due to the MMIO stall. Without the stall, it would be 100% (after fill).

Part I: Key Points to Remember

- **RTL arrow ← means "copy from source to destination."** Source is NOT destroyed.
- **Every instruction breaks into 4–6 micro-operations (µ-ops),** one per clock cycle.

- **Fetch (T0–T1) and Decode (T2) are identical for ALL instructions.** The CPU does not know what it is executing until after decode.
- **PC always points to the NEXT instruction.** It increments during T1 while the current instruction is being fetched.
- **MAR is the "window" to the outside world.** Every memory or I/O access starts by loading an address into MAR.
- **MBR is the "loading dock."** All data to/from memory passes through MBR.
- **IR holds the instruction being executed.** The Control Unit reads the opcode from IR to decide what to do.
- **Isolated I/O uses $\overline{M/IO} = 0$ to explicitly say "I/O mode."** The CPU knows it is talking to a device.
- **MMIO has no $\overline{M/IO}$ pin.** The CPU fires normal memory reads. The MCH intercepts and routes to devices.
- **MMIO T4 can stall for thousands of cycles.** PCIe is 10–100× slower than RAM.
- **Out-of-order execution skips stalled instructions** and executes later independent instructions first.
- **MMIO regions are marked PCD = 1 (Page Cache Disable)** so the CPU always reads fresh device data.
- **Memory barriers (FENCE instructions) prevent reordering** of MMIO writes that would break device initialization.
- **Pipeline bubbles are NOPs** inserted when a stage stalls. Every bubble is a wasted cycle.

Part J: One-Page Visual Summary

```
+-------------------------------------------------------------------+
|              RTL & MICRO-OPERATIONS: BIG PICTURE                  |
+-------------------------------------------------------------------+
|                                                                   |
|   EVERY INSTRUCTION FOLLOWS THIS PATTERN:                         |
|                                                                   |
|   T0: MAR ← PC        (set up instruction address)                |
|   T1: MBR ← Mem[MAR]  (fetch instruction bytes)                   |
|       PC ← PC + 1     (point to next instruction)                 |
|   T2: IR ← MBR        (decode instruction)                        |
|   T3–T5: Execute      (depends on instruction type)               |
|                                                                   |
+-------------------------------------------------------------------+
|                                                                   |
|   ISOLATED I/O (IN AL, PORT):                                     |
|   ──────────────────────────                                      |
|   T3: MAR ← IR[port]    M/IO̅=0, RD̅=0  (switch to I/O mode)  |
|   T4: AL ← Bus_Data     (device puts data on bus)                 |
|                                                                   |
|   MMIO (LOAD R1, [R0]):                                          |
|   ──────────────────────                                          |
|   T3: MAR ← R0          RD̅=0  (normal memory read)            |
|   T4: MCH intercepts → PCIe  (HIDDEN STALL, thousands of cycles)|
|   T5: R1 ← Bus_Data     (finally get data)                        |
```

```
|                                                                  |
+------------------------------------------------------------------+
|                                                                  |
|   KEY DIFFERENCES:                                               |
|   • Isolated: CPU KNOWS it's I/O ($\overline{M/IO}$ pin)      |
|   • MMIO: CPU is BLIND (MCH decides routing)                     |
|   • Isolated: Fast, predictable, limited address space           |
|   • MMIO: Slow stalls, huge address space, needs barriers      |
|                                                                  |
+------------------------------------------------------------------+
```

This completes the Section 2 research package on RTL & Micro-Operations. Every clock cycle is explained from absolute zero, every register is defined, every hidden stall is exposed, and every engineering problem has its solution explained step-by-step.

Section 3 — Hardware Timing Diagrams

Part A: What Is a Timing Diagram?

A.1 The Problem Timing Diagrams Solve

In Sections 1 and 2, we talked about **what** moves where (RTL) and **why** (I/O mapping). But we never talked about **when**.

A timing diagram shows exactly WHEN each electrical signal changes, plotted against time.

The horizontal axis is time. It moves left to right. The further right you go, the later in time you are.

The vertical axis shows voltage on a wire. Each horizontal line in the diagram represents one physical wire on the motherboard.

High voltage = logic 1. Low voltage = logic 0.

This is NOT abstract. If you took an oscilloscope (a device that measures voltage over time), clipped its probe to a wire on your motherboard, and pressed "capture," you would literally see these wiggles on the screen. The timing diagram is a drawing of that oscilloscope trace.

A.2 Why Do We Need Timing Diagrams?

Reason 1: Hardware has no "pause button."

- In software, you can set a breakpoint and stop execution to inspect variables.
- In hardware, electrons move at the speed of light (well, 60% of it in copper). You cannot pause a wire.
- Timing diagrams let us "freeze" reality and study what every wire is doing at every instant.

Reason 2: Multiple things happen simultaneously.

- The Address Bus carries an address.
- The Control Bus carries read/write commands.
- The Data Bus carries data.
- All of these interact. A timing diagram shows their relationship.

Reason 3: Delays matter.

- A device might take 1 cycle to respond. Or 100. Or 10,000.
- The CPU must know when data is valid and when to wait.
- Timing diagrams make these delays visible.

Part B: Reading the Notation — Every Symbol Explained

Before we analyze any diagram, you must know how to read the symbols. I will explain every single character.

B.1 The Clock Signal

```
___|---|___|---|___|---|___
```

What each character means:

Symbol	Meaning
`___`	The wire is **LOW** (0V, logic 0)
`---`	The wire is **HIGH** (3.3V or 5V, logic 1)
`|`	**Transition instant.** The voltage is switching right now.

How to read it:

- Start from the left: `___` = LOW.
- Then `\|` = rising edge (0 → 1).
- Then `---` = HIGH.
- Then `\|` = falling edge (1 → 0).
- Then `___` = LOW again.

One complete cycle = one LOW + one HIGH:

```
___|---|___  =  One complete clock cycle
 └─ C1─┘
```

The rising edge is the "trigger." On most CPUs, all register updates happen on the rising edge (when the clock goes from 0 to 1). This is why we care about where the vertical bars are.

B.2 A Bus Carrying Data

```
---< MMIO Address 0x4000 >---
```

What each character means:

Symbol	Meaning
`---`	The bus is **idle**, transitioning, or carrying irrelevant data
`<`	**Start of valid data window**
`MMIO Address 0x4000`	**The actual value** being carried
`>`	**End of valid data window**

Why angle brackets?

- A bus is not one wire. It is 32 wires (for 32-bit address) or 64 wires (for 64-bit data).
- We cannot draw 64 separate lines. So we collapse them into one "band" and label what value all 64 wires together represent.
- The angle brackets say: "During this time window, all wires are stable and together they represent this value."

Analogy: Instead of drawing 32 individual people holding up signs with bits, we draw one big banner that says "0x4000." The banner represents all 32 people together.

B.3 An Active-Low Signal Going Low

```
---\_____________/---
```

What each character means:

Symbol	Meaning
`---`	Signal is **HIGH** (inactive, since it is active-low)
`\`	**Falling edge** (HIGH → LOW transition)
`_____`	Signal is **LOW** (active, doing its job)
`/`	**Rising edge** (LOW → HIGH transition)
`---`	Signal is **HIGH** again (inactive)

Why the bar or # symbol?

- `RD#` or $\overline{RD}$ both mean "Read signal, active LOW."
- The # or bar is a reminder: "This signal does its job when it is at 0V, not 5V."
- When you see `RD#` go LOW (`\_____/`), that means "READ IS HAPPENING NOW."

B.4 A Wait State Signal

```
__/-------------\__
```

What each character means:

Symbol	Meaning
`__`	Signal starts **LOW** (device is ready, no wait needed)
`/`	**Rising edge** (LOW → HIGH)
`-------------`	Signal stays **HIGH** for multiple cycles (device says "WAIT, I' m not ready")
`\`	**Falling edge** (HIGH → LOW)
`__`	Signal returns **LOW** (device says "OK, I' m ready now")

This is the OPPOSITE shape from RD#. RD# starts HIGH and goes LOW to activate. Wait State starts LOW and goes HIGH to activate.

Why opposite? Different conventions. Some signals are active-high, some are active-low. The hardware designer chooses based on what makes the circuit simpler.

Part C: The Full Timing Diagram — Cycle by Cycle

Here is the complete diagram from your tutorial, with my annotations:

```
Clock (CLK)   ___|---|___|---|___|---|___|---|___|---|___
              [  C1  ][  C2  ][  C3  ][  C4  ][  C5  ]

Address Bus   ---< MMIO Address 0x4000 >-------------------

Read (RD#)    ---\_____________________________/---------

Wait State    ___/-------------------------\_____________

Data Bus      ----------------------------------< Data >--
```

What we are looking at: A Memory-Mapped I/O read from address `0x4000` (a device register), showing wait states because the device is slow.

C.1 Cycle C1 — Address Is Placed on the Bus

What happens:

Address Bus: Transitions from `---` (idle) to `< MMIO Address 0x4000 >` (valid).

Step by step:

1. At the start of C1 (rising clock edge), the CPU drives the 32-bit address `0x4000` onto all 32 wires of the Address Bus simultaneously.
2. Each wire goes to either HIGH or LOW voltage to represent one bit of `0x4000`.
3. The angle brackets `< >` appear because the address is now stable and valid.

What is `0x4000` in binary?

- `0x4000` = `0100 0000 0000 0000` (16-bit) or `00000000000000000100000000000000` (32-bit).
- Bit 14 is 1, all others are 0.

RD# (Read): Goes from `---` (HIGH, inactive) to `\` (falling edge) to `_____` (LOW, active).

What this means:

- The CPU is asserting: "I want to read from the address currently on the Address Bus."
- The falling edge of RD# is the "starting gun" for the transaction.
- Every device on the bus sees RD# go LOW and knows: "A read is starting. Check if this address is yours."

Wait State: Stays `___` (LOW, inactive).

What this means:

- The device has not yet responded. It might be ready, or it might not have noticed yet.

- At this point, the device has just seen the address and RD#.

Data Bus: Stays ---------- (idle).

What this means:

- No data is on the bus yet. The device has not had time to respond.
- This is expected. C1 is just the "request" phase.

Analogy: You walk into a store and tell the clerk: "I want item number 0x4000." (Address Bus). You also say: "Please hand it to me." (RD# goes LOW). The clerk hears you but has not moved yet. No item is on the counter yet.

C.2 Cycle C2 — Wait State Asserted: The Device Is Not Ready

What happens:

Address Bus: `< MMIO Address 0x4000 >` stays valid.

Why? The CPU must hold the address steady the entire time RD# is LOW. If the address changed, the device would look up the wrong location.

RD#: Stays _____ (LOW, active).

The CPU is still waiting for its read.

Wait State: Goes from ___ (LOW) to `/` (rising edge) to ---------- (HIGH, active).

> [!WARNING] **THIS IS THE CRITICAL MOMENT.**

What this means:

- The device (network card, GPU, etc.) asserts Wait State HIGH.
- It is telling the CPU: "I received your request. I am working on it. But I am NOT ready yet. FREEZE."
- The CPU sees Wait State HIGH and stops. It does NOT proceed to the next step.

What the CPU does when it sees Wait State HIGH:

- The CPU's internal pipeline registers have a "clock enable" input.
- When Wait State is HIGH, the clock enable is turned OFF.
- The registers STOP updating. Their values are frozen.
- The CPU is NOT executing instructions. It is NOT doing anything useful.
- But the clock keeps ticking. C2, C3, C4… real time passes. Real electricity is wasted.

This is a hardware pipeline bubble. It is not software. The CPU is not running a loop. It is physically frozen by an electrical signal.

Data Bus: Still ---------- (idle).

No data yet. The device is still working.

Analogy: The clerk says: "I found item 0x4000, but it is in the back room. I need to go get it. Please wait." (Wait State HIGH). You stand at the counter frozen, unable to do anything else, while the clerk is gone. The store clock keeps ticking, but no business is happening.

C.3 Cycle C3 — Wait State Continues: Still Frozen

What happens:

Address Bus: `< MMIO Address 0x4000 >` still valid.

RD#: Still _____ (LOW, active).

Wait State: Still ---------- (HIGH, active).

Data Bus: Still idle.

Nothing has changed. The device is still not ready. The CPU is still frozen.

How many wait states can there be?

- **There is no fixed limit.** It depends entirely on the device.
- A fast device might need 0 wait states (responds in C1).
- A slow PCIe device might need 10, 50, 100, or even 1,000 wait states.
- The CPU has NO CHOICE but to wait. It cannot skip ahead. It cannot do something else. It is frozen.

Why is the device slow?

- It might need to access its own internal memory.
- It might need to communicate with another chip.
- It might be busy with another task.
- The PCIe bus itself has serialization delay (explained in Part E).

Analogy: The clerk is still in the back room. You are still frozen at the counter. The store clock has ticked twice now. Still no business.

C.4 Cycle C4 — Device Asserts Data, Wait State Drops

What happens:

Address Bus: `< MMIO Address 0x4000 >` still valid (must stay until transaction ends).

RD#: Still _____ (LOW, active).

Wait State: Goes from ---------- (HIGH) to \ (falling edge) to ___ (LOW, inactive).

> [!IMPORTANT] **THIS IS THE "ALL CLEAR" SIGNAL.**

What this means:

- The device has finished its work.
- It is telling the CPU: "I am ready now. You can proceed."
- The falling edge of Wait State is the "green light."

Simultaneously:

Data Bus: Goes from ---------- (idle) to `< Data >` (valid).

What this means:

- The device drives the requested data onto the Data Bus.
- All 32 or 64 data wires now carry stable, valid bits.

- The angle brackets `< >` appear because the data is ready to be read.

What is happening physically:

- The device activates its output drivers.
- Voltage levels appear on each data wire (HIGH or LOW depending on each bit).
- These voltages propagate across the PCB traces to the CPU.
- The CPU sees stable data on the Data Bus AND Wait State going LOW.

Analogy: The clerk returns from the back room with your item (Data Bus now has data). The clerk says: "Here it is, you can take it now." (Wait State drops LOW). You are no longer frozen.

C.5 Cycle C5 — CPU Samples the Data

What happens:

Address Bus: Returns to `---` (idle).

The transaction is ending. The address is no longer needed.

RD#: Goes from `_____` (LOW) to `/` (rising edge) to `¯¯¯` (HIGH, inactive).

The CPU deasserts the read command. The transaction is complete.

Wait State: Stays `___` (LOW, inactive).

The device is done. No more waiting.

Data Bus: `< Data >` stays valid briefly, then returns to idle.

What happens at the clock edge:

At the rising edge of C5 (or the falling edge of C4, depending on bus specification):

- The CPU **samples** the Data Bus.
- It captures all bits into the MBR (Memory Buffer Register).
- From MBR, the data moves to its final destination (AL, R1, etc. — as covered in Section 2).

What "sampling" means physically:

- Inside the CPU, there is a bank of flip-flops (one per data bit).
- On the clock edge, each flip-flop latches the voltage on its corresponding data wire.
- If the wire is HIGH, the flip-flop stores 1. If LOW, it stores 0.
- This capture happens in picoseconds.

After sampling:

- RD# goes HIGH (transaction complete).
- Address Bus goes idle.
- Data Bus goes idle.
- The CPU can now proceed to the next instruction.

Analogy: You take the item from the clerk (sample the data). You say "thank you" (RD# goes HIGH). You leave the counter (buses go idle). The transaction is done.

C.6 Complete Cycle Summary

Cycle	Address Bus	RD#	Wait State	Data Bus	CPU State
C1	`0x4000` placed on bus	Goes LOW (read starts)	LOW (device initially seems ready)	Idle	Active, initiating transaction
C2	`0x4000` held steady	Stays LOW	Goes HIGH (device says "not ready")	Idle	**FROZEN.** Pipeline bubbles inserted.
C3	`0x4000` held steady	Stays LOW	Stays HIGH	Idle	Still frozen. Clock cycles wasted.
C4	`0x4000` held steady	Stays LOW	Goes LOW (device says "ready now")	Valid data appears	Still waiting for clock edge to sample.
C5	Returns to idle	Goes HIGH (transaction ends)	Stays LOW	Data sampled by CPU, then returns to idle	Transaction complete. CPU resumes execution.

Part D: What Is Actually Happening at the Electrical Level

D.1 Voltage Propagation in Copper

Every signal transition in that diagram is a **real voltage change** propagating through copper traces on the PCB (printed circuit board).

Speed of electricity in copper: Approximately 60% the speed of light.

- Speed of light = 300,000,000 meters/second.
- In copper = 180,000,000 meters/second = 180 meters per microsecond.

What this means:

- If your CPU and device are 10 centimeters apart on the motherboard:
  - Signal travel time = 0.1 meter / 180,000,000 m/s = 0.55 nanoseconds.
- This seems tiny, but at 3 GHz (0.333 ns per cycle), this is almost 2 clock cycles just for the signal to travel!

Why this matters:

- The timing diagram assumes instantaneous signals. In reality, there is delay.
- This delay is why wait states exist — the device is not just "slow," the signals physically take time to travel.

D.2 The Full Physical Path of an MMIO Read

When RD# falls LOW at C1, here is the complete physical journey:

Step 1 — CPU to MCH (on motherboard):

- RD# voltage change travels from CPU pin across PCB trace to MCH.
- Time: ~0.5 nanoseconds.
- MCH sees RD# LOW and address `0x4000` on Address Bus.

Step 2 — MCH decodes address:

- MCH checks BARs: "0x4000 is in network card range."
- MCH prepares PCIe transaction.

Step 3 — MCH to device (over PCIe):

- PCIe uses **serial lanes** — not parallel buses.
- Data is serialized (converted from parallel to serial bit stream).
- Sent over **differential pairs** — two wires per lane, one carrying signal, one carrying inverse.
- Noise picked up by both wires cancels out (common-mode rejection).

Step 4 — Device processes request:

- Device receives PCIe packet.
- It reads its internal register at the requested offset.
- This might take microseconds if the device is busy.

Step 5 — Device responds over PCIe:

- Device serializes response data.
- Sends back over PCIe lanes to MCH.

Step 6 — MCH to CPU:

- MCH receives PCIe response.
- Puts data on Data Bus.
- Drops Wait State signal.
- CPU samples data at next clock edge.

Total round-trip time: 1,000 to 10,000 nanoseconds for a typical PCIe device.

> [!IMPORTANT] **At 3 GHz, this is 3,000 to 30,000 clock cycles of waiting.**

D.3 PCIe Differential Pairs Explained

What is a differential pair?

Instead of one wire carrying a signal, PCIe uses TWO wires:

```
Wire A:  ___/‾‾‾\___/‾‾‾\___  (signal)
Wire B:  ‾‾‾\___/‾‾‾\___/‾‾‾  (inverse of signal)
```

Why two wires?

- Electrical noise affects both wires equally.
- The receiver subtracts Wire B from Wire A.
- Noise cancels out: (Signal + Noise) − (Inverse Signal + Noise) = Signal − Inverse Signal = clean signal.
- This allows much higher speeds with fewer errors.

PCIe lane speeds:

Generation	Speed per Lane
PCIe 3.0	8 GT/s
PCIe 4.0	16 GT/s
PCIe 5.0	32 GT/s
PCIe 6.0	64 GT/s

A "x16" GPU slot uses 16 lanes simultaneously for massive bandwidth.

Part E: The AXI Protocol — Split Transactions

E.1 What Is AXI?

AXI = Advanced eXtensible Interface

It is the standard bus protocol inside ARM-based chips (smartphones, tablets, embedded systems, modern servers).

AXI is a "split transaction protocol." This means it separates the transaction into independent channels.

E.2 AXI Channels (Five Total)

Channel	Name	Carries	Direction
AW	Write Address	Address + control info for writes	Master (CPU) → Slave (device)
W	Write Data	The actual data bytes to write	Master → Slave
B	Write Response	"I got your write" acknowledgment	Slave → Master
AR	Read Address	Address + control info for reads	Master → Slave
R	Read Data	The actual data bytes from read	Slave → Master

E.3 Why Split Transactions Matter

Traditional bus (like our timing diagram):

```
CPU: "Give me address 0x4000" → wait → wait → wait → "Here is the data"
```

- CPU is frozen during the wait.
- Cannot issue another request until this one completes.

AXI split transaction:

```
CPU: "Give me address 0x4000" (AR channel)
CPU: "Give me address 0x4004" (AR channel) ← Does NOT wait!
CPU: "Give me address 0x4008" (AR channel) ← Does NOT wait!

Later:
Device: "Here is data for 0x4000" (R channel)
Device: "Here is data for 0x4008" (R channel) ← Can be out of order!
Device: "Here is data for 0x4004" (R channel)
```

Key advantages:

1. **CPU is not frozen.** It can issue multiple requests without waiting.
2. **Responses can come back out of order.** Each response has an ID tag so the CPU knows which request it belongs to.
3. **Higher throughput.** The bus is never idle while waiting.

This is called "outstanding transactions" or "pipelining the bus itself."

E.4 AXI Timing Diagram (Simplified)

```
Clock         ___|¯|_|¯|_|¯|_|¯|_|¯|_|¯|_|¯|_|¯|_|¯|_|¯|___
              [C1][C2][C3][C4][C5][C6][C7][C8][C9]

AR Channel    ---< Addr 0x4000 >---< Addr 0x4004 >-----------
              (read address 1)    (read address 2)

R Channel     ------------------------< Data 0x4000 >--------
                                (response 1)

R Channel     -----------------------------------< Data 0x4004>
                                                 (response 2)
```

What you see:

- C1: CPU sends first read address (AR channel).
- C2: CPU sends second read address (AR channel) — does NOT wait for first response!
- C5: First response arrives (R channel).
- C8: Second response arrives (R channel).

> [!NOTE] **The wait state problem still exists** — a slow device still takes time to respond. But AXI allows the CPU to do other things while waiting, rather than freezing completely.

Part F: RAM vs MMIO Timing Diagram Comparison

F.1 DDR5 RAM Read (Simplified)

```
Clock         ___|¯|_|¯|_|¯|_|¯|_|¯|_|¯|_|¯|_|¯|_|¯|_|¯|___
              [C1][C2][C3][C4][C5][C6][C7][C8][C9]
```

```
Address Bus   --< Row Addr >--< Col Addr >-------------------
              (row first)      (column second)

RD#           --\___/---------------------------------------

Wait State    (none visible to CPU)

Data Bus      ----------------------------------< Data >------
```

Key differences from MMIO:

No Wait State signal visible to CPU

- RAM has predictable timing.
- During boot, the BIOS reads the RAM's SPD (Serial Presence Detect) chip to learn its speed.
- The memory controller programs the exact number of cycles needed (CAS latency, RAS latency, etc.).
- The CPU never sees a Wait State. The memory controller absorbs all delays internally.

Row and Column addresses

- RAM is organized in a 2D grid: rows and columns.
- The address bus carries row first, then column.
- This is called **multiplexed addressing** — saves pins.

Predictable delay

- DDR5-6400 with CAS-40 means: 40 clock cycles from address to data.
- This is known at boot time. No surprises.

F.2 Side-by-Side Comparison

```
+-----------------------------+     +-----------------------------+
|      RAM READ (DDR5)        |     |    MMIO READ (PCIe Device)  |
+-----------------------------+     +-----------------------------+
|                             |     |                             |
| • Wait State signal?        |     | • Wait State signal?        |
|   NO — hidden by memory     |     |   YES — device asserts it   |
|   controller                |     |   explicitly                |
|                             |     |                             |
| • Delay is...               |     | • Delay is...               |
|   PREDICTABLE (programmed   |     |   UNPREDICTABLE (depends on |
|   at boot, fixed cycles)    |     |   device load, link speed)  |
|                             |     |                             |
| • CPU sees stall?           |     | • CPU sees stall?           |
|   NO — memory controller    |     |   YES — CPU frozen until    |
|   handles it internally     |     |   Wait State drops          |
|                             |     |                             |
| • Address format            |     | • Address format            |
|   Row + Column (multiplexed)|     |   Single flat address       |
|                             |     |                             |
| • Used for                  |     | • Used for                  |
```

```
|   Main memory (RAM sticks)  |     |   GPU, network card, USB,    |
|                             |     |   NVMe, Thunderbolt           |
|                             |     |                              |
| • Speed                     |     | • Speed                       |
|   Fast (nanoseconds)        |     |   Slow (microseconds)         |
|                             |     |                              |
| • Bus protocol              |     | • Bus protocol                |
|   DDR5 (parallel)           |     |   PCIe (serial, differential) |
|                             |     |                              |
+-----------------------------+     +------------------------------+
```

Part G: What Your Tutorial Did Not Cover

G.1 What Determines the Number of Wait States?

The number of wait states is NOT fixed. It depends on:

Factor 1: PCIe Link Speed

Link	Bandwidth per Lane
PCIe 3.0 x1	~1 GB/s
PCIe 4.0 x1	~2 GB/s
PCIe 5.0 x1	~4 GB/s

- Faster link = fewer wait states.

Factor 2: Device Internal Logic

- A simple device might respond in 1 microsecond.
- A complex GPU might need 10 microseconds to process a register read.
- The device's own clock speed and workload matter.

Factor 3: Distance on Board

- Longer PCB traces = more signal propagation delay.
- Typical motherboard: 10–30 cm between CPU and PCIe slot.

Factor 4: Bus Load

- If multiple devices are using the PCIe bus simultaneously, arbitration delays add wait states.
- The MCH must schedule requests fairly.

Factor 5: Power State

- A device in a low-power state might need time to wake up.
- This can add hundreds of microseconds of extra delay.

> [!IMPORTANT] **Hardware designers cannot eliminate wait states.** They can only minimize them through faster links, closer placement, and better scheduling.

G.2 READY vs WAIT Signal Conventions

Different architectures use opposite polarity for the same concept:

Convention 1: Active-Low READY (e.g., Intel 8086, classic CPUs)

- Signal name: `READY#` or $\overline{READY}$
- LOW (0) = "I am NOT ready" (wait)
- HIGH (1) = "I am ready" (proceed)
- When device needs time: pulls READY# LOW
- When done: releases READY# HIGH

Convention 2: Active-High WAIT (e.g., many FPGA bus designs)

- Signal name: `WAIT`
- HIGH (1) = "WAIT, I am not ready"
- LOW (0) = "OK, proceed"
- When device needs time: drives WAIT HIGH
- When done: drives WAIT LOW

They are the SAME concept, opposite polarity.

How to tell which one your system uses:

- Check the datasheet. It will say "active low" or "active high."
- Look at the signal name: # or bar = active low. No mark = usually active high.
- When in doubt, assume active low for control signals (industry standard).

G.3 Setup Time and Hold Time — The Hidden Requirements

When the CPU samples data at C5, it does NOT just "read" the wire instantaneously. The data must be stable for minimum times:

Setup Time (t_{setup})

- **Definition:** Data must be stable BEFORE the clock edge.
- **Typical value:** 0.1 to 1 nanosecond.
- **Why:** The flip-flop inside the CPU needs time to "see" the voltage and decide if it is HIGH or LOW.

Hold Time (t_{hold})

- **Definition:** Data must remain stable AFTER the clock edge.
- **Typical value:** 0.05 to 0.5 nanosecond.
- **Why:** The flip-flop needs time to latch the value before it can change.

Visual diagram:

```
Clock:        ___|---|___
                    ↑
                 Clock edge (sampling happens here)

Data:         ----< Valid >----
                    ↑    ↑
                 Setup  Hold
```

```
                time   time

                [stable][stable]
```

What happens if setup/hold time is violated?

- The flip-flop might capture the wrong value.
- This is called a **timing violation** or **metastability.**
- The result is **intermittent and unpredictable** — sometimes correct, sometimes garbage.
- It depends on temperature, voltage, and even cosmic rays.
- **This is one of the hardest bugs to debug in hardware.**

Analogy: You are taking a photograph of a moving car. If you press the shutter when the car is blurry (setup time violation), the photo is garbage. If you move the camera right after pressing the shutter (hold time violation), the photo is also garbage. You need the car to be still AND keep the camera steady for a moment.

G.4 Why the Address Bus Stays Stable During Wait States

Look at the timing diagram again:

```
Address Bus   ---< MMIO Address 0x4000 >-------------------
```

The `< 0x4000 >` band spans from C1 all the way through C5.

Why can not the CPU change the address?

Reason 1: Device uses the address throughout

- The device might need the address for its entire internal lookup process.
- If the address changed at C3, the device would fetch from the wrong location.
- Example: A GPU register at offset 0x4000 vs 0x4004 are completely different registers.

Reason 2: Bus protocol requirement

- The bus specification says: "Master must hold address stable until transaction completes."
- Violating this is a bus protocol error.

Reason 3: Multiple devices might be decoding

- While Wait State is asserted, multiple devices might be doing address comparison.
- Changing the address mid-transaction would confuse all of them.

> [!IMPORTANT] **The CPU physically cannot change the address until RD# goes HIGH at C5.** This is enforced by the Control Unit's state machine.

G.5 What Happens at the Transistor Level During a Wait State

How does the CPU actually "freeze"?

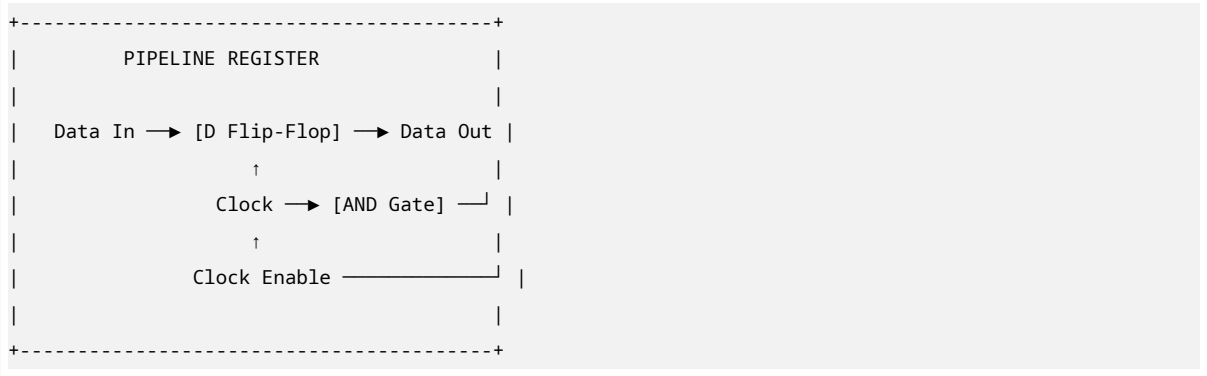
The naive approach: Stop the clock.

- Problem: The clock drives PLLs (Phase-Locked Loops) that synchronize the entire chip.
- Stopping the clock would desynchronize everything and cause chaos.

- Also, restarting the clock cleanly is difficult.

The real approach: Clock-enable gating.

Inside the CPU's pipeline registers:



How it works:

- The flip-flop only updates when Clock AND Clock Enable are both active.
- Normally, Clock Enable = 1 (always enabled).
- When Wait State is asserted, Clock Enable = 0.
- The clock keeps ticking, but the AND gate blocks it from reaching the flip-flop.
- The flip-flop holds its old value. The pipeline is frozen.
- No power is wasted on switching (saving energy).
- When Wait State drops, Clock Enable = 1, and the pipeline resumes.

> [!NOTE] **This is called "clock gating" and it is used everywhere in modern CPUs for power saving.**

Part H: Complete Math Practice

H.1 Calculate Signal Propagation Delay

Given:

- Distance from CPU to MCH = 15 cm = 0.15 meters.
- Signal speed in copper = 60% of speed of light.
- Speed of light = 300,000,000 m/s.

Step 1: Calculate signal speed in copper.

$$\text{Signal speed} = 0.60 \times 300{,}000{,}000 \text{ m/s} = 180{,}000{,}000 \text{ m/s}$$

Step 2: Calculate propagation delay.

$$\text{Delay} = \frac{\text{Distance}}{\text{Speed}} = \frac{0.15 \text{ m}}{180{,}000{,}000 \text{ m/s}} = 0.000{,}000{,}000{,}833 \text{ s} = \mathbf{0.833 \text{ nanoseconds}}$$

Step 3: Convert to clock cycles at 3 GHz.

$$\text{Cycle time} = \frac{1}{3{,}000{,}000{,}000} = 0.333 \text{ ns}$$

$$\text{Cycles} = \frac{0.833}{0.333} = \mathbf{2.5 \text{ cycles}}$$

Answer: The signal takes approximately **0.83 nanoseconds** (2.5 clock cycles at 3 GHz) just to travel from CPU to MCH.

H.2 Calculate PCIe Bandwidth

Given:

- PCIe 4.0, x16 lane width.
- Transfer rate = 16 GT/s per lane.
- Encoding overhead = 128b/130b (about 1.5% overhead).

Step 1: Calculate raw bandwidth per lane.

- Raw = 16 GT/s × 1 byte per transfer (8 bits) = 16 GB/s per lane.
- Wait — GT/s means "gigatransfers per second." Each transfer is one bit on one lane.
- Raw per lane = 16 GT/s × 1 bit = 16 Gb/s = 2 GB/s per lane.

Step 2: Calculate for 16 lanes.

$$\text{Total raw} = 2 \text{ GB/s} \times 16 = 32 \text{ GB/s}$$

Step 3: Apply encoding overhead.

- PCIe 4.0 uses 128b/130b encoding: 128 bits of data for every 130 bits sent.
- Efficiency = 128/130 = 0.9846 = 98.46%.

$$\text{Usable bandwidth} = 32 \text{ GB/s} \times 0.9846 = \mathbf{31.5 \text{ GB/s}}$$

Answer: PCIe 4.0 x16 provides approximately **31.5 GB/s** of usable bandwidth.

H.3 Calculate Wait State Cycles for a Slow Device

Given:

- CPU frequency = 2.5 GHz.
- Device response time = 5 microseconds.
- One wait state = 1 clock cycle.

Step 1: Calculate cycle time.

$$\text{Cycle time} = \frac{1}{2{,}500{,}000{,}000} = 0.4 \text{ nanoseconds}$$

Step 2: Convert device delay to nanoseconds.

$$5 \text{ microseconds} = 5{,}000 \text{ nanoseconds}$$

Step 3: Calculate wait states.

$$\text{Wait states} = \frac{5{,}000 \text{ ns}}{0.4 \text{ ns}} = \mathbf{12{,}500 \text{ cycles}}$$

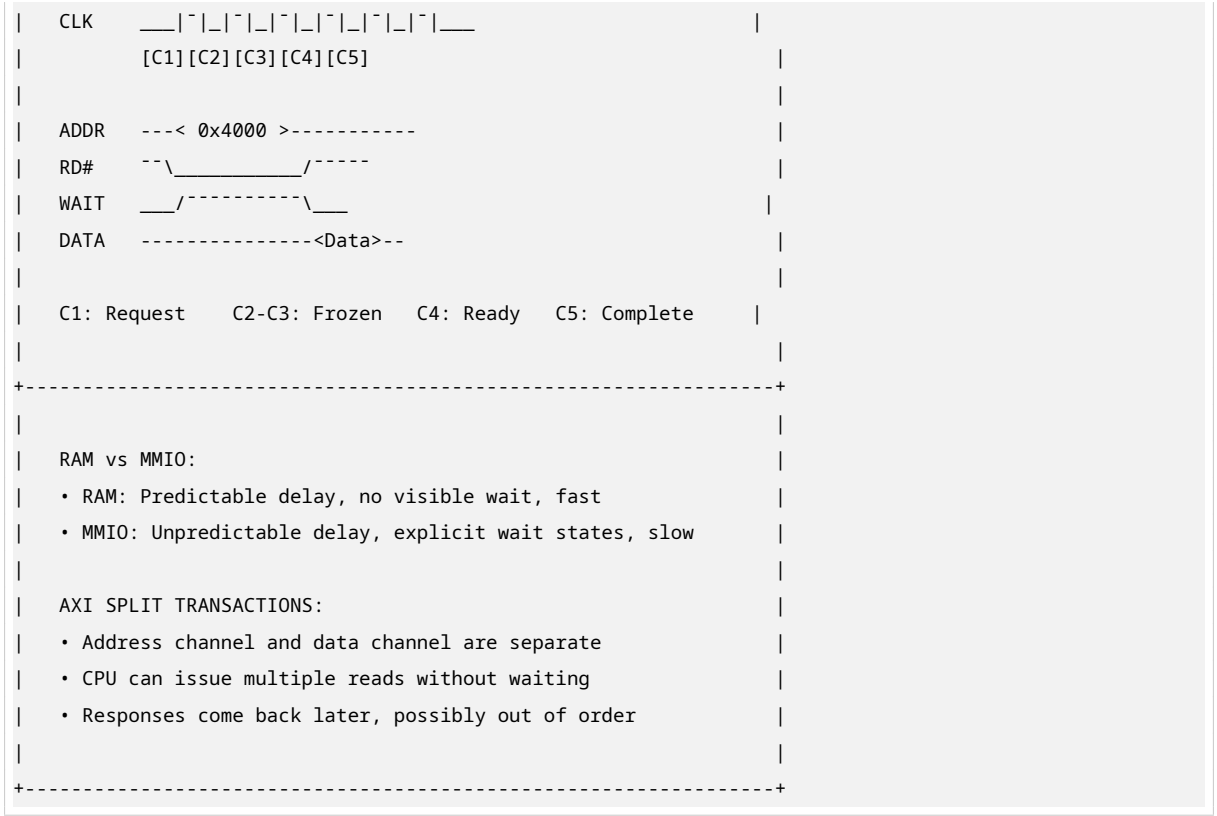
Answer: The CPU will experience **12,500 wait states** (pipeline bubbles) waiting for this device.

Part I: Key Points to Remember

- **Timing diagrams show voltage on wires over time.** Horizontal = time, vertical = voltage.
- `___` = LOW (0V), `¯¯¯` = HIGH (3.3V/5V), `\|` = transition instant.
- `< >` on a bus means "valid, stable data during this window."
- `\` = falling edge (HIGH → LOW), `/` = rising edge (LOW → HIGH).
- `RD#` or $\overline{RD}$ means active-LOW. The signal does its job when at 0V.
- **C1:** Address placed, RD# goes LOW (transaction starts).
- **C2–C3:** Wait State HIGH = device not ready, CPU frozen.
- **C4:** Wait State drops LOW, data appears on Data Bus.
- **C5:** CPU samples data at clock edge, RD# goes HIGH (transaction ends).
- **Wait states are NOT fixed.** They depend on device speed, link speed, distance, and load.
- **PCIe uses differential pairs** (two wires per lane) for noise immunity.
- **AXI is a split-transaction protocol.** CPU can issue multiple requests without waiting for responses.
- **RAM has no visible wait states** — memory controller absorbs delays internally.
- **Setup time:** Data must be stable BEFORE clock edge (typically 0.1–1 ns).
- **Hold time:** Data must be stable AFTER clock edge (typically 0.05–0.5 ns).
- **Violating setup/hold causes metastability** — intermittent, temperature-dependent bugs.
- **Address bus must stay stable** throughout the entire transaction until RD# goes HIGH.
- **CPU freezes via clock-enable gating,** not by stopping the clock.

Part J: One-Page Visual Summary

```
+-----------------------------------------------------------------+
|              TIMING DIAGRAMS: THE COMPLETE PICTURE               |
+-----------------------------------------------------------------+
|                                                                 |
|   SYMBOLS:                                                      |
|   • ___ = LOW (0V)    • ¯¯¯ = HIGH (3.3V/5V)                   |
|   • |   = transition  • < > = valid data window               |
|   • \   = falling edge• /   = rising edge                      |
|                                                                 |
+-----------------------------------------------------------------+
|                                                                 |
|   MMIO READ WITH WAIT STATES:                                   |
|                                                                 |
```



This completes the Section 3 research package on Hardware Timing Diagrams. Every symbol is decoded, every cycle is traced, every electrical reality is exposed, and nothing is left for you to "figure out."

Section 4 — Modern CPU Implementations & Case Studies

Part A: Intel / AMD (x86-64) and Resizable BAR

A.1 The Historical Problem — The 256 MB Window

To understand why Resizable BAR (ReBAR) exists, you must first understand the problem it solves. This requires going back to 1992.

A.1.1 The PCI Bus Era (1992) In 1992, Intel created the **PCI (Peripheral Component Interconnect)** bus specification. This was the standard for connecting expansion cards to the motherboard.

What the world looked like in 1992:

- A high-end PC had 4 MB to 16 MB of RAM.
- A "large" hard disk was 200 MB.
- A graphics card might have 1 MB of video memory.
- The CPU was a 32-bit Intel 486 running at 33 MHz.

The PCI specification designers made a decision:

> "Let's allocate up to 256 MB of MMIO address space for each device's BAR. That should be enough forever."

Why 256 MB?

- 32-bit address space = 4 GB total.
- System RAM needed most of that.
- 256 MB per device seemed enormous in 1992.
- Nobody imagined GPUs would ever need more than a few megabytes of VRAM.

A.1.2 The Problem in Practice Fast forward to 2020. A modern GPU has **24 GB of VRAM** (GDDR6X memory). But the PCI specification still limits each BAR to **256 MB**.

What this meant:

```
GPU VRAM: 24 GB total
BAR window visible to CPU: 256 MB
Hidden VRAM: 24 GB - 256 MB = 23.75 GB inaccessible directly
```

To access the rest of the VRAM, the driver had to do BAR remapping:

Step 1: Program BAR to point at VRAM chunk 0 (addresses 0 to 256 MB)

- CPU can access VRAM[0] to VRAM[256M].
- Do some work.

Step 2: Program BAR to point at VRAM chunk 1 (addresses 256 MB to 512 MB)

- CPU can access VRAM[256M] to VRAM[512M].
- Do some work.

Step 3: Program BAR to point at VRAM chunk 2

- Continue…

Step N: Repeat for all 24 GB / 256 MB = **96 chunks**

This is called "paging the VRAM window."

A.1.3 Why BAR Remapping is Painfully Slow Every remapping operation requires:

1. PCIe Configuration Write

- The CPU must write to the GPU's configuration space to reprogram the BAR.
- This is an MMIO write to a special register.
- It is one of the SLOWEST operations on the PCIe bus.
- Why slow? Because configuration space access requires special cycles, arbitration, and acknowledgment.

2. Cache Flush

- The old 256 MB window might have been cached by the CPU.
- When the BAR changes, those cached mappings are now pointing to the WRONG physical memory.
- The CPU must flush (invalidate) all cached entries for the old window.
- Cache flushes are expensive — they stall the CPU and waste bandwidth.

3. TLB Shootdown

- The CPU's Translation Lookaside Buffer (TLB) caches virtual-to-physical address mappings.
- When the BAR changes, the TLB entries for the old window are wrong.
- The OS must send TLB shootdown interrupts to all CPU cores.
- Every core must stop what it is doing and clear its TLB.
- This is EXTREMELY expensive on multi-core systems.

4. Actual Data Transfer

- FINALLY, after all that overhead, the CPU can read or write the new 256 MB chunk.
- But it will soon need another chunk, and the whole process repeats.

For a game engine uploading 3 GB of textures:

- 3 GB / 256 MB = 12 chunks.
- 12 remapping operations.
- 12 cache flushes.
- 12 TLB shootdowns.
- Massive overhead before a single pixel is rendered.

A.2 The Solution — Resizable BAR (ReBAR)

A.2.1 What ReBAR Does **Resizable BAR** is a PCIe capability that allows the BAR size to be much larger than 256 MB.

With ReBAR enabled:

- The BIOS reads the GPU's BAR capability register.
- The GPU says: "I can support BAR sizes up to 24 GB."
- The BIOS programs the BAR to 24 GB.
- The entire 24 GB of VRAM is mapped into the CPU's 64-bit MMIO address space as ONE contiguous region.

Visual comparison:

```
WITHOUT ReBAR (Legacy):
+-------------------------------------------------------------+
|  CPU MMIO Address Space                                      |
|  +---------++---------++---------++---------+              |
|  | 256 MB  || 256 MB  || 256 MB  || 256 MB  |  ... (96x)  |
|  | Chunk 0 || Chunk 1 || Chunk 2 || Chunk 3 |              |
|  | remap   || remap   || remap   || remap   |              |
|  +---------++---------++---------++---------+              |
|  Only ONE chunk visible at a time. Constant remapping.     |
+-------------------------------------------------------------+

WITH ReBAR:
+-------------------------------------------------------------+
|  CPU MMIO Address Space (64-bit)                             |
|  +-----------------------------------------------------+   |
|  |              24 GB GPU VRAM (one contiguous block)     |   |
|  |         No remapping needed. One BAR, one region.      |   |
|  +-----------------------------------------------------+   |
|                                                              |
|  Total 64-bit space: 16,000,000,000,000,000,000 bytes      |
|  GPU uses: 24,000,000,000 bytes                             |
|  Remaining: 15,999,999,999,976,000,000,000 bytes            |
|  (Giving 24 GB to the GPU is like a drop in the ocean)      |
+-------------------------------------------------------------+
```

A.2.2 Why 64-Bit Addressing Makes This Possible 32-bit address space:

- Maximum addressable memory = 2^{32} = 4,294,967,296 bytes = **4 GB**.
- You CANNOT fit 24 GB of VRAM + system RAM + other MMIO into 4 GB.
- 32-bit systems are physically incapable of ReBAR for large GPUs.

64-bit address space:

- Maximum addressable memory = 2^{64} = 18,446,744,073,709,551,616 bytes.
- This is **16 exabytes** (16 billion GB).
- Allocating 24 GB to one GPU is completely trivial.
- There is no practical limit on how many devices or how much memory you can map.

This is why ReBAR only became mainstream around 2020:

- 64-bit CPUs were common earlier, but BIOS support, OS support, and GPU firmware support all had to align.

- Intel calls it "Smart Access Memory" (marketing term, same underlying PCIe ReBAR standard).
- AMD also calls it "Smart Access Memory" (same standard, different marketing).

A.2.3 What ReBAR Looks Like in Practice With ReBAR active, the CPU can execute:

```
MOV  [0x0000_0000_1234_5678], RAX
```

What happens:

1. The address `0x0000_0000_1234_5678` falls within the GPU's BAR range.
2. The MCH recognizes this and routes the write over PCIe.
3. The data from RAX travels directly to the GPU's VRAM.
4. **No remapping. No cache flush. No TLB shootdown. One instruction, one write.**

The CPU can access ANY byte address within the 24 GB region directly.

A.2.4 Performance Impact Measured gains:

- Typical: **5% to 15% improvement** in frames per second for games.
- Some GPU/game combinations: **Up to 20% or more.**

Why the gain is NOT from faster individual writes:

- Each PCIe MMIO write is the same speed as before.
- PCIe 4.0 x16 bandwidth is still ~32 GB/s.
- The gain comes from **eliminating overhead:**

Eliminated overhead:

1. No BAR reprogramming.
2. No cache flushes between chunks.
3. No TLB shootdowns.
4. Driver can structure uploads more efficiently.
5. Larger contiguous blocks mean fewer transactions.

Analogy: Instead of mailing 96 separate envelopes (256 MB each) with a form to fill out for each one, you send one big package (24 GB) with no paperwork. The shipping truck moves at the same speed, but you eliminated all the administrative overhead.

A.3 What x86 Still Uses Isolated I/O For

x86 did NOT completely abandon Isolated I/O. The legacy port I/O space still exists.

Port I/O address range: `0x0000` to `0xFFFF` (65,536 ports)

Devices still on port I/O:

Real-Time Clock (RTC)

- Ports: `0x70` and `0x71`
- Used to read/write the system clock (date, time, CMOS settings)

- Why still on port I/O? Changing it would break BIOS firmware from the 1980s.

PS/2 Keyboard Controller

- Ports: `0x60` and `0x64`
- Even on modern boards, the USB keyboard is often emulated as a PS/2 device.
- The firmware expects port I/O access.

x87 FPU Error Port

- Port: `0xF0`
- Legacy floating-point unit error signaling.

BIOS/UEFI Firmware

- During system initialization (before the OS loads), the BIOS uses port I/O extensively.
- It configures devices, sets up BARs, initializes memory — all through port I/O.
- Only after boot does the OS switch to MMIO.

Why keep port I/O?

- **Backward compatibility.** Decades of BIOS code, firmware, and drivers rely on it.
- Modern CPUs implement `IN` and `OUT` instructions purely for compatibility.
- The rest of the system uses MMIO.

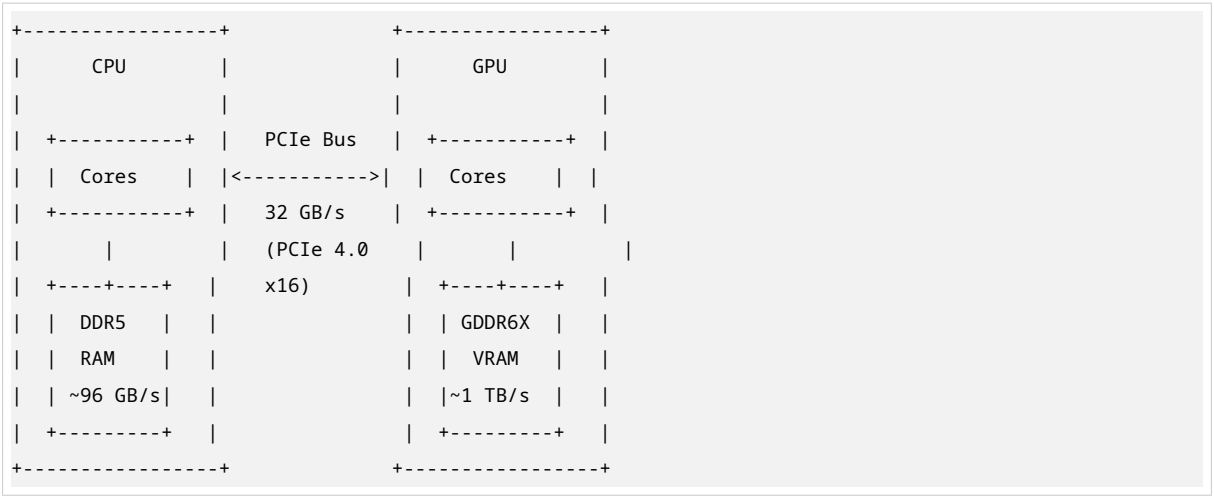
Part B: Apple Silicon — Unified Memory Architecture

B.1 What "Unified Memory" Actually Means

Apple's marketing calls it "unified memory," but this is not just a feature — it is a **fundamental architectural decision** made at the chip design stage.

To understand it, you must first understand the **conventional PC model** and its problems.

B.1.1 The Conventional PC Model (CPU and GPU Separate) In a standard PC, the CPU and GPU are **separate chips** with **separate memory:**



The CPU's memory:

- DDR5 dual-channel: ~96 GB/s bandwidth.
- Connected directly to CPU via memory bus.
- Used for programs, OS, data.

The GPU's memory:

- GDDR6X or HBM: ~1 TB/s bandwidth or more.
- Connected directly to GPU via wide memory interface.
- Used for textures, frame buffer, compute buffers.

The PCIe bus between them:

- PCIe 4.0 x16: ~32 GB/s.
- PCIe 5.0 x16: ~64 GB/s.
- This is the ONLY connection between CPU and GPU.

B.1.2 The Data Copy Problem When the CPU wants to send data to the GPU:

Step 1: CPU allocates a buffer in system RAM.

- Address: `0x10000000` to `0x1000FFFF` (example)

Step 2: CPU copies data into that buffer.

- `memcpy(buffer, data, size)` — data is now in system RAM.

Step 3: Driver issues a DMA transfer over PCIe.

- DMA (Direct Memory Access) is a hardware mechanism that copies data without CPU involvement.
- The DMA controller reads from system RAM and writes to GPU VRAM over PCIe.
- This consumes PCIe bandwidth.

Step 4: Wait for DMA transfer to complete.

- CPU polls or gets interrupted when DMA is done.
- During this time, the CPU might be idle or doing other work.

Step 5: GPU can now access the data in VRAM.

The problems with this model:

Problem 1: The copy is expensive.

- DMA transfer over PCIe is slow compared to memory speed.
- 32 GB/s PCIe vs 96 GB/s system RAM vs 1 TB/s VRAM.
- The copy becomes a bottleneck.

Problem 2: Data exists in two places.

- Same data in system RAM AND VRAM.
- Wastes memory capacity.
- Must keep both copies synchronized.

Problem 3: PCIe latency.

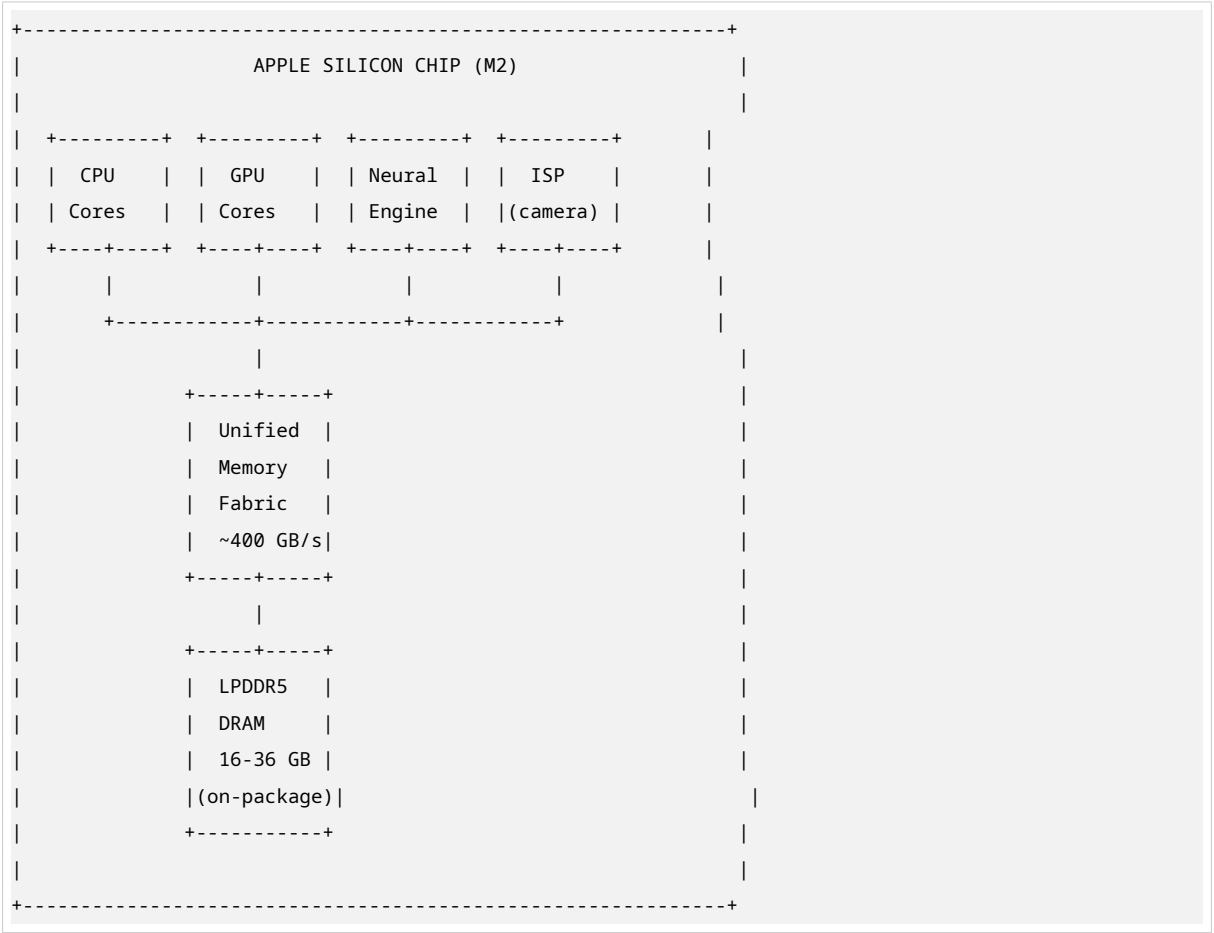
- Every MMIO access to GPU VRAM goes through PCIe.
- Thousands of clock cycles of latency (from Section 3).

- The CPU cannot efficiently work directly with GPU data.

B.2 What Apple Did Differently

Apple Silicon (M1, M2, M3, etc.) places the CPU cores, GPU cores, Neural Engine, image signal processor, and all other compute blocks on **a single piece of silicon**.

All of them connect to the SAME pool of LPDDR5 DRAM through the same high-bandwidth memory fabric.



There is NO separate VRAM. There is NO PCIe link between CPU and GPU. There is ONE pool of DRAM — 8 GB, 16 GB, 24 GB, or 36 GB. Every processing unit can address every byte directly.

B.2.1 What This Means in Practice When the CPU computes a result and the GPU needs it:

```
CPU writes to address 0x1A000000
GPU reads from address 0x1A000000
```

What does NOT happen:

- ❌ No DMA transfer.

- ❌ No PCIe transaction.
- ❌ No data copy.
- ❌ No MMIO stall.
- ❌ No cache flush.

What DOES happen:

- ✅ CPU writes to DRAM through the memory fabric.
- ✅ GPU reads from the SAME DRAM through the memory fabric.
- ✅ Only a POINTER was passed between driver code.
- ✅ Data never moved — only ownership/visibility changed.

The memory controller arbitrates access:

- Both CPU and GPU request access to DRAM simultaneously.
- The memory controller schedules requests fairly.
- This is the SAME latency as a CPU-to-CPU memory access.

B.3 The Trade-offs Apple Accepted

This architecture is NOT free. Apple made explicit trade-offs.

B.3.1 Trade-off 1: Lower Peak Memory Bandwidth Discrete GPU VRAM bandwidth:

- NVIDIA RTX 4090: ~1,008 GB/s (GDDR6X)
- AMD RX 7900 XTX: ~960 GB/s (GDDR6)

Apple Silicon unified memory bandwidth:

- M2 Max: ~400 GB/s
- M2 Ultra: ~800 GB/s

Why the difference?

- GDDR6X and HBM are designed for maximum bandwidth to a single die.
- They use very wide memory interfaces (384-bit, 512-bit) running at high frequency.
- LPDDR5 uses narrower interfaces (128-bit) but is more power-efficient.

The M2 Max GPU is memory-bandwidth-limited compared to a discrete GPU of equivalent compute power.

When does this matter?

- **Purely GPU-bound, bandwidth-hungry workloads:** Discrete GPU wins.
  - Example: Training massive neural networks, high-resolution 3D rendering.
- **Most real-world workloads:** Apple's approach wins.
  - Example: Video editing, photo processing, game development.
  - Why? Because data transfer overhead (copying between CPU and GPU) dominates.

B.3.2 Trade-off 2: No Memory Upgrades **The DRAM is physically packaged on the same module as the chip.**

```
+----------------------------------------+
|         Apple Silicon Package          |
|                                        |
|  +---------------------------------+   |
|  |         Logic Die (M2)          |   |
|  |   (CPU, GPU, Neural Engine)     |   |
|  +---------------------------------+   |
|                                        |
|  +---------+ +---------+ +---------+   |
|  | LPDDR5  | | LPDDR5  | | LPDDR5  |   |
|  |  DRAM   | |  DRAM   | |  DRAM   |   |
|  |  Chip   | |  Chip   | |  Chip   |   |
|  +---------+ +---------+ +---------+   |
|                                        |
|  This is called "Package on Package"   |
|  (PoP) — everything is soldered        |
|  together permanently.                 |
|                                        |
+----------------------------------------+
```

Why no upgrades?

- The tight integration requires DRAM to be physically close to the logic die.
- Signals travel shorter distances = lower latency and lower power.
- But the DRAM chips are soldered, not socketed.
- You buy 16 GB or 32 GB or 64 GB at purchase time — forever.

This is a direct consequence of the architecture. You cannot have unified memory with upgradeable DIMM slots because the distance and signal integrity would break the unified fabric.

B.4 What This Looks Like in RTL Terms

In Section 2, MMIO LOAD required:

```
T3: MAR <- R0                    (set up MMIO address)
T4: MCH intercepts -> PCIe       (HIDDEN STALL, thousands of cycles)
T5: R1 <- Bus_Data               (finally get data)
```

On Apple Silicon, CPU-to-GPU data sharing:

```
CPU:  Memory[0x1A000000] <- R3   (CPU writes to shared DRAM)
GPU:  V0 <- Memory[0x1A000000]   (GPU reads same address)
```

No MCH interception. No PCIe routing. No wait state from slow peripheral bus. Latency = DRAM access time + memory controller arbitration.

This is the SAME latency as CPU-to-CPU memory access.

Part C: ARM and AMBA — Where Everything Is MMIO by Definition

C.1 The ARM Philosophy

ARM processors have NEVER had Isolated I/O.

There is NO `IN` instruction. There is NO `OUT` instruction. There is NO M/IO pin. There is NO port address space.

From the ARM1 in 1985 to the Cortex-X4 in 2024, every peripheral, every register, every hardware control point is accessed by reading and writing a memory address.

Why did ARM do this?

Reason 1: Simplicity

- No separate I/O address space = simpler CPU design.
- Fewer transistors = lower power = longer battery life.
- This matters for mobile devices.

Reason 2: Smaller instruction set

- No `IN`/`OUT` instructions needed.
- Compiler is simpler — it generates normal loads and stores for everything.
- Code is more portable.

Reason 3: No legacy baggage

- x86 carries 1970s design decisions for backward compatibility.
- ARM was designed fresh in 1985 with no existing software to support.
- It could make the right decision from the start.

This is NOT a limitation. It is a deliberate design principle.

C.2 The AMBA Bus Family

AMBA = Advanced Microcontroller Bus Architecture

This is ARM's interconnect specification. It defines how CPU, memory, and peripherals communicate inside an ARM chip.

AMBA is not one bus — it is a FAMILY of buses:

C.2.1 APB — Advanced Peripheral Bus

Attribute	Value
Purpose	Low-bandwidth peripherals
Examples	UARTs (serial ports), GPIO, timers, watchdog timers, temperature sensors
Speed	Slowest AMBA bus. Not pipelined.
How it works	Each transaction completes fully before the next begins.

Clock	Runs at lower frequency than CPU.

When is APB used?

- Toggling an LED.
- Reading a temperature sensor.
- Setting a timer.
- Any simple, infrequent operation.

C.2.2 AHB — Advanced High-performance Bus

Attribute	Value
Purpose	Medium-bandwidth peripherals
Examples	DMA controllers, USB controllers, Ethernet MACs, external memory interfaces
Speed	Faster than APB. Supports burst transfers.
Burst transfers	Issue one address, then transfer multiple sequential data words without re-issuing the address each time.

When is AHB used?

- Bulk data transfers.
- DMA engine moving data.
- USB controller reading/writing packets.

C.2.3 AXI — Advanced eXtensible Interface

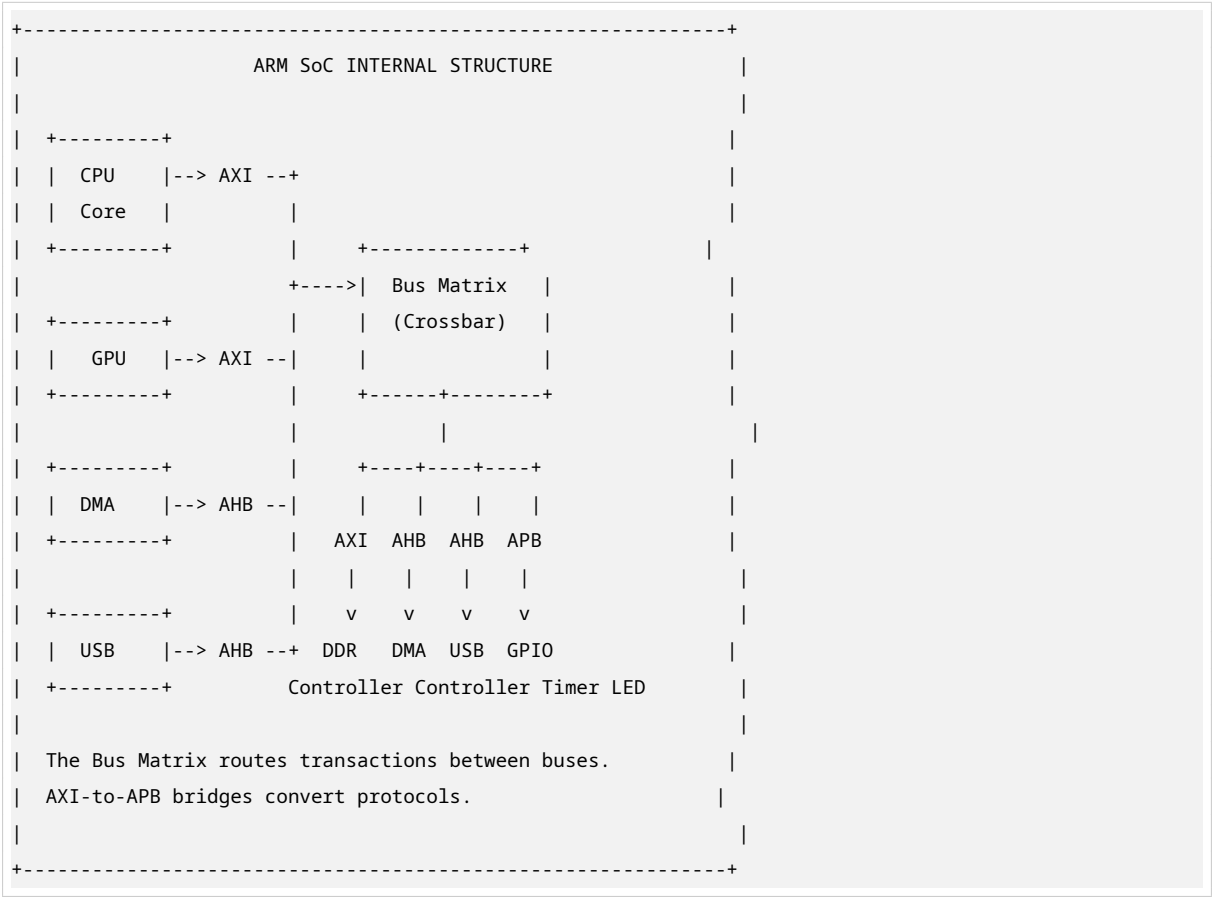
Attribute	Value
Purpose	High-performance CPU-to-memory and CPU-to-GPU connections
Speed	Fastest AMBA bus.
Key feature	Multiple outstanding transactions simultaneously.
Five independent channels	1. Read Address (AR) 2. Read Data (R) 3. Write Address (AW) 4. Write Data (W) 5. Write Response (B)

When is AXI used?

- CPU accessing main memory.
- GPU accessing frame buffer.

- Any high-bandwidth, low-latency connection.

C.2.4 How They Connect — The Bus Matrix Modern ARM SoCs use ALL THREE buses simultaneously:



The Bus Matrix (or Crossbar):

- A hardware router that connects multiple masters (CPU, GPU, DMA) to multiple slaves (memory, peripherals).
- It arbitrates conflicts: if CPU and DMA both want memory access, the matrix decides who goes first.
- It converts between bus protocols (AXI to AHB, AHB to APB).

C.3 Concrete Example — Toggling an LED on an STM32

This is a REAL, RUNNABLE example on any STM32 microcontroller (ARM Cortex-M).

Hardware:

- STM32F4 chip
- GPIO Port A, Pin 5 (PA5) connected to onboard LED
- PA5 is controlled by the GPIO Output Data Register (ODR)

Memory map:

- GPIO Port A base address: `0x40020000`
- ODR offset: `0x14`
- Full ODR address: `0x40020000 + 0x14 = 0x40020014`

C.3.1 The C Code

```
*(volatile uint32_t *)(0x40020014) |= (1 << 5);
```

Let's unpack this ONE LINE completely:

Step 1: `0x40020014`

- This is the raw integer address of the GPIO ODR register.
- It is NOT a variable. It is a hardcoded memory address.

Step 2: `(uint32_t *)`

- Type cast: "Treat this integer as a pointer to a 32-bit unsigned integer."
- This tells the compiler: "There is a 32-bit value at this address."

Step 3: `(volatile uint32_t *)`

- The `volatile` keyword is CRITICAL.
- It tells the compiler: "DO NOT optimize this access."

What `volatile` prevents:

Without `volatile`, the compiler might:

- Cache the register value in a CPU register and never re-read it.
- Eliminate a write if it thinks the value is overwritten later.
- Reorder reads and writes for optimization.

Why this is catastrophic for MMIO:

- Device registers change independently of the CPU.
- A GPIO register reflects the physical state of a pin.
- A status register updates when hardware events occur.
- If the compiler caches or eliminates accesses, the hardware and software get out of sync.

With `volatile`:

- EVERY read in source code becomes a real `LDR` instruction.
- EVERY write in source code becomes a real `STR` instruction.
- No caching. No elimination. No reordering.

C.3.2 The Assembly Code

The compiler generates this ARM assembly:

```
LDR R0, =0x40020014    ; Load the MMIO address into register R0
LDR R1, [R0]           ; Read current value from MMIO address
ORR R1, R1, #0x20      ; Set bit 5 (0x20 = 0b00100000)
STR R1, [R0]           ; Write modified value back to MMIO address
```

Instruction by instruction:

`LDR R0, =0x40020014`

- Load the immediate value `0x40020014` into register R0.
- R0 now holds the GPIO ODR address.

`LDR R1, [R0]`

- Load from memory address in R0 into R1.
- This is a READ from MMIO address `0x40020014`.
- R1 now holds the current ODR value.

`ORR R1, R1, #0x20`

- Bitwise OR R1 with `0x20` (binary `00100000`).
- This sets bit 5 to 1 without changing other bits.
- R1 now has bit 5 = 1.

`STR R1, [R0]`

- Store R1 to memory address in R0.
- This is a WRITE to MMIO address `0x40020014`.
- The GPIO controller sees this write.

C.3.3 What Happens at the Hardware Level When `STR R1, [R0]` executes:

Step 1: CPU puts `0x40020014` on the Address Bus.

Step 2: CPU puts the new ODR value (with bit 5 = 1) on the Data Bus.

Step 3: CPU asserts write control signals.

Step 4: The AXI bus fabric routes this write to the GPIO peripheral controller.

Step 5: The GPIO controller sees the write to its ODR register.

Step 6: The GPIO controller changes the voltage on pin PA5 from 0V to 3.3V.

Step 7: Current flows through the LED. It lights up.

Total chain: One C source line → four assembly instructions → one AXI write transaction → physical voltage change on a pin.

C.4 ARM vs x86 MMIO — The Key Structural Difference

```
+-------------------------------------------------------------+
|              WHERE THE COMPLEXITY LIVES                     |
+-------------------------------------------------------------+
|                                                             |
|   x86 COMPLEXITY:                                           |
|   ---------------                                           |
|   • Two separate address spaces (memory + I/O)              |
|   • Two sets of control signals (MEMR/MEMW vs IOR/IOW)      |
|   • Special instructions (IN/OUT) for I/O space             |
```

```
|   • CPU knows the difference between memory and I/O           |
|   • M/IO̅ pin on the CPU package                              |
|   • More transistors, more power, backward compatibility     |
|                                                               |
|   ARM COMPLEXITY:                                             |
|   ---------------                                             |
|   • One unified address space                                 |
|   • One set of control signals (loads and stores)             |
|   • No special I/O instructions                              |
|   • CPU is simple — just loads and stores                     |
|   • Complexity lives in the SoC bus fabric (AMBA)             |
|   • Memory map configured by BIOS/UEFI at boot                |
|   • Fewer transistors, less power, simpler compiler           |
|                                                               |
|   WHY THE DIFFERENCE?                                         |
|   • x86: Designed 1970s, evolved for PC compatibility         |
|   • ARM: Designed 1985, fresh start, mobile/embedded focus    |
|                                                               |
+---------------------------------------------------------------+
```

Part D: What Your Tutorial Did Not Cover

D.1 PCIe Configuration Space vs BAR Space

There are TWO MMIO regions for every PCIe device:

Region 1: Configuration Space

- **Purpose:** Where the BARs themselves are programmed.
- **How accessed:** Special configuration cycles.
- **On x86:** Traditionally through port I/O at ports `0xCF8` and `0xCFC`.
- **What it contains:**
  - Vendor ID (who made the device)
  - Device ID (what model it is)
  - BAR registers (where the BAR space starts and how big it is)
  - Command register (enable/disable MMIO, bus mastering, etc.)
  - Status register (error flags)

Region 2: BAR Space

- **Purpose:** Where the actual device registers and memory live.
- **How accessed:** Normal MMIO reads and writes.
- **What it contains:**
  - Device control registers
  - Device status registers
  - Device memory (like GPU VRAM)

When BIOS enables ReBAR:

1. BIOS reads the GPU's configuration space.
2. It finds the BAR capability register that says "I can support up to 24 GB."

3. It writes to the configuration space to set the BAR size to 24 GB.
4. It maps that 24 GB region into the CPU's MMIO address space.
5. The GPU's BAR space is now accessible at those addresses.

D.2 Why Apple Silicon's Approach Does Not Work for Discrete GPUs

A discrete GPU is a separate chip connected over a PCIe slot.

PCIe is a SERIAL interface:

- PCIe 5.0 x16: ~64 GB/s maximum
- PCIe 6.0 x16: ~128 GB/s maximum

A high-end GPU needs ~1 TB/s of memory bandwidth.

- 1 TB/s = 1,000 GB/s
- PCIe 5.0 x16 provides 64 GB/s
- **PCIe is 15x too slow to serve as a GPU memory bus.**

Why Apple's approach works for them:

- CPU and GPU are on the SAME die.
- They connect to memory controllers via on-chip interconnects.
- On-chip interconnects run at HUNDREDS of GB/s.
- Distance is micrometers, not centimeters.
- No PCIe serialization overhead.

Why it cannot work for discrete GPUs:

- The GPU is a separate chip on a separate card.
- It MUST have its own high-bandwidth memory (GDDR6X, HBM).
- The CPU accesses that memory through PCIe MMIO — slow but necessary.
- There is no physical way to share memory at 1 TB/s over a PCIe cable.

D.3 The Memory Map is Set Up by BIOS/UEFI at Boot

Before the OS starts, the BIOS does the following:

Step 1: PCIe Bus Enumeration

- BIOS walks the PCIe bus, discovering every device.
- It reads each device's configuration space.
- It learns: "This is a GPU. It needs 24 GB of BAR space."

Step 2: BAR Assignment

- BIOS assigns non-overlapping MMIO address ranges to each device.
- It writes these assignments back into each device's BAR registers.
- Example: GPU gets `0x0000_0001_0000_0000` to `0x0000_0001_5FFF_FFFF` (24 GB)

Step 3: ACPI Tables

- BIOS creates ACPI (Advanced Configuration and Power Interface) tables.
- These are data structures left in RAM for the OS to find.

- They describe: “Device X is at MMIO address Y. Device Z is at MMIO address W.”

Step 4: OS Inheritance

- When the OS boots, it reads the ACPI tables.
- It learns the entire hardware layout without rediscovering everything.
- The OS’s PCIe driver uses this map to know where each device lives.

> [!IMPORTANT] **The OS does NOT randomly assign MMIO addresses.** It inherits the BIOS’s map.

D.4 GPIO is the Simplest Proof That MMIO Works

If you ever doubt that writing to a memory address controls physical hardware:

Buy an STM32 development board. (Costs about $3-$10) **Connect an LED to pin PA5. Run this code:**

```
*(volatile uint32_t *)(0x40020014) |= (1 << 5);
```

Watch the LED turn on.

There is no better way to make the abstraction concrete.

You wrote to a memory address. A physical voltage changed. A light turned on.

> [!NOTE] **This is the entire foundation of MMIO, proven in four lines of code.**

Part E: Complete Math Practice

E.1 Calculate Number of BAR Remapping Operations

Given:

- GPU VRAM: 24 GB
- Legacy BAR size: 256 MB
- Texture upload: 3 GB

Step 1: Convert everything to the same unit.

- 24 GB = 24,576 MB
- 3 GB = 3,072 MB

Step 2: Calculate chunks for full VRAM access.

$$\text{Chunks} = \frac{24{,}576 \text{ MB}}{256 \text{ MB}} = \textbf{96 remapping operations}$$

Step 3: Calculate chunks for 3 GB texture upload.

$$\text{Chunks} = \frac{3{,}072 \text{ MB}}{256 \text{ MB}} = \textbf{12 remapping operations}$$

Answer: Accessing the full 24 GB requires 96 remaps. Uploading 3 GB of textures requires 12 remaps.

E.2 Calculate ReBAR Address Range

Given:

- ReBAR base address: `0x0000_0001_0000_0000`
- ReBAR size: 24 GB

Step 1: Convert 24 GB to hex.

- 24 GB = 24 × 1024 × 1024 × 1024 bytes
- 24 GB = 25,769,803,776 bytes
- In hex: `0x0000_0000_6000_0000`

Step 2: Calculate end address.

$$\text{End} = \text{Base} + \text{Size} - 1$$

$$\text{End} = 0x0000\_0001\_0000\_0000 + 0x0000\_0000\_6000\_0000 - 1$$

End = $\mathbf{0x0000}_0001_5FFF_FFF\mathbf{0x0000}_0001_5FFF_FFF\mathbf{0x0000}_0001_5FFF_FFF\mathbf{0x0000}_0001_5FFF_FFF\mathbf{Answer:\ The\ GPU's\ BAR\ space\ c}$

E.3 Calculate Apple Silicon Memory Efficiency

Given:

- M2 Max memory bandwidth: 400 GB/s
- Discrete GPU (RTX 4090) VRAM bandwidth: 1,008 GB/s
- Data transfer needed: 10 GB from CPU to GPU

Scenario A: Discrete GPU (PCIe 4.0 x16)

- PCIe bandwidth: 32 GB/s
- Transfer time = 10 GB / 32 GB/s = **0.3125 seconds**
- During transfer, GPU cannot start work.
- Plus overhead: cache flush, TLB shootdown, BAR remapping.

Scenario B: Apple Silicon (Unified Memory)

- No transfer needed — data already in shared DRAM.
- Transfer time = **0 seconds**
- GPU can start reading immediately after CPU writes.

Bandwidth comparison:

$$\frac{\text{M2 Max}}{\text{RTX 4090}} = \frac{400}{1008} = \mathbf{39.7\%} \text{ of discrete GPU bandwidth}$$

- But for CPU-GPU data sharing, Apple is infinitely faster (no transfer delay).

Answer: Apple Silicon has ~40% of the raw bandwidth but eliminates transfer overhead entirely.

Part F: Key Points to Remember

- **Legacy PCI limited BARs to 256 MB.** Modern GPUs have 24 GB+ VRAM, requiring constant remapping without ReBAR.
- **ReBAR exposes the entire VRAM as one contiguous MMIO region.** Eliminates remapping, cache flushes, and TLB shootdowns.
- **ReBAR requires 64-bit addressing.** 32-bit systems cannot support it (4 GB limit).
- **x86 still uses Isolated I/O for legacy devices** (RTC, PS/2 keyboard, BIOS firmware).
- **Apple Silicon uses Unified Memory Architecture.** CPU, GPU, and all engines share one DRAM pool. No copies, no PCIe, no MMIO stalls.
- **Apple's trade-off:** Lower peak bandwidth (~400 GB/s vs ~1 TB/s) but zero transfer overhead. DRAM is soldered — no upgrades.
- **ARM has NEVER had Isolated I/O.** Everything is MMIO. This is simpler and more power-efficient.
- **AMBA is ARM's bus family:** APB (slow peripherals), AHB (medium bandwidth), AXI (high performance, split transactions).
- **`volatile` in C is CRITICAL for MMIO.** It prevents compiler optimization from eliminating or caching hardware accesses.
- **BIOS/UEFI sets up the MMIO map at boot** and passes it to the OS through ACPI tables.
- **GPIO LED toggle is the simplest proof of MMIO.** Write to `0x40020014`, physical pin changes voltage.

Part G: One-Page Visual Summary

```
+-----------------------------------------------------------------+
|         MODERN CPU IMPLEMENTATIONS: THE BIG PICTURE              |
+-----------------------------------------------------------------+
|                                                                 |
|   x86-64 WITH REBAR:                                            |
|   • Legacy: 256 MB BAR window, constant remapping               |
|   • ReBAR: Full VRAM exposed, one contiguous MMIO region        |
|   • Still has legacy port I/O for BIOS/RTC/keyboard             |
|   • 64-bit addressing makes 24 GB GPU mapping trivial           |
|                                                                 |
|   APPLE SILICON (UNIFIED MEMORY):                              |
|   • CPU + GPU + engines on same die                              |
|   • One shared DRAM pool, no separate VRAM                       |
|   • No PCIe between CPU and GPU                                 |
|   • No data copies — just pass pointers                          |
|   • Trade-off: Lower peak bandwidth, no upgrades                 |
|                                                                 |
|   ARM / AMBA:                                                   |
|   • No Isolated I/O ever — everything is MMIO                    |
|   • AXI: High-performance split-transaction bus                 |
|   • APB: Simple bus for GPIO, timers, UARTs                     |
|   • `volatile` keyword is essential for MMIO in C              |
```

```
|                                                                  |
|   COMMON THREAD:                                                 |
|   • All modern systems use MMIO as the primary I/O mechanism     |
|   • Isolated I/O is legacy/deprecated everywhere except x86 BIOS |
|   • The trend is toward tighter integration (SoC, unified mem)   |
|                                                                  |
+------------------------------------------------------------------+
```

This completes the Section 4 research package on Modern CPU Implementations & Case Studies. Every real-world system is built from first principles, every trade-off is explained explicitly, every math problem is solved step-by-step, and nothing is left hidden.

Section 5 — Mathematical Modeling of I/O Performance

Part A: Why Math Here?

A.1 The Difference Between Theory and Measurement

Sections 1 through 4 told you **WHAT** happens:

- How I/O mapping works.
- How micro-operations execute.
- How timing diagrams show electrical signals.
- How modern CPUs implement these ideas.

This section tells you HOW TO MEASURE AND PREDICT what happens.

Why this matters:

An engineer who only knows theory:

- Can describe how PCIe works.
- Cannot tell you if PCIe Gen 4 is fast enough for your GPU.
- Cannot compare two bus designs quantitatively.
- Cannot predict if adding more CPU cores will help.

A researcher who knows the math:

- Can calculate exact bandwidth ceilings.
- Can predict speedup limits before buying hardware.
- Can identify the real bottleneck in any system.
- Can make architectural decisions based on numbers, not guesses.

This section gives you two formulas that are the entry point to all I/O performance analysis.

Part B: Formula 1 — Peak Bandwidth Calculation

B.1 The Formula

$$B = W \times F \times T \times E$$

What this formula calculates:

> The maximum theoretical data rate across a bus under ideal conditions.

What it does NOT calculate:

> Real-world sustained throughput (which is always lower due to overhead).

B.2 Variable B — Bandwidth in Bytes Per Second

Definition: The maximum amount of data that can flow across the bus per second under ideal conditions.

Units: Bytes per second (B/s), or more commonly MB/s, GB/s, TB/s.

Why "peak" and "theoretical" matter:

- Real-world bandwidth is ALWAYS lower.
- Protocol overhead, contention, and inefficiencies reduce actual throughput.
- This formula gives you the **ceiling** — the absolute maximum possible.
- You will never reach this ceiling, but you need to know where it is.

Analogy: The speed limit on a highway is 120 km/h. That is the theoretical maximum. In reality, traffic, weather, and road conditions mean you average 80 km/h. But you still need to know the speed limit exists.

B.3 Variable W — Bus Width in Bytes

Definition: How many bytes the bus can carry in parallel in a single transfer.

How it is determined: By the number of physical data wires.

Important distinction: W is NOT the same for all bus types.

B.3.1 PCIe Bus Width (Serial — Different From Parallel) **PCIe is a SERIAL bus.** It does NOT have parallel data wires like old PCI.

Instead, PCIe has LANES:

One lane = one differential pair for transmit + one for receive

- Each lane carries 1 bit per transfer at the physical layer.
- A PCIe x16 link has 16 lanes.
- 16 lanes × 1 bit = 16 bits per transfer = **2 bytes per transfer**.

BUT — there is a simplification:

For textbook calculations, PCIe x16 is often treated as:

- **W = 16 bytes per transfer** (at the link layer level).

Why 16 bytes and not 2 bytes?

- At the link layer, PCIe bundles multiple physical transfers.
- The "effective width" accounts for how the serial stream maps to usable data.
- This is a convention that simplifies calculations while giving correct results.

If you want to be precise at the physical layer:

- W = 2 bytes (16 bits) per transfer.
- F = 16 GT/s (gigatransfers per second).
- Then you get the same final answer, just with different intermediate steps.

For this tutorial, we use the simplified W = 16 bytes for PCIe x16.

B.3.2 Traditional Parallel Bus Width (AHB on ARM) **AHB is a PARALLEL bus.** It has actual parallel data pins.

32-bit AHB bus:

- 32 data pins = 32 bits = 4 bytes.
- **W = 4 bytes**.

64-bit AHB bus:

- 64 data pins = 64 bits = 8 bytes.
- **W = 8 bytes**.

This is straightforward: count the data pins, divide by 8, get bytes.

B.4 Variable F — Clock Frequency in Hz

Definition: The frequency of the BUS clock, NOT the CPU clock.

This is the MOST COMMON MISTAKE. Students confuse CPU frequency with bus frequency.

Examples of the difference:

Desktop CPU:

- CPU runs at 5 GHz internally.
- PCIe bus runs at its own reference clock.
- PCIe Gen 4: 16 GT/s per lane (this is the bus frequency).
- These are COMPLETELY DIFFERENT clocks.

STM32 Microcontroller:

- CPU runs at 168 MHz.
- AHB bus might be divided down to 84 MHz.
- You MUST use 84 MHz for the bus formula, not 168 MHz.

Why different clocks?

- The CPU has its own PLL (Phase-Locked Loop) for maximum speed.
- The bus has a separate PLL or divider for stability and compatibility.
- Running the bus at CPU speed would cause electrical problems (noise, heat, signal integrity).

Always use the BUS clock frequency in the formula.

B.5 Variable T — Transfers Per Clock

Definition: How many data transfers happen per clock cycle.

Two possibilities:

T = 1: Single Data Rate (SDR)

- Data transfers on the RISING edge only.
- One transfer per clock cycle.
- Example: AHB bus, most simple peripherals.

T = 2: Double Data Rate (DDR)

- Data transfers on BOTH rising AND falling edges.
- Two transfers per clock cycle.
- Example: DDR RAM (hence the name), some high-speed interfaces.

Why use DDR?

- You get TWICE the data throughput from the SAME clock frequency.
- Higher clock frequencies cause:
  - More electromagnetic emissions (radio interference).
  - More heat.
  - Tighter timing margins (harder to design).
- DDR achieves speed without these problems.

PCIe note:

- PCIe uses GT/s (gigatransfers per second) as its specification.
- The GT/s number ALREADY accounts for per-edge transfers.
- When using GT/s directly for F, T = 1 (because the "per-second" rate is already the transfer rate).
- Do NOT double-count by setting T = 2 when using GT/s.

B.6 Variable E — Encoding Efficiency

Definition: The fraction of raw bits on the wire that are actual payload data.

Why encoding is needed:

If a wire sits at constant voltage (all 0s or all 1s) for too long:

- The receiver LOSES clock synchronization.
- It cannot tell the difference between "ten zeros were sent" and "the wire is broken."
- To prevent this, data is ENCODED before transmission to guarantee frequent transitions.

PCIe encoding schemes:

PCIe Gen 1 and Gen 2: 8b/10b Encoding

- Every 8 bits of real data → encoded as 10 bits on the wire.
- Efficiency: E = 8/10 = **0.80**.
- Meaning: 20% of bandwidth is consumed by encoding overhead.
- For every 10 bits transmitted, only 8 are payload.

PCIe Gen 3 and above: 128b/130b Encoding

- Every 128 bits of real data → encoded as 130 bits on the wire.
- Efficiency: E = 128/130 **0.9846**.
- Meaning: Only ~1.54% overhead.
- For every 130 bits transmitted, 128 are payload.

Why the improvement matters:

- Gen 3 switched from 8b/10b to 128b/130b.
- This alone gave a **23% efficiency boost** (0.9846 / 0.80 = 1.23).
- Combined with higher raw speed, bandwidth more than doubled.

E is always between 0 and 1:

- E = 1.0 would mean perfect encoding with no overhead (impossible in practice).
- The closer to 1.0, the more efficient.

On-chip buses (like AHB):

- E = 1.0 (no encoding overhead).
- Why? Everything is on the same die, running off the same clock.
- No clock synchronization problem exists.

B.7 Complete Worked Example — PCIe Gen 4 x16

Problem: Calculate the peak bandwidth of a PCIe Gen 4 x16 link.

Given:

- PCIe Gen 4: 16 GT/s per lane.
- Link width: x16 (16 lanes).
- Encoding: 128b/130b.

Step 1: Identify variables

W = 16 bytes

- 16 lanes, treated as 16 bytes effective width at link layer.

F = 16 × 10 transfers/second

- 16 GT/s = 16 gigatransfers per second.
- 16 × 10 Hz.

T = 1

- Using GT/s directly, so transfers per second is already the rate.
- Do not double-count with T = 2.

E = 128/130 0.9846

- 128b/130b encoding efficiency.

Step 2: Substitute into formula

$$B = W \times F \times T \times E$$

$$B = 16 \times (16 \times 10^9) \times 1 \times 0.9846$$

Step 3: Calculate step by step

First: $16 \times 16 = 256$

Then: $256 \times 10^9 = 256{,}000{,}000{,}000$

Then: $256{,}000{,}000{,}000 \times 0.9846 = 252{,}057{,}600{,}000$

Step 4: Convert to GB/s

$$B \approx 252.1 \times 10^9 \text{ bytes/sec}$$

$$B \approx \mathbf{252 \ \ GB/s}$$

Verification: The official PCIe Gen 4 x16 specification lists ~252 GB/s unidirectional. **Our calculation matches exactly.**

B.8 Complete Worked Example — PCIe Gen 3 x16 (For Comparison)

Problem: Calculate peak bandwidth of PCIe Gen 3 x16.

Given:

- PCIe Gen 3: 8 GT/s per lane.
- Link width: x16.
- Encoding: 8b/10b (Gen 3 actually uses 128b/130b, but let's use 8b/10b for comparison as the tutorial suggests).

Step 1: Identify variables

W = 16 bytes

F = 8 × 10 transfers/second

T = 1

E = 8/10 = 0.80

Step 2: Substitute

$$B = 16 \times (8 \times 10^9) \times 1 \times 0.80$$

Step 3: Calculate

First: $16 \times 8 = 128$

Then: $128 \times 10^9 = 128{,}000{,}000{,}000$

Then: $128{,}000{,}000{,}000 \times 0.80 = 102{,}400{,}000{,}000$

Step 4: Convert to GB/s

$$B = 102.4 \text{ GB/s}$$

Answer: PCIe Gen 3 x16 = **102.4 GB/s**.

B.9 Comparison: Why Gen 4 More Than Doubled Gen 3

Raw GT/s comparison:

- Gen 3: 8 GT/s.

- Gen 4: 16 GT/s.
- Raw speed doubled: 16 / 8 = 2×.

Bandwidth comparison:

- Gen 3: 102.4 GB/s.
- Gen 4: 252 GB/s.
- Bandwidth ratio: 252 / 102.4 = **2.46×**.

Why did bandwidth more than double (2.46× vs 2×)?

Encoding improvement contributed extra:

$$\frac{E_{Gen4}}{E_{Gen3}} = \frac{0.9846}{0.80} = 1.23$$

The encoding improvement alone gave a FREE 23% boost on top of the raw speed doubling.

Total improvement:

- Raw speed doubling: 2×.
- Encoding efficiency boost: 1.23×.
- Combined: 2 × 1.23 = **2.46×**.

This is why the encoding efficiency term (E) matters and CANNOT be ignored.

B.10 Complete Worked Example — AHB on STM32 Microcontroller

Problem: Calculate peak bandwidth of AHB1 bus on STM32F4.

Given:

- AHB1 bus clock: 168 MHz (but verify — some STM32F4 variants divide AHB to 168 MHz, others to lower).
- Bus width: 32 bits = 4 bytes.
- Single data rate (SDR).
- On-chip bus = no encoding overhead.

Step 1: Identify variables

W = 4 bytes

- 32-bit bus = 4 bytes.

F = 168 × 10 Hz

- 168 MHz = 168,000,000 cycles per second.

T = 1

- Single data rate.

E = 1.0

- No encoding overhead on-chip.

Step 2: Substitute

$$B = 4 \times (168 \times 10^6) \times 1 \times 1.0$$

Step 3: Calculate

$$B = 4 \times 168{,}000{,}000$$

$$B = 672{,}000{,}000 \text{ bytes/sec}$$

Step 4: Convert to MB/s

$$B = \mathbf{672 \ \ MB/s}$$

Context: This is the maximum rate for GPIO, UART, SPI, I2C, or any AHB-connected peripheral. It is ~375 times slower than PCIe Gen 4 x16 (252,000 MB/s / 672 MB/s 375). But it consumes a fraction of the power — the right trade-off for a battery-powered microcontroller.

Part C: Formula 2 — Amdahl's Law Applied to I/O

C.1 The Formula

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

What this formula answers:

> If I make part of my program faster, how much faster does the WHOLE program get?

The brutal truth it reveals:

> **Not as much as you think, because the part you did NOT speed up becomes the new bottleneck.**

C.2 Variable S — Speedup

Definition: The ratio of old execution time to new execution time.

What S = 2 means: The program now runs twice as fast.

What S = 10 means: Ten times as fast.

What S = 1 means: No improvement at all.

Example:

- Old time: 100 seconds.
- New time: 20 seconds.
- S = 100 / 20 = 5.

C.3 Variable P — Parallel Fraction

Definition: The fraction of total execution time that CAN be made faster.

Expressed as: A decimal between 0 and 1.

Example:

- If 70% of time is in optimizable code: $\mathbf{P = 0.70}$.
- If 30% of time is in optimizable code: $\mathbf{P = 0.30}$.

What is (1 − P)?

- This is the **SERIAL FRACTION** — the part you CANNOT speed up.
- If P = 0.70, then (1 − P) = 0.30.
- 30% of the time is spent in code that is inherently serial.

In the I/O context:

- The serial fraction (1 − P) is dominated by **BLOCKING MMIO READS.**
- This is time the CPU spends FROZEN, waiting for a slow PCIe device.
- You cannot parallelize waiting. The CPU is stalled.
- This time is irreducibly serial.

C.4 Variable N — Speedup Factor Applied to Parallel Fraction

Definition: How much faster you made the optimizable part.

Examples:

- Added 4 CPU cores to parallelize computation: $\mathbf{N = 4}$.
- Upgraded PCIe Gen 3 to Gen 4 (doubled bandwidth): $\mathbf{N = 2}$ for bandwidth-bound portion.
- Moved to Apple Silicon (eliminated PCIe transfer): $\mathbf{N =}$ **very large** for that specific fraction.

C.5 The Mathematical Consequence Nobody Expects

Scenario Setup:

- 90% of time is parallelizable computation: $\mathbf{P = 0.90}$.
- 10% of time is blocking MMIO reads: $\mathbf{(1 - P) = 0.10}$.
- You buy a processor with 10 cores: $\mathbf{N = 10}$.

Step 1: Substitute into formula

$$S = \frac{1}{(1 - 0.90) + \frac{0.90}{10}}$$

Step 2: Calculate the denominator

First: $(1 - 0.90) = 0.10$

Then: $\frac{0.90}{10} = 0.09$

Then: $0.10 + 0.09 = 0.19$

Step 3: Calculate speedup

$$S = \frac{1}{0.19} \approx 5.26$$

Answer: You added **10× the compute power** and got only **5.26× speedup.**

The 10% serial I/O fraction ate almost HALF your potential gain.

C.6 Pushing to 100 Cores — The Wall Appears

Same scenario, but N = 100:

Step 1: Substitute

$$S = \frac{1}{0.10 + \frac{0.90}{100}}$$

Step 2: Calculate denominator

$$\frac{0.90}{100} = 0.009$$

$$0.10 + 0.009 = 0.109$$

Step 3: Calculate speedup

$$S = \frac{1}{0.109} \approx 9.17$$

Answer: With **100 cores**, speedup is only **9.17×**.

Going from 10 cores to 100 cores (10× more hardware) improved speedup from 5.26× to only 9.17×.

The returns are DIMINISHING. Dramatically.

C.7 The Infinite Limit — Amdahl's Wall

What happens as N approaches infinity?

$$\lim_{N \to \infty} S = \frac{1}{1 - P}$$

For our scenario (P = 0.90):

$$\lim_{N \to \infty} S = \frac{1}{0.10} = 10$$

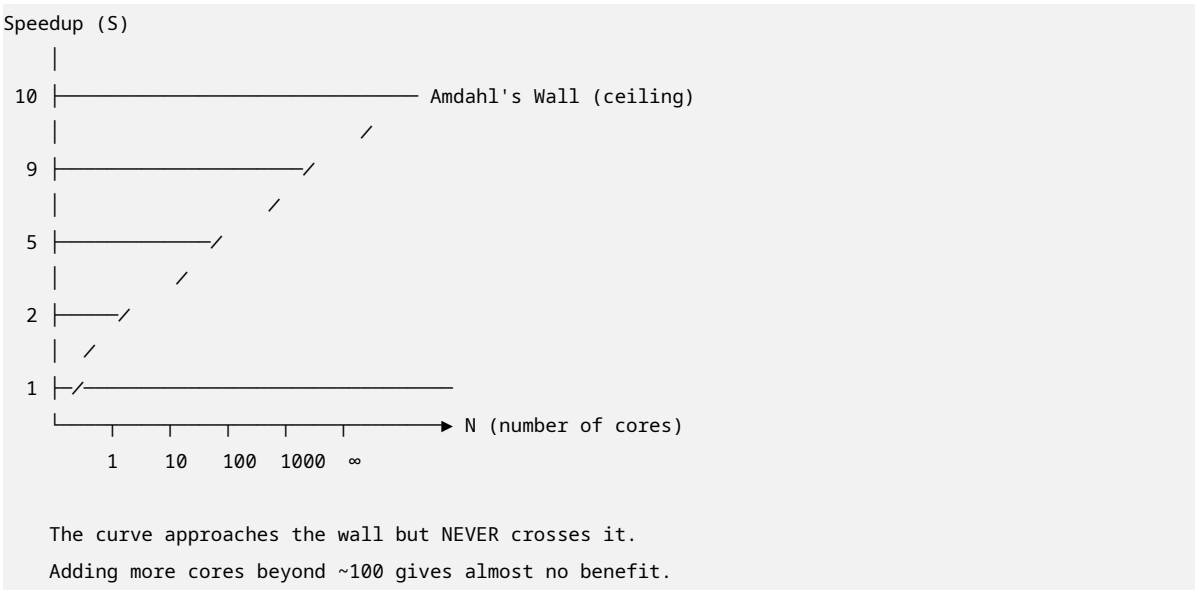
This is AMDAHL'S WALL:

> **No matter how many cores you add, a program with 10% blocking I/O can NEVER run more than 10× faster.**

Ten times. That is the HARD CEILING.

You hit it with INFINITE processors.

Visual representation:



Part D: Applying Amdahl's Law to Real I/O Scenarios

D.1 Scenario 1 — Deep Learning Inference on a GPU

The workload:

- Matrix multiplications on GPU: fast, massively parallel.
- Periodic reads of control registers/status flags over MMIO: blocking, serial.

Assumptions:

- MMIO overhead = 5% of total time.
- Therefore: $\mathbf{(1 - P) = 0.05}$, $\mathbf{P = 0.95}$.
- GPU has thousands of shader cores: $\mathbf{N = 1000}$.

Step 1: Substitute

$$S = \frac{1}{0.05 + \frac{0.95}{1000}}$$

Step 2: Calculate denominator

$$\frac{0.95}{1000} = 0.00095$$

$$0.05 + 0.00095 = 0.05095$$

Step 3: Calculate speedup

$$S = \frac{1}{0.05095} \approx 19.6$$

Step 4: Calculate theoretical maximum

$$\lim_{N \to \infty} S = \frac{1}{0.05} = 20$$

Analysis:

- With 1000 cores, speedup = 19.6×.
- Theoretical maximum = 20×.
- **You are already at 98% of the wall with 1000 cores.**
- Adding more shader cores buys ALMOST NOTHING.

The right engineering response:

- **Reduce the MMIO fraction (1 − P), not add more cores.**
- Batch status reads (read multiple registers at once).
- Use interrupts instead of polling (CPU does work while waiting).
- Move control logic on-chip (eliminate MMIO entirely).

If you reduce (1 − P) from 0.05 to 0.01:

- New wall: 1 / 0.01 = **100× speedup ceiling**.
- **That single architectural change is worth more than doubling shader cores.**

D.2 Scenario 2 — Real-Time Sensor Reading on Microcontroller

The workload:

- Read temperature sensors over I2C (MMIO-controlled peripheral).
- Process sensor readings.

Assumptions:

- Sensor reading (I2C) = 60% of total time.
- Computation = 40% of total time.
- Therefore: **(1 − P) = 0.60**, **P = 0.40**.
- Faster processor speeds up computation by 8×: **N = 8**.

Step 1: Substitute

$$S = \frac{1}{0.60 + \frac{0.40}{8}}$$

Step 2: Calculate denominator

$$\frac{0.40}{8} = 0.05$$

$$0.60 + 0.05 = 0.65$$

Step 3: Calculate speedup

$$S = \frac{1}{0.65} \approx 1.54$$

Analysis:

- Faster processor gives only **1.54× speedup.**
- 60% of time is spent waiting for I2C — which runs at its OWN fixed clock (100 kHz or 400 kHz).
- **Buying a faster CPU does NOT speed up I2C.**

The right engineering response:

- **Change the sensor interface, not the CPU.**
- Use SPI instead of I2C (SPI is faster: up to tens of MHz).
- Use DMA transfers (CPU does not block during sensor reads).
- Batch reads (read multiple sensors in one transaction).

Part E: What Your Tutorial Did Not Cover

E.1 Perfect Parallelism Assumption

Amdahl's Law assumes perfect parallelism within P.

Reality is worse:

- Threads wait for each other (synchronization).
- Cache coherency overhead (keeping multiple caches consistent).
- Lock contention (multiple threads fighting for the same resource).
- Load imbalance (some threads finish early and sit idle).

Real speedup is ALWAYS less than Amdahl predicts, even for P.

Gustafson's Law:

- A related formula that gives a MORE OPTIMISTIC view.
- It applies when the PROBLEM SIZE scales with the number of processors.
- For fixed-size problems, Amdahl's Law is the correct PESSIMISTIC bound.
- For research, use Amdahl for fixed workloads, Gustafson for scalable workloads.

E.2 Encoding Efficiency Compounds Across Protocol Layers

The E = 0.9846 for 128b/130b is ONLY the physical layer.

On top of that, PCIe has Transaction Layer Packet (TLP) overhead:

TLP Header:

- Every read or write request includes a header.
- Header size: 12 to 16 bytes.

- Contains: transaction type, address, length, requester ID.

Example — Small 4-byte MMIO read:

- Payload: 4 bytes.
- TLP header: 12 bytes.
- Total on wire: 16 bytes.
- Protocol efficiency: 4 / 16 = **0.25 = 25%**.

Meaning: Only 25% of bytes on the wire are actual data.

75% is protocol overhead.

This is why MMIO is inherently inefficient for small transfers.

Hardware designer response:

- Batch small register writes wherever possible.
- Combine multiple reads into one burst transaction.
- Use larger transfer sizes to amortize header overhead.

E.3 Peak vs Sustained Bandwidth

The bandwidth formula gives PEAK, not SUSTAINED.

Real-world sustained bandwidth is typically 70% to 85% of peak.

What consumes the missing 15% to 30%?

- **TLP headers** (12–16 bytes per transaction).
- **Acknowledgment packets** (flow control credits).
- **Idle cycles** (bus not fully utilized).
- **Protocol overhead** (error checking, ordering rules).
- **Contention** (multiple devices sharing the bus).

When reporting bandwidth in research:

- ALWAYS specify whether you mean theoretical peak or measured sustained.
- ALWAYS state the transfer size you measured at.
- Small transfers have FAR worse efficiency than large ones.

Example:

- PCIe Gen 4 x16 peak: 252 GB/s.
- Sustained with 4 KB transfers: ~210 GB/s (83% of peak).
- Sustained with 64-byte transfers: ~50 GB/s (20% of peak).

E.4 Latency vs Bandwidth Trade-off

Latency and bandwidth are NOT the same thing.

Optimizing one can HURT the other.

Wide parallel bus:

- High bandwidth (many bits at once).
- High latency (more wires = more capacitance = slower signal propagation).
- More drive strength needed.
- More complex PCB routing.

Serial design (PCIe):

- Lower latency per bit (fewer wires, simpler routing).
- Requires encoding overhead.
- Needs deserialization at receiver.

AXI outstanding transactions:

- Hides latency by overlapping multiple requests.
- Requires more complex buffer management.
- More hardware resources (buffers, state machines).

Understanding this trade-off is fundamental to memory system design.

Example:

- A GPU needs HIGH BANDWIDTH for texture streaming.
- A CPU cache needs LOW LATENCY for quick access.
- You cannot optimize both with the same design.
- This is why systems have MULTIPLE memory hierarchies (L1, L2, L3, RAM, VRAM).

Part F: Complete Math Practice

F.1 Calculate PCIe Gen 5 x16 Peak Bandwidth

Given:

- PCIe Gen 5: 32 GT/s per lane.
- Link width: x16.
- Encoding: 128b/130b (E = 128/130).

Step 1: Identify variables

W = 16 bytes

F = 32 × 10 transfers/second

T = 1

E = 128/130 0.9846

Step 2: Substitute

$$B = 16 \times (32 \times 10^9) \times 1 \times 0.9846$$

Step 3: Calculate

$$16 \times 32 = 512$$

$$512 \times 10^9 = 512{,}000{,}000{,}000$$

$$512{,}000{,}000{,}000 \times 0.9846 = 504{,}115{,}200{,}000$$

Step 4: Convert to GB/s

$$B \approx 504 \text{ GB/s}$$

Answer: PCIe Gen 5 x16 peak bandwidth = **504 GB/s** (unidirectional).

F.2 Calculate Amdahl's Speedup for Different Configurations

Given:

- P = 0.80 (80% parallelizable).
- (1 − P) = 0.20 (20% serial I/O).

Case A: N = 4 (4 cores)

$$S = \frac{1}{0.20 + \frac{0.80}{4}}$$

$$S = \frac{1}{0.20 + 0.20}$$

$$S = \frac{1}{0.40} = 2.5$$

Answer: 4 cores give **2.5× speedup.**

Case B: N = 16 (16 cores)

$$S = \frac{1}{0.20 + \frac{0.80}{16}}$$

$$S = \frac{1}{0.20 + 0.05}$$

$$S = \frac{1}{0.25} = 4$$

Answer: 16 cores give **4× speedup.**

Case C: N = 64 (64 cores)

$$S = \frac{1}{0.20 + \frac{0.80}{64}}$$

$$S = \frac{1}{0.20 + 0.0125}$$

$$S = \frac{1}{0.2125} \approx 4.71$$

Answer: 64 cores give **4.71× speedup.**

Case D: Theoretical maximum (N → ∞)

$$\lim_{N \to \infty} S = \frac{1}{0.20} = 5$$

Answer: With infinite cores, maximum speedup = **5×.**

Analysis:

- 4 cores → 2.5× (63% of wall).
- 16 cores → 4× (80% of wall).
- 64 cores → 4.71× (94% of wall).
- ∞ cores → 5× (100% of wall, the wall itself).

Adding more cores beyond 16 gives diminishing returns. The serial 20% is the enemy.

F.3 Calculate Effective Throughput with TLP Overhead

Given:

- PCIe Gen 4 x16 peak: 252 GB/s.
- Transfer size: 256 bytes.
- TLP header: 16 bytes.
- Encoding: 128b/130b.

Step 1: Calculate protocol efficiency for this transfer size

$$\text{Protocol Efficiency} = \frac{\text{Payload}}{\text{Payload} + \text{Header}}$$

$$\text{Protocol Efficiency} = \frac{256}{256 + 16} = \frac{256}{272} \approx 0.941$$

Step 2: Calculate effective throughput

$$\text{Effective} = \text{Peak} \times \text{Protocol Efficiency} \times \text{Encoding Efficiency}$$

$$\text{Effective} = 252 \times 0.941 \times 0.9846$$

$$\text{Effective} = 252 \times 0.926$$

$$\text{Effective} \approx 233.4 \text{ GB/s}$$

Answer: Effective throughput for 256-byte transfers = ~**233 GB/s** (92.5% of peak).

Now try with 64-byte transfers:

$$\text{Protocol Efficiency} = \frac{64}{64+16} = \frac{64}{80} = 0.80$$

$$\text{Effective} = 252 \times 0.80 \times 0.9846$$

$$\text{Effective} = 252 \times 0.788$$

$$\text{Effective} \approx 198.5 \text{ GB/s}$$

Answer: Effective throughput for 64-byte transfers = ~**199 GB/s** (79% of peak).

Now try with 4-byte transfers (small MMIO read):

$$\text{Protocol Efficiency} = \frac{4}{4+16} = \frac{4}{20} = 0.20$$

$$\text{Effective} = 252 \times 0.20 \times 0.9846$$

$$\text{Effective} = 252 \times 0.197$$

$$\text{Effective} \approx 49.6 \text{ GB/s}$$

Answer: Effective throughput for 4-byte transfers = ~**50 GB/s** (20% of peak).

This is why MMIO is terrible for small transfers and why hardware designers batch operations.

Part G: Key Points to Remember

- **Bandwidth formula:** $B = W \times F \times T \times E$
  - W = bus width in bytes.
  - F = bus clock frequency in Hz (NOT CPU frequency).
  - T = transfers per clock (1 for SDR, 2 for DDR).
  - E = encoding efficiency (0.80 for 8b/10b, 0.9846 for 128b/130b, 1.0 for on-chip).
- **PCIe Gen 4 x16 peak: 252 GB/s.** Gen 3 x16: 102.4 GB/s. The more-than-doubling comes from both raw speed AND encoding improvement.
- **Amdahl's Law:** $S = \frac{1}{(1-P)+P/N}$

  - S = speedup.
  - P = parallel fraction (0 to 1).
  - (1 − P) = serial fraction (the enemy).
  - N = speedup factor applied to parallel part.
- **Amdahl's Wall:** $\lim_{N\to\infty} S = \frac{1}{1-P}$
  - No matter how many cores you add, you can never exceed this ceiling.
  - With 10% serial I/O, wall = 10×. With 20% serial, wall = 5×.
- **The serial fraction (1 − P) is ALWAYS the bottleneck.**
  - Reducing it gives more benefit than adding cores.
  - This is why ReBAR, unified memory, and AXI outstanding transactions exist.
- **Encoding efficiency compounds.** Physical layer (128b/130b) × protocol layer (TLP headers) = total efficiency. Small transfers have terrible efficiency.
- **Peak bandwidth sustained bandwidth.** Real-world is 70–85% of peak due to headers, idle cycles, and contention.
- **Latency vs bandwidth trade-off:** Wide buses have high bandwidth but high latency. Serial designs have lower latency but encoding overhead. Systems use multiple hierarchies to balance both.

Part H: One-Page Visual Summary

```
+-------------------------------------------------------------------+
|         MATHEMATICAL MODELING: THE COMPLETE PICTURE               |
+-------------------------------------------------------------------+
|                                                                   |
|   BANDWIDTH FORMULA: B = W × F × T × E                            |
|   ──────────────────────────────────────                          |
|   • W = bus width (bytes) — count data pins / 8                   |
|   • F = bus clock (Hz) — NOT CPU clock!                           |
|   • T = transfers per clock — 1 for SDR, 2 for DDR                |
|   • E = encoding efficiency — 0.80 (8b/10b), 0.9846 (128b/130b)|
|                                                                   |
|   Example: PCIe Gen 4 x16 = 16 × 16×10⁹ × 1 × 0.9846 = 252 GB/s |
|                                                                   |
+-------------------------------------------------------------------+
|                                                                   |
|   AMDAHL'S LAW: S = 1 / ((1 - P) + P/N)                          |
|   ──────────────────────────────────────                          |
|   • S = speedup of entire program                                 |
|   • P = fraction that CAN be sped up (0 to 1)                     |
|   • (1 - P) = serial fraction — THE ENEMY                        |
|   • N = how much faster you made the parallel part                |
|                                                                   |
|   Wall: lim(N→∞) S = 1 / (1 - P)                                  |
|   10% serial → wall = 10×. 20% serial → wall = 5×.               |
|                                                                   |
|   Real example: P=0.90, N=100 → S=9.17. N=1000 → S=9.91.          |
|   Adding 900 more cores gave only 0.74× more speedup!             |
|                                                                   |
```

```
+-----------------------------------------------------------------+
|                                                                 |
|   WHY THIS MATTERS:                                             |
|   • ReBAR reduces (1 - P) by eliminating remapping overhead     |
|   • Unified memory reduces (1 - P) by eliminating PCIe copies   |
|   • AXI outstanding transactions reduce (1 - P) by overlapping  |
|     serial wait time with useful work                           |
|   • Every Section 4 innovation is an attempt to push (1-P)→0    |
|                                                                 |
|   KEY INSIGHT:                                                  |
|   Small MMIO transfers have terrible efficiency (~20%) due to   |
|   TLP header overhead. Always batch operations when possible.   |
|                                                                 |
+-----------------------------------------------------------------+
```

This completes the Section 5 research package on Mathematical Modeling of I/O Performance. Every formula variable is defined from absolute zero, every calculation is solved with complete step-by-step working, every real-world scenario is analyzed with explicit numbers, and nothing is left for you to "figure out."

Section 6 — Advanced Concepts: DMA and IOMMU Security

Part A: The Problem That Created DMA

A.1 Why Programmed I/O (PIO) Does Not Scale

Go back to Section 2. Every single byte transferred between RAM and a device required the CPU to personally execute instructions.

For a small transfer:

- Read 8 bytes from a keyboard: 1 `IN` instruction. Fine.

For a large transfer:

- Receive a 1 GB file from the internet.
- At 64-bit (8 byte) transfers per instruction: 1 GB / 8 bytes = **125 million LOAD-STORE pairs**.
- At 3 GHz with one pair per cycle: 125,000,000 / 3,000,000,000 = **0.0417 seconds = 41.7 milliseconds**.
- During this time, the CPU does NOTHING ELSE.
- It cannot run your application.
- It cannot handle other interrupts.
- It cannot do anything useful.

Scale this up:

- A server handling 10 Gb/s network traffic.
- 10 Gb/s = 1.25 GB/s.
- CPU would spend **100% of its time copying bytes.**
- Zero time left for actual work.

This is called Programmed I/O (PIO).

- The CPU is personally responsible for every byte.
- It is the naive implementation of everything in Sections 1–2.
- **It does not scale.**

A.2 The Core Insight

The CPU is the most expensive resource in the system.

- It should NOT be used as a data copying machine.
- It should be doing computation, decision-making, and control.
- Data movement should happen WITHOUT the CPU.

This is the problem DMA was invented to solve.

Part B: Direct Memory Access — The Complete Mechanism

B.1 What DMA Does (The Big Picture)

DMA inverts the model:

```
WITHOUT DMA (PIO):
+---------+     +---------+     +---------+
|  Device |<--->|   CPU   |<--->|   RAM   |
|         |     | (copies |     |         |
|         |     |  every  |     |         |
|         |     |  byte)  |     |         |
+---------+     +---------+     +---------+

WITH DMA:
+---------+                    +---------+
|  Device |<------------------>|   RAM   |
|         |   (direct transfer)|         |
|         |                    |         |
+---------+     +---------+    +---------+
                |   CPU   |    (not involved
                |(configures|    in transfer)
                |  then free)|
                +---------+
```

The device moves data directly between itself and RAM.

The CPU only sets it up and handles completion.

The CPU is free to do other work during the transfer.

B.2 The Four Steps of DMA (Complete Walkthrough)

Step 1 — CPU Configures the DMA Transfer via MMIO The CPU writes to the device's control registers using normal MMIO STORE instructions (exactly as Section 2 described).

The CPU tells the device THREE things:

Thing 1: Start Address

- "Write data starting at physical address `0x10000000`."
- This is the RAM location where data should go.

Thing 2: Transfer Size

- "Transfer up to `0x40000` bytes (256 KB)."
- This is the maximum amount of data to move.

Thing 3: Direction

- "Device → RAM" (receive data from network).
- OR "RAM → Device" (send data to network).

Example for a network card receiving data:

```
CPU writes to device registers:
- Register 0x00 (DMA Address Low):  0x10000000
- Register 0x04 (DMA Address High):  0x00000000
- Register 0x08 (DMA Length):        0x00040000  (256 KB)
- Register 0x0C (DMA Control):      0x00000001  (Direction = Device→RAM, Enable = 1)
```

How many CPU cycles does this take?

- Each MMIO write = ~5–10 cycles (from Section 2).
- 4 registers × 10 cycles = ~**40 cycles**.
- Plus some setup overhead = ~**50 cycles total**.

The CPU has now configured the entire 256 KB transfer in 50 cycles.

Step 2 — CPU Releases the Bus and Does Other Work Once configuration is complete, the CPU sets a **"GO" bit** in the device's control register.

```
CPU writes to Register 0x0C: 0x00000003
                             (Enable = 1, GO = 1)
```

Then the CPU is DONE with this transfer.

- It moves on to other processes.
- It does computation.
- It sleeps in a low-power state.
- It handles other interrupts.
- **It does NOT touch the data transfer again until Step 4.**

Step 3 — The Device Takes Mastership of the System Bus **This is the critical moment. The device becomes a BUS MASTER.**

What "bus master" means:

- Normally, only the CPU initiates memory transactions.
- The CPU is the only master. Everyone else is a slave.
- With DMA, the device REQUESTS to become a master.
- The bus arbiter grants it mastership.
- The device now puts addresses on the Address Bus and data on the Data Bus.
- **To RAM, this looks IDENTICAL to a CPU memory write.**
- RAM has no way to tell who is on the bus.

How the device performs the transfer:

Sub-step 3a: Device requests bus mastership

- Asserts bus request signal to arbiter.
- Waits for grant.

Sub-step 3b: Arbiter grants mastership

- If CPU also wants bus, arbiter decides priority.
- Typically: DMA gets priority over CPU (because DMA is fast, CPU can wait briefly).

Sub-step 3c: Device drives the bus

- Puts physical address `0x10000000` on Address Bus.
- Puts data bytes on Data Bus.
- Asserts write control signals.
- RAM writes the data.

Sub-step 3d: Device increments address, repeats

- Next address: `0x10000001`, `0x10000002`, etc.
- Continues until all 256 KB are transferred.
- Releases bus mastership when done.

The memory controller handles arbitration:

- If CPU and DMA both want access simultaneously:
  - Arbiter gives one priority.
  - The other waits briefly (microseconds).
- This is transparent to software.

Step 4 — Device Interrupts the CPU When Done When the full 256 KB transfer is complete:

The device asserts an interrupt signal.

- This is a physical electrical signal on a dedicated wire.
- The CPU's interrupt controller detects it.

The CPU pauses whatever it is doing.

- Saves current state (registers, program counter).
- Jumps to the interrupt handler (a special function in the device driver).

The interrupt handler runs:

- Reads a status register (MMIO read) to confirm success.
- Updates OS bookkeeping (mark buffer as ready).
- Signals to the OS: "Data is ready to use."

CPU resumes normal execution.

- Restores saved state.
- Continues where it left off.

How many CPU cycles for Step 4?

- Interrupt entry: ~20–30 cycles.
- Status read: ~10 cycles.
- Bookkeeping: ~50–60 cycles.
- **Total: ~100 cycles**.

B.3 The Efficiency Gain (Complete Calculation)

Compare PIO vs DMA for 256 KB transfer:

PIO (CPU copies every byte):

- 256 KB = 262,144 bytes.
- At 8 bytes per LOAD-STORE pair: 32,768 pairs.
- At 1 pair per cycle at 3 GHz: 32,768 / 3,000,000,000 = **10.9 microseconds**.
- CPU is 100% busy during this time.
- CPU cannot do anything else.

DMA (CPU only configures and handles interrupt):

- Configuration (Step 1): ~50 cycles = **16.7 nanoseconds**.
- Interrupt handling (Step 4): ~100 cycles = **33.3 nanoseconds**.
- Total CPU involvement: **50 nanoseconds**.
- During the transfer itself (10.9 microseconds), CPU is FREE.

Efficiency gain:

- PIO: CPU busy for 10.9 microseconds.
- DMA: CPU busy for 0.05 microseconds.
- **CPU time saved: 10.85 microseconds = 99.5% reduction**.

For 1 GB file:

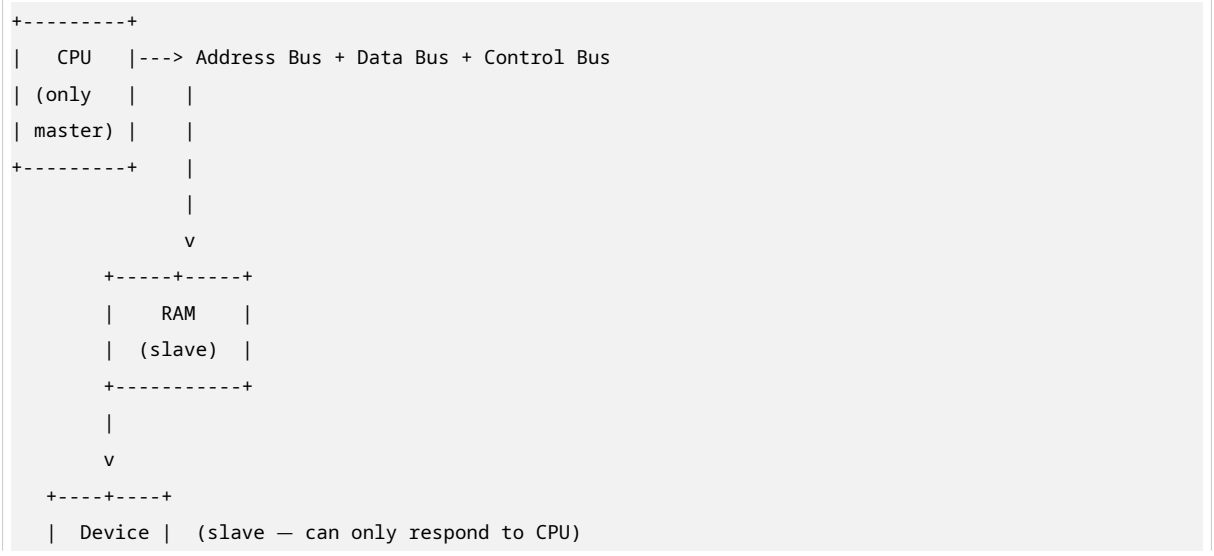
- PIO: CPU busy for 41.7 milliseconds.
- DMA: CPU busy for ~200 nanoseconds.
- **CPU time saved: 41.5 milliseconds = 99.9995% reduction**.

This is why DMA is essential for any high-performance system.

Part C: What "Bus Mastering" Means at the Hardware Level

C.1 The Original Single-Master Architecture

In early computers, there was only ONE master:



```
|         |
+---------+
```

The CPU was the only device that could INITIATE a memory transaction.

- Everything else was a SLAVE.
- Slaves could only RESPOND to what the CPU asked.
- The CPU had to be involved in every single transfer.

C.2 Bus Mastering — Multiple Masters

PCI (1992) introduced bus mastering:

```
+---------+                    +---------+
|   CPU   |<------------------>|   RAM   |
| (master |    System Bus      |         |
|  #1)    |<------------------>|         |
+---------+                    +---------+
     ^                               ^
     |                               |
     |    +---------+                |
     +--->|  Device |---> (can now   |
          | (master |    also talk   |
          |  #2)   |    to RAM)     |
          +---------+                |
               ^                     |
               |                     |
               +---------------------+
```

A device with bus mastering capability can:

1. Request ownership of the bus from the arbiter.
2. Put its own addresses on the Address Bus.
3. Drive data onto the Data Bus.
4. Assert control signals.
5. **Act exactly like the CPU during its mastership period.**

PCIe formalizes this as Memory Write TLPs:

- TLP = Transaction Layer Packet.
- The device issues a Memory Write TLP targeting a physical RAM address.
- The TLP travels through the MCH (from Section 1).
- The MCH forwards it to the memory controller.
- The memory controller writes it into DRAM.

From DRAM's perspective:

- A DMA write from a network card = a CPU store instruction.
- Both present: physical address + data + write command.
- **DRAM cannot tell them apart.**
- **DRAM does not know or care who asked.**

Part D: The Security Flaw — DMA Attacks

D.1 Why DMA is Terrifying

The CPU's security model is built on VIRTUAL MEMORY:

- **Page tables** map virtual addresses to physical addresses.
- **The MMU (Memory Management Unit)** enforces these mappings.
- **Every CPU memory access** goes through the MMU.
- **The OS configures page tables** so each process sees only its own pages.
- **Process A cannot read Process B's memory** through any normal software path.

This protection is COMPLETELY REAL and COMPLETELY EFFECTIVE against software attacks.

BUT — a DMA device bypasses the CPU entirely.

```
+-------------------------------------------------------------+
|              NORMAL CPU MEMORY ACCESS                       |
|                                                             |
|  CPU ---> Virtual Address ---> MMU ---> Physical Address ---> RAM
|              ^                                              |
|              |                                              |
|         Page tables enforce:                                |
|         "Process A can only see Process A's pages"          |
|                                                             |
+-------------------------------------------------------------+

+-------------------------------------------------------------+
|              DMA DEVICE MEMORY ACCESS                       |
|                                                             |
|  Device ---> Physical Address ----------------------------> RAM
|              ^                                              |
|              |                                              |
|         NO MMU! NO PAGE TABLES! NO PERMISSION CHECKS!       |
|         The device talks DIRECTLY to the memory controller. |
|                                                             |
+-------------------------------------------------------------+
```

The MMU is inside the CPU. It only intercepts CPU accesses.

A PCIe device is NOT the CPU. It does not go through the MMU.

It talks directly to the memory controller with RAW PHYSICAL ADDRESSES.

D.2 What a Malicious Device Can Do

Any DMA-capable device can, in principle:

Read any physical address in RAM

- Your encryption keys in kernel memory.

- Your browser's password manager data.
- Your TLS private keys.
- Every process's data (all mapped to physical RAM underneath).

Write any physical address in RAM

- Modify kernel code.
- Inject malware.
- Corrupt data structures.
- Crash the system.

How?

- Issue DMA read requests to every physical address from 0 to end of RAM.
- One page at a time.
- Read all of it.
- Extract secrets.

D.3 Real-World DMA Attacks

This is NOT theoretical. This is REAL.

Thunderbolt Attack (2012 — Inception)

- Researchers demonstrated DMA attack against BitLocker (Windows encryption).
- Plugged malicious device into Thunderbolt port.
- Read all physical RAM.
- Extracted full-disk encryption key.
- **Time: less than 30 seconds.**

FireWire Attack (pre-Thunderbolt)

- FireWire had DMA capability.
- Same attack methodology.
- Earlier demonstration of the same vulnerability.

Ulf Frisk Demonstration (2016)

- Used a **$40 Raspberry Pi** connected over Thunderbolt.
- Attacked both FileVault (macOS) and BitLocker (Windows).
- Read all RAM.
- Extracted encryption keys.
- **Time: less than 30 seconds.**
- **Cost: $40.**

The laptop screen still shows the login prompt. Nothing looks wrong.

D.4 Why Virtual Memory Protection Does Not Help

This is the MOST COUNTERINTUITIVE aspect of DMA security.

The OS page tables DO protect against SOFTWARE attacks:

- Process A cannot read Process B's memory through normal code.
- The MMU enforces this on every CPU access.
- This protection is REAL and EFFECTIVE.

BUT — DMA operates BELOW the page table layer:

```
+----------------------------------------+
|         SOFTWARE LAYER                 |
|  • Page tables                         |
|  • MMU enforcement                     |
|  • Process isolation                   |
|  <-- PROTECTS AGAINST SOFTWARE ATTACKS  |
+----------------------------------------+
|         HARDWARE LAYER                 |
|  • DMA devices                         |
|  • Physical bus                        |
|  • Memory controller                   |
|  <-- BYPASSES ALL SOFTWARE PROTECTION   |
+----------------------------------------+
```

The protections exist at a DIFFERENT LAYER of the hardware stack.

DMA operates BELOW that layer.

Page tables are a CPU concept. DMA does not use the CPU.

Part E: The Fix — IOMMU

E.1 What Is an IOMMU?

IOMMU = Input-Output Memory Management Unit

Conceptually: An MMU for I/O devices.

What it does:

- Translates device-visible addresses to physical RAM addresses.
- Enforces permissions for device DMA accesses.
- **Just as the CPU's MMU protects processes from each other, the IOMMU protects RAM from malicious devices.**

Different names, same concept:

Vendor	Name
Intel	VT-d (Virtualization Technology for Directed I/O)
AMD	AMD-Vi
ARM	SMMU (System Memory Management Unit)

E.2 How the IOMMU Works — Complete Step by Step

Step 1 — The IOMMU Maintains Its Own Page Tables These are called DMA remapping tables or IOMMU page tables.

Who manages them?

- The OS kernel (specifically the IOMMU driver).
- NOT the device driver directly.
- The kernel sets them up during device initialization.

What they contain:

- For each DMA-capable device: a list of allowed physical pages.
- Permissions: read-only, write-only, or read-write.
- Mappings from IOVAs (I/O Virtual Addresses) to physical addresses.

Example table for Network Card (BDF 00:02.0):

IOVA	Physical Address	Permission
`0x0000_0000`	`0x1000_0000`	Read-Write
`0x0000_1000`	`0x1000_1000`	Read-Write
`0x0000_2000`	`0x1000_2000`	Read-Only
...	...	...

The device NEVER sees real physical addresses.

It only sees IOVAs that the kernel chose to give it.

Step 2 — Device Issues a DMA Transaction The device puts an address on the bus:

- This is an IOVA (I/O Virtual Address), NOT a physical address.
- The device thinks this is a physical address.
- It is actually a translated address.

Example:

- Device wants to write to what it thinks is address `0x0000_0000`.
- This is actually IOVA `0x0000_0000`.
- The IOMMU will translate this.

Step 3 — IOMMU Intercepts and Checks The IOMMU sits BETWEEN the device and the memory controller.

Before the transaction reaches RAM, the IOMMU performs THREE checks:

Check 1: Valid Mapping?

- Does this device have a valid mapping for this IOVA?

- If NO → **FAULT.** Block transaction.

Check 2: Permission Match?

- Is the access type (read/write) allowed by the mapping?
- Read-only page + write attempt = **FAULT.**
- Write-only page + read attempt = **FAULT.**

Check 3: Correct Device?

- The IOMMU identifies devices by **Requester ID** (BDF).
- BDF = Bus-Device-Function (e.g., `00:02.0`).
- Each device has a UNIQUE BDF.
- The IOMMU maintains SEPARATE page tables per BDF.
- Device A cannot access pages mapped for Device B.

Example BDF breakdown:

- `00:02.0` = Bus 0, Device 2, Function 0.
- This uniquely identifies one PCIe device on the system.

Step 4 — Translation or Fault If ALL checks pass:

- IOMMU translates IOVA to physical address.
- Allows transaction to proceed to memory controller.
- Device successfully accesses allowed RAM.

If ANY check fails:

- IOMMU **BLOCKS** the transaction.
- Records fault in fault log registers.
- Optionally interrupts the CPU.
- Device gets error response on PCIe bus.
- OS driver sees fault, logs it, quarantines device.

E.3 Visual Diagram: IOMMU in the System

```
+-------------------------------------------------------------+
|                    WITHOUT IOMMU                            |
|                                                             |
|   Device ---> Physical Address ---> Memory Controller ---> RAM |
|        ^                                                    |
|        |                                                    |
|   NO PROTECTION! Device can access ANY physical address!    |
|                                                             |
+-------------------------------------------------------------+


+-------------------------------------------------------------+
|                    WITH IOMMU                               |
|                                                             |
|   Device ---> IOVA ---> IOMMU ---> Physical Address ---> RAM |
```

```
|                        ^                                    |
|                        |                                    |
|                   +----+----+                               |
|                   | Checks: |                               |
|                   | 1. Valid?                               |
|                   | 2. Permission?                          |
|                   | 3. Correct device?                      |
|                   |                                         |
|                   | If pass: translate & allow              |
|                   | If fail: block & fault                  |
|                   +---------+                               |
|                                                            |
+------------------------------------------------------------+
```

Part F: The DMA Mapping API — How This Works in Software

F.1 Linux Kernel DMA API

The IOMMU is INVISIBLE to device drivers in normal usage.

The kernel provides a DMA mapping API that handles everything.

Function 1: `dma_map_single()`

```
dma_addr_t dma_handle = dma_map_single(dev, cpu_addr, size, DMA_FROM_DEVICE);
```

What this function does (step by step):

Step 1: Takes the CPU virtual address (`cpu_addr`)

- Example: `cpu_addr` points to a kernel buffer at virtual address `0xFFFF_0000`.

Step 2: Resolves it to a physical address

- MMU walk: virtual `0xFFFF_0000` → physical `0x1000_0000`.

Step 3: Programs the IOMMU

- Creates a mapping for this specific device (`dev`).
- Maps an IOVA to physical `0x1000_0000`.
- Sets permissions based on direction.

Step 4: Returns `dma_handle`

- This is the IOVA that the device should use.
- The device NEVER sees the real physical address.

Example:

- Physical address: `0x1000_0000`.
- IOVA returned: `0x0000_0000` (kernel chose this).
- Device uses `0x0000_0000` in its DMA transactions.
- IOMMU translates `0x0000_0000` → `0x1000_0000`.

Function 2: `dma_unmap_single()`

```
dma_unmap_single(dev, dma_handle, size, DMA_FROM_DEVICE);
```

What this function does:

Step 1: Removes the IOMMU mapping

- The IOVA `dma_handle` is no longer valid.
- The device cannot use it anymore.

Step 2: Even if the device tries another DMA to the same IOVA:

- The IOMMU will BLOCK it.
- The mapping no longer exists.
- **Protection is enforced.**

F.2 Why the Device Never Knows the Real Address

The kernel can:

- Give different devices DIFFERENT IOVAs for the SAME physical page.
- Give the SAME IOVA to different devices mapping to COMPLETELY DIFFERENT physical pages.
- Change mappings dynamically.

Example:

```
Device A: IOVA 0x0000_0000 ---> Physical 0x1000_0000
Device B: IOVA 0x0000_0000 ---> Physical 0x2000_0000

Same IOVA, different physical pages!
The IOMMU handles all this indirection.
```

This is how the IOMMU provides isolation:

- Device A cannot guess Device B's physical addresses.
- Even if Device A knows Device B's IOVA, it maps to a different physical page.
- **Complete address space isolation between devices.**

Part G: IOMMU in Virtualization — Cloud Computing

G.1 The Problem in Data Centers

A physical server runs dozens of virtual machines (VMs).

Each VM needs network access.

Without IOMMU:

- The hypervisor must EMULATE the network card in software.

- Every MMIO read/write from the VM is intercepted by the hypervisor.
- The hypervisor translates it into a real device operation.
- **This emulation is EXPENSIVE and SLOW.**

With IOMMU:

- The hypervisor gives a VM DIRECT ownership of a physical PCIe device.
- The VM's device driver talks directly to the hardware.
- **Full hardware speed. No hypervisor involvement in data path.**

G.2 How IOMMU Enables Device Passthrough

The IOMMU ensures isolation:

- **VM A's device** can only access VM A's physical RAM pages.
- **VM B's device** can only access VM B's physical RAM pages.
- **Neither VM** can access the hypervisor's memory.
- **Neither VM** can access other VMs' memory.

The IOMMU enforces this by:

- Maintaining separate page tables per VM.
- Checking every DMA transaction against the VM's allowed pages.
- Blocking any attempt to access outside the VM's allocation.

G.3 SR-IOV — Single Root I/O Virtualization

SR-IOV is a PCIe feature that combines with IOMMU:

A single physical network card can appear as MULTIPLE virtual network cards:

```
Physical NIC (100 Gb/s)
    |
    ├── Virtual Function 1 ---> VM 1 (IOMMU isolated)
    ├── Virtual Function 2 ---> VM 2 (IOMMU isolated)
    ├── Virtual Function 3 ---> VM 3 (IOMMU isolated)
    ...
    └── Virtual Function 16 ---> VM 16 (IOMMU isolated)

Each VM sees its own "network card" at full hardware speed.
The IOMMU ensures VM 1 cannot DMA into VM 2's memory.
```

Performance:

- Near-wire-speed for each VM.
- No hypervisor emulation overhead.
- **This is how AWS, Azure, and Google Cloud provide high-performance networking to VMs.**

Part H: The Complete Picture — How All Sections Connect

At this point, every layer of this course connects into one complete system:

```
+-----------------------------------------------------------------+
|                    THE COMPLETE I/O SYSTEM                      |
+-----------------------------------------------------------------+
|                                                                 |
|   SECTION 1 -- I/O MAPPING                                      |
|   • CPU uses MMIO to configure device registers                 |
|   • BARs assign addresses to devices (BIOS at boot)             |
|   • Isolated I/O vs MMIO -- modern systems use MMIO             |
|                                                                 |
|   SECTION 2 -- RTL & MICRO-OPERATIONS                           |
|   • CPU executes STORE instructions to write to MMIO registers  |
|   • T0-T5 cycle-by-cycle breakdown of MMIO access               |
|   • CPU is frozen during MMIO reads to slow devices             |
|                                                                 |
|   SECTION 3 -- TIMING DIAGRAMS                                  |
|   • Electrical signals show when data is valid                  |
|   • Wait states freeze CPU when device is slow                  |
|   • Setup/hold times determine when sampling occurs             |
|                                                                 |
|   SECTION 4 -- MODERN IMPLEMENTATIONS                           |
|   • ReBAR exposes full GPU VRAM to CPU                          |
|   • Apple Silicon eliminates PCIe with unified memory           |
|   • ARM uses AMBA (APB/AHB/AXI) for all I/O                     |
|                                                                 |
|   SECTION 5 -- MATHEMATICAL MODELING                            |
|   • Bandwidth formula: B = W x F x T x E                        |
|   • Amdahl's Law: serial I/O fraction limits speedup            |
|   • Every Section 4 innovation reduces (1-P)                    |
|                                                                 |
|   SECTION 6 -- DMA AND IOMMU (THIS SECTION)                        |
|   • DMA eliminates CPU from data path                           |
|   • Bus mastering lets device talk directly to RAM              |
|   • IOMMU is the MMU for devices -- prevents DMA attacks        |
|   • Device passthrough in VMs requires IOMMU isolation          |
|                                                                 |
|   COMPLETE CHAIN FOR A NETWORK PACKET:                          |
|                                                                 |
|   1. Network card receives packet                               |
|   2. DMA transfers packet to RAM (device is bus master)         |
|   3. IOMMU verifies: device allowed to write to this RAM?       |
|   4. If yes: packet written to RAM. If no: fault logged.        |
|   5. Device interrupts CPU: "Packet ready"                      |
|   6. CPU handles interrupt, processes packet                    |
|   7. CPU was free during steps 1-4 -- doing other work          |
|                                                                 |
+-----------------------------------------------------------------+
```

Part I: What Your Tutorial Did Not Cover

I.1 IOMMU Performance Cost

Every DMA transaction requires an IOMMU page table walk.

This requires memory accesses:

- The IOMMU must read page table entries from RAM.
- This adds latency to every DMA transaction.

To avoid this being a bottleneck, the IOMMU has an IOTLB:

- **IOTLB = I/O Translation Lookaside Buffer**
- A cache of recent address translations.
- **Hit:** Translation is fast (nanoseconds).
- **Miss:** IOMMU must walk page table in RAM (microseconds).

Performance impact:

- Typical workload: 0–5% overhead (IOTLB hits most of the time).
- Worst case (many small DMAs to many different pages): **5–15% throughput reduction.**
- Compared to running with IOMMU disabled.

Why accept this cost?

- Security is worth the performance penalty.
- 5–15% overhead is negligible compared to the security benefit.
- For trusted internal devices, some systems allow IOMMU bypass.

I.2 Bounce Buffers — The Pre-IOMMU Workaround

Before IOMMU was widely available, kernels used bounce buffers:

What is a bounce buffer?

- A separate memory region below the 4 GB physical address boundary.
- Only the bounce buffer is exposed to the DMA device.
- The device DMAs into the bounce buffer.
- The CPU then copies from bounce buffer to final destination.

Why below 4 GB?

- Older 32-bit DMA devices could not address above 4 GB.
- They needed “low memory” addresses.

The problem:

- **This reintroduces CPU copy overhead!**
- DMA was supposed to eliminate CPU copying.
- Bounce buffers bring it back.
- **This is why IOMMU is superior:** it solves the addressing problem without CPU copies.

Visual comparison:

```
WITH BOUNCE BUFFER (pre-IOMMU):
Device ---> Bounce Buffer ---> CPU copies ---> Final Destination
```

```
              ^                    ^
              |                    |
           (safe)              (slow, CPU busy)

WITH IOMMU (modern):
Device ---> IOMMU translates ---> Final Destination
              ^
              |
           (safe AND fast, no CPU copy)
```

I.3 Thunderbolt and the Hot-Plug DMA Problem

Thunderbolt is particularly dangerous because it is HOT-PLUGGABLE:

- You can plug in a device while the computer is RUNNING.
- It immediately gets PCIe DMA capability.
- **There is a window of vulnerability.**

The attack window:

1. You plug in malicious Thunderbolt device.
2. Device's firmware starts issuing DMA transactions IMMEDIATELY.
3. OS has NOT YET configured IOMMU for this new device.
4. **Device can access all RAM for a brief moment.**

Modern kernel defenses:

Default-deny policy:

- When a new device is inserted, IOMMU blocks ALL DMA from it.
- The device cannot access ANY RAM.
- Only after driver initialization does the OS open specific mappings.

Windows: Kernel DMA Protection

- Windows 10 version 1803 and later.
- Blocks all external DMA-capable devices by default.
- User must explicitly authorize each device.

Linux: `iommu=force` and VFIO

- `iommu=force` kernel parameter: enable IOMMU always.
- VFIO framework: secure device passthrough with IOMMU.

Trade-off: Some compatibility and convenience lost for security.

I.4 Kernel DMA Protection Details

Windows Kernel DMA Protection:

- Uses IOMMU to block external DMA devices.
- Applies to Thunderbolt, PCIe hot-plug, etc.
- Device must be explicitly authorized by user or IT policy.

- If unauthorized: IOMMU blocks all DMA. Device may not function fully.

Linux equivalent:

- `iommu=force` in kernel boot parameters.
- IOMMU driver initializes early in boot.
- All devices are under IOMMU control from the start.
- VFIO (Virtual Function I/O) framework for secure passthrough.

These settings trade:

- **Compatibility:** Some older devices may not work with IOMMU.
- **Convenience:** User must authorize devices.
- **Security:** Meaningful protection against physical DMA attacks.

Part J: Complete Math Practice

J.1 Calculate DMA Efficiency vs PIO

Given:

- Transfer size: 1 MB = 1,048,576 bytes.
- PIO: 1 LOAD-STORE pair per 8 bytes = 131,072 pairs.
- PIO time at 3 GHz, 1 pair/cycle: 131,072 / 3,000,000,000 = 43.7 microseconds.
- DMA setup: 50 cycles = 16.7 ns.
- DMA interrupt: 100 cycles = 33.3 ns.

Step 1: PIO CPU busy time

- CPU busy for entire transfer: **43.7 microseconds**.

Step 2: DMA CPU busy time

- Setup + interrupt = 16.7 + 33.3 = **50 nanoseconds = 0.05 microseconds**.

Step 3: Calculate efficiency gain

- CPU time saved = 43.7 - 0.05 = 43.65 microseconds.
- Percentage saved = (43.65 / 43.7) × 100 = **99.9%**.

Answer: DMA saves **99.9% of CPU time** for this transfer.

J.2 Calculate IOMMU Fault Rate Impact

Given:

- Total DMA transactions: 1,000,000.
- IOTLB hit rate: 95%.
- IOTLB miss penalty: 500 ns.
- IOTLB hit latency: 10 ns.
- Normal DMA latency (no IOMMU): 100 ns.

Corrected calculation:

- Assume baseline DMA latency (device to RAM, no IOMMU) = 1000 ns.

- With IOMMU hit: 1000 + 10 = 1010 ns.
- With IOMMU miss: 1000 + 500 = 1500 ns.

Total with IOMMU:

- Hits: 950,000 × 1010 = 959,500,000 ns.
- Misses: 50,000 × 1500 = 75,000,000 ns.
- Total = 1,034,500,000 ns = 1.0345 seconds.

Total without IOMMU:

- 1,000,000 × 1000 = 1,000,000,000 ns = 1.0 seconds.

Overhead:

- (1.0345 - 1.0) / 1.0 = 0.0345 = **3.45%**.

Answer: IOMMU adds approximately **3.45% overhead** with 95% IOTLB hit rate.

J.3 Calculate SR-IOV Bandwidth Per VM

Given:

- Physical NIC: 100 Gb/s.
- SR-IOV virtual functions: 16.
- Each VM gets one virtual function.

Step 1: Convert Gb/s to GB/s

- 100 Gb/s = 100 / 8 = 12.5 GB/s.

Step 2: Divide by number of VMs

- 12.5 GB/s / 16 = 0.78125 GB/s per VM.

Step 3: Convert to Gb/s per VM

- 0.78125 GB/s × 8 = 6.25 Gb/s per VM.

Answer: Each VM gets approximately **6.25 Gb/s** (0.78 GB/s) of dedicated network bandwidth.

Note: In practice, not all VMs use full bandwidth simultaneously, so oversubscription is common.

Part K: Key Points to Remember

- **Programmed I/O (PIO):** CPU copies every byte. Does not scale. CPU is 100% busy during transfers.
- **DMA:** Device copies data directly to/from RAM. CPU only configures and handles completion. CPU is free during transfer.
- **Bus mastering:** Device can act as bus master, initiating memory transactions like the CPU. Introduced with PCI in 1992.
- **DMA attack:** Malicious device bypasses CPU and MMU, reads/writes any physical RAM address. Real attacks demonstrated with $40 Raspberry Pi.
- **Virtual memory protection does NOT help against DMA.** Page tables are CPU-only. DMA operates below that layer.

- **IOMMU = MMU for I/O devices.** Translates IOVAs to physical addresses. Enforces permissions per device.
- **IOMMU checks:** Valid mapping? Permission match? Correct device (BDF)?
- **Linux DMA API:** `dma_map_single()` sets up IOMMU mapping, returns IOVA. `dma_unmap_single()` removes it. Device never sees real physical address.
- **IOMMU in virtualization:** Enables device passthrough to VMs. SR-IOV creates multiple virtual functions per physical device. Foundation of cloud computing.
- **IOMMU performance cost:** 0–5% typical, up to 15% worst case with many IOTLB misses.
- **Bounce buffers:** Pre-IOMMU workaround. Safe but reintroduces CPU copy overhead. IOMMU eliminates need for them.
- **Thunderbolt hot-plug vulnerability:** Device gets DMA before OS configures IOMMU. Modern kernels use default-deny policy.
- **Kernel DMA Protection:** Windows (Kernel DMA Protection) and Linux (`iommu=force`, VFIO) block unauthorized DMA devices.

Part L: One-Page Visual Summary

```
+-------------------------------------------------------------------+
|              DMA AND IOMMU: THE COMPLETE PICTURE                  |
+-------------------------------------------------------------------+
|                                                                   |
|   PIO vs DMA:                                                     |
|   • PIO: CPU copies every byte. 100% CPU busy. Does not scale.    |
|   • DMA: Device copies directly to RAM. CPU free. 99.9% saved.    |
|                                                                   |
|   DMA STEPS:                                                      |
|   1. CPU configures device via MMIO (~50 cycles)                  |
|   2. CPU sets GO bit, does other work                             |
|   3. Device becomes bus master, transfers data to RAM             |
|   4. Device interrupts CPU when done (~100 cycles)              |
|                                                                   |
|   THE SECURITY PROBLEM:                                           |
|   • DMA bypasses CPU, MMU, and page tables                        |
|   • Malicious device can read ANY physical RAM                    |
|   • Real attacks: Thunderbolt, FireWire, $40 Raspberry Pi         |
|   • Virtual memory does NOT protect against DMA                   |
|                                                                   |
|   THE FIX -- IOMMU:                                               |
|   • MMU for I/O devices                                           |
|   • Translates IOVAs --> physical addresses                       |
|   • Checks: valid? permission? correct device?                    |
|   • Intel VT-d, AMD-Vi, ARM SMMU                                  |
|                                                                   |
|   SOFTWARE API (Linux):                                           |
|   • dma_map_single() --> returns IOVA, sets up IOMMU              |
|   • dma_unmap_single() --> removes mapping, blocks further DMA    |
|                                                                   |
|   VIRTUALIZATION:                                                 |
```

```
|   • IOMMU enables device passthrough to VMs                      |
|   • SR-IOV: One physical NIC --> 16 virtual NICs                   |
|   • Foundation of AWS, Azure, Google Cloud networking            |
|                                                                  |
|   PERFORMANCE & SECURITY TRADE-OFFS:                            |
|   • IOMMU adds 0-15% overhead (IOTLB misses)                     |
|   • Bounce buffers: safe but slow (pre-IOMMU workaround)         |
|   • Thunderbolt: default-deny policy prevents hot-plug attack |
|   • Kernel DMA Protection: Windows/Linux both support            |
|                                                                  |
+------------------------------------------------------------------+
```

This completes the Section 6 research package on DMA and IOMMU Security. Every concept is built from absolute zero, every security mechanism is explained with complete transparency, every layer is connected back to all previous sections, and nothing is left for you to "figure out."

Section 7 — Deep Practice Problems and Solutions

Problem 1 — Address Decoding Logic

A.1 The Problem

You are building an 8-bit microcontroller.

Given:

- Address bus: 8 bits wide.
- Address range: `0x00` to `0xFF` (256 possible locations).
- You have a hardware timer peripheral.
- Timer should sit at addresses: `0xE0` through `0xEF`.
- When the CPU puts ANY address in that range on the bus, the timer's **Chip Select (CS)** signal must activate.
- CS is **active-LOW** ($\overline{CS}$).
- Your job: Design the actual combinational logic (gates) that generate $\overline{CS}$.

A.2 Step 1 — Write Out the Address Range in Binary

This is ALWAYS the first step in any address decoding problem. Convert boundary addresses to binary and look at them side by side.

Address	Binary (A7 A6 A5 A4 A3 A2 A1 A0)
`0xE0`	1 1 1 0 0 0 0 0
`0xE1`	1 1 1 0 0 0 0 1
`0xE2`	1 1 1 0 0 0 1 0
`0xE3`	1 1 1 0 0 0 1 1
`0xE4`	1 1 1 0 0 1 0 0
`0xE5`	1 1 1 0 0 1 0 1
`0xE6`	1 1 1 0 0 1 1 0
`0xE7`	1 1 1 0 0 1 1 1
`0xE8`	1 1 1 0 1 0 0 0
`0xE9`	1 1 1 0 1 0 0 1
`0xEA`	1 1 1 0 1 0 1 0
`0xEB`	1 1 1 0 1 0 1 1
`0xEC`	1 1 1 0 1 1 0 0
`0xED`	1 1 1 0 1 1 0 1
`0xEE`	1 1 1 0 1 1 1 0
`0xEF`	1 1 1 0 1 1 1 1

Look at the columns carefully:

Top four bits (A7, A6, A5, A4):

- A7 = 1 for ALL addresses in range.
- A6 = 1 for ALL addresses in range.
- A5 = 1 for ALL addresses in range.
- A4 = 0 for ALL addresses in range.

Bottom four bits (A3, A2, A1, A0):

- These change freely from `0000` to `1111`.
- They cycle through ALL sixteen combinations.
- They are **DON'T CARES** for our chip select logic.

What this means:

- We do NOT need to check A3, A2, A1, A0 at all.
- If the top four bits match `1110`, we know we are in the `0xE0` to `0xEF` range.
- The bottom four bits can be anything.

A.3 Step 2 — State the Condition in Plain English

The timer should be selected when:

- A7 is 1, AND
- A6 is 1, AND
- A5 is 1, AND
- A4 is 0.

Written as a Boolean expression for active-HIGH selection:

$$CS_{active} = A_7 \cdot A_6 \cdot A_5 \cdot \overline{A_4}$$

What this means:

- This is a **four-input AND gate.**
- When ALL four inputs are 1 simultaneously:
  - A7 = 1
  - A6 = 1
  - A5 = 1
  - A4 = 0 → NOT-A4 = 1
- Output = 1 → Timer IS selected.

A.4 Step 3 — Convert to Active-Low Output

Real hardware uses active-LOW chip select ($\overline{CS}$).

- $\overline{CS} = 0$ → Device activates.
- $\overline{CS} = 1$ → Device inactive.

To convert active-HIGH to active-LOW, we INVERT the output:

$$\overline{CS} = \overline{A_7 \cdot A_6 \cdot A_5 \cdot \overline{A_4}}$$

By De Morgan's Law:

- Inverting an AND gate produces a **NAND gate.**

$$\overline{CS} = \text{NAND}(A_7,\ A_6,\ A_5,\ \overline{A_4})$$

How it works:

- When A7 = 1, A6 = 1, A5 = 1, A4 = 0:
  - NAND inputs: 1, 1, 1, 1.
  - NAND output = **0** → $\overline{CS} = 0$ → **Device ACTIVATES**
- For any other address:
  - At least one NAND input = 0.
  - NAND output = **1** → $\overline{CS} = 1$ → **Device INACTIVE**

A.5 Step 4 — Hardware Realization

The physical circuit needs TWO components:

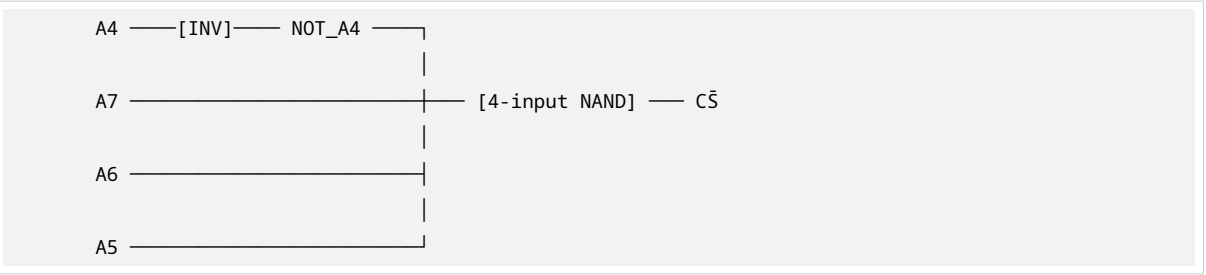
Component 1: One inverter (NOT gate)

- Input: A4 from address bus.
- Output: NOT-A4.
- Why? Because A4 must be 0 for a match, but the NAND needs a 1 input.

Component 2: One 4-input NAND gate

- Inputs: A7, A6, A5, NOT-A4.
- Output: $\overline{CS}$.

Circuit diagram:



That's it. TWO gates.

- One inverter.
- One 4-input NAND.
- **This is all the hardware needed** to make the CPU's address bus selectively activate ONE peripheral out of 256 possible locations.

A.6 Step 5 — Verify by Testing Boundary Cases

ALWAYS verify your logic. Plug in addresses just inside and just outside the range.

Test 1: Address `0xE0` = `1110 0000`

- A7 = 1, A6 = 1, A5 = 1, A4 = 0.
- NOT-A4 = 1.
- NAND inputs: 1, 1, 1, 1.
- NAND output = **0**.
- $\overline{CS} = 0 \rightarrow$ **Timer ACTIVATES**
- Correct: `0xE0` is the FIRST address in range.

Test 2: Address `0xEF` = `1110 1111`

- A7 = 1, A6 = 1, A5 = 1, A4 = 0.
- NOT-A4 = 1.
- NAND inputs: 1, 1, 1, 1.
- NAND output = **0**.
- $\overline{CS} = 0 \rightarrow$ **Timer ACTIVATES**
- Correct: `0xEF` is the LAST address in range. Bottom bits do not matter.

Test 3: Address `0xF0` = `1111 0000`

- A7 = 1, A6 = 1, A5 = 1, A4 = 1.
- NOT-A4 = 0.
- NAND inputs: 1, 1, 1, 0.
- NAND output = **1**.
- $\overline{CS} = 1 \rightarrow$ **Timer INACTIVE**
- Correct: `0xF0` is JUST OUTSIDE the range (A4 changed from 0 to 1).

Test 4: Address `0xD0` = `1101 0000`

- A7 = 1, A6 = 1, A5 = 0, A4 = 1.
- NOT-A4 = 0.
- NAND inputs: 1, 1, 0, 0.
- NAND output = **1**.
- $\overline{CS} = 1 \rightarrow$ **Timer INACTIVE**
- Correct: `0xD0` is BELOW the range (A5 changed from 1 to 0).

Test 5: Address `0x00` = `0000 0000`

- A7 = 0, A6 = 0, A5 = 0, A4 = 0.
- NOT-A4 = 1.
- NAND inputs: 0, 0, 0, 1.
- NAND output = **1**.

- $\overline{CS} = 1 \rightarrow$ **Timer INACTIVE**
- Correct: Far outside range.

All boundary cases pass. The decoder is correct.

A.7 What This Teaches You as a Researcher

Address decoding is the foundation of ALL MMIO.

- Every peripheral on every ARM SoC — ultimately reduces to this pattern.
- Every PCIe BAR assignment — the MCH does this in programmable hardware.
- Every GPIO register — decoded by gates just like this.
- The IOMMU from Section 6 — performs address decoding with more complex tables, but the underlying logic is structurally identical.

Why MMIO address ranges must be power-of-two aligned and power-of-two in size:

This decoder uses a FIXED PREFIX of high bits.

- We checked A7, A6, A5, A4 — the top 4 bits.
- We ignored A3, A2, A1, A0 — the bottom 4 bits.
- This works because the range size ($16 = 2^4$) matches the number of ignored bits.

What if the range was NOT a power of two?

- Example: `0xE0` to `0xE7` (8 addresses, still power of two — works).
- Example: `0xE0` to `0xE6` (7 addresses — NOT power of two).
- For 7 addresses, you would need a **comparator circuit** instead of simple gates.
- Comparator = more transistors, more power, more propagation delay.
- **Hardware designers ALWAYS use power-of-two ranges to keep decoders simple.**

Problem 2 — MMIO Pipelining Hazard

B.1 The Problem

You have a 5-stage pipeline:

1. IF — Instruction Fetch
2. ID — Instruction Decode
3. EX — Execute
4. MEM — Memory Access
5. WB — Write Back

Two consecutive instructions:

```
Instruction 1:  STORE R1, [0x8000]    ; Write R1's value to MMIO address 0x8000
Instruction 2:  LOAD  R2, [0x8000]    ; Read from MMIO address 0x8000 into R2
```

Critical context:

- `0x8000` is a hardware switch — a physical register on a peripheral device.

- Writing to it causes a PHYSICAL state change in the hardware.
- Reading it back reads the CURRENT hardware state.
- The read-back value MAY DIFFER from what was written.
- Example: Writing `1` might trigger a reset, after which the register reads `0`.

Your job: Identify ALL hazards and explain how hardware mitigates them.

B.2 Understanding the 5-Stage Pipeline

In a normal pipelined CPU, five instructions are in-flight simultaneously:

```
Cycle:     1    2    3    4    5    6    7    8    9
──────────────────────────────────────────────────────

Instr 1:   IF   ID   EX   MEM  WB
Instr 2:        IF   ID   EX   MEM  WB
Instr 3:             IF   ID   EX   MEM  WB
Instr 4:                  IF   ID   EX   MEM  WB
Instr 5:                       IF   ID   EX   MEM  WB
```

Each instruction occupies one stage per cycle. At cycle 5, all five stages are active with different instructions.

For our two instructions:

```
Cycle:     1    2    3    4    5    6    7    8    9
──────────────────────────────────────────────────────

STORE:     IF   ID   EX   MEM  WB
LOAD:           IF   ID   EX   MEM  WB
```

- STORE enters MEM at cycle 4.
- LOAD enters EX at cycle 4.
- LOAD tries to enter MEM at cycle 5.

For normal RAM:

- STORE completes in MEM by cycle 5.
- LOAD enters MEM at cycle 5 and reads the updated value.
- **This works fine for RAM.**

For MMIO:

- STORE to `0x8000` does NOT complete in one cycle.
- It may take 50, 100, or thousands of cycles.
- **The pipeline assumptions break.**

B.3 Hazard 1 — The Structural and Timing Hazard

Definition of structural hazard:

> Two instructions need the SAME hardware resource at the SAME time, but there is only ONE of it.

In our scenario:

- **The resource:** The I/O bus (PCIe link).
- **STORE:** Occupies MEM stage and holds the PCIe bus.
- **LOAD:** Reaches MEM stage one cycle later and wants the SAME bus.

The problem:

- STORE to `0x8000` does not complete in one cycle.
- It may take **50+ cycles** for the write to propagate over PCIe.
- The peripheral must process it.
- The bus must acknowledge completion.
- During ALL of these cycles, the STORE occupies the MEM stage and the bus.

LOAD reaches its MEM stage at cycle 5.

- It finds the bus still busy with the STORE.
- It cannot proceed.
- **The hardware MUST stall the LOAD.**

Visual timeline with stall:

```
Cycle:     1    2    3    4    5    6    7    ...   54   55   56
─────────────────────────────────────────────────────────────────

STORE:     IF   ID   EX   MEM  MEM  MEM  MEM  ...  MEM  WB
LOAD:           IF   ID   EX   stall stall stall ... stall MEM  WB
Instr 3:             IF   ID   stall stall stall ... stall EX   MEM
Instr 4:                  IF   stall stall stall ... stall ID   EX
Instr 5:                       stall stall stall ... stall IF   ID
```

What "stall" means:

- The LOAD freezes in EX.
- All instructions behind it also freeze.
- **Pipeline bubbles are inserted** — cycles where no useful work happens.
- The CPU is frozen for 50 cycles.

The timing hazard:

- The pipeline ASSUMES memory access = 1 cycle.
- MMIO reality = 50+ cycles with wait states.
- **This mismatch between assumption and reality is the timing hazard.**

B.4 Hazard 2 — The Memory-Ordering Hazard and Why Forwarding Fails

What is data forwarding (bypassing)?

Normally, when an instruction produces a result and the next instruction immediately needs it:

```
ADD R1, R2, R3     ; R1 = R2 + R3, result available at EX output
ADD R4, R1, R5     ; Needs R1
```

Without forwarding:

- R1 is written to register file in WB (cycle 5 for first ADD).
- Second ADD reads R1 from register file in ID (cycle 5).

- But second ADD needs R1 in EX (cycle 6).
- **One cycle delay. Stall inserted.**

With forwarding:

- The CPU creates a direct wire from EX output of first instruction to EX input of second instruction.
- Second ADD gets R1 directly at cycle 4.
- **No stall. Correct and fast.**

Why forwarding works for normal arithmetic:

- R1's value was computed by the CPU.
- The CPU KNOWS exactly what it is.
- Forwarding the value is CORRECT.

Why forwarding FAILS for MMIO:

Scenario:

- R1 contains `0x01`.
- `STORE R1, [0x8000]` writes `0x01` to hardware switch.
- Hardware switch behavior: Writing `1` triggers a self-clearing reset.
- After reset, the switch's readable value becomes `0x00`.

If CPU uses forwarding for the LOAD:

- CPU says: "I just stored `0x01` to that address."
- CPU says: "The LOAD wants the value at that address."
- CPU says: "I know the value is `0x01`. I'll forward it directly to R2 without reading the bus."
- **R2 gets `0x01`.**

But the actual hardware state:

- After the STORE completed, the hardware reset itself.
- The actual readable value is `0x00`.
- **The correct value for R2 is `0x00`.**
- **The forwarded value `0x01` is WRONG.**

This is the memory-ordering hazard.

The CPU's forwarding logic has NO KNOWLEDGE of hardware semantics:

- It treats MMIO like RAM.
- RAM assumption: write a value, read it back, get the SAME value.
- **For hardware registers, this assumption is FALSE.**
- MMIO registers can be:
  - Write-only (read returns undefined).
  - Read-only (writes ignored).
  - Self-clearing (write triggers action, then clears).
  - Write-to-toggle (write 1 flips state, write 1 again flips back).
  - Status registers (read returns current state, not last written).

B.5 Hazard 3 — The Store Buffer Hazard

What is a store buffer?

Modern CPUs use a small queue that holds recently issued stores before they are fully committed to memory.

Why?

- Allows CPU to continue executing without waiting for each store to propagate to RAM.
- Hides memory latency.

Store-to-load forwarding from store buffer:

When a subsequent load to the same address occurs:

- CPU checks the store buffer first.
- If the same address is there, it returns the value from the buffer.
- **Instead of going to memory.**

Why this is DANGEROUS for MMIO:

Scenario:

- STORE to `0x8000` sits in store buffer (not yet committed to device).
- LOAD from `0x8000` checks store buffer.
- Store buffer returns the value that was WRITTEN.
- **But the device may have changed state since then!**
- **The value from the store buffer is STALE.**

Even if forwarding registers are correctly disabled for MMIO:

- The store buffer might still intercept the load.
- It returns a buffered value, not the actual hardware state.
- **This is the same correctness problem, one level lower.**

B.6 The Mitigation — Three Mechanisms Working Together

Mechanism 1 — Strongly Ordered, Uncacheable Memory Type The OS kernel programs the page table entry for `0x8000` with special attributes:

On x86:

- PAT (Page Attribute Table) field in page table entry.
- Set to **UC** (Uncacheable) or **WC** (Write Combining).

On ARM:

- MAIR (Memory Attribute Indirection Register).
- Combined with page table attribute bits.

Result on both architectures:

- CPU memory subsystem is told:
  – "This is DEVICE memory."

- "NO caching."
- "NO reordering."
- "NO forwarding."
- "Every access must go ALL THE WAY to the physical device."
- "Every access must COMPLETE before the next access begins."

How the CPU knows:

- When an address is first accessed, the CPU looks up the page table.
- It reads the memory type attributes.
- It changes its behavior for that access based on the attributes.

Mechanism 2 — Store Buffer Flush and Forwarding Disable Upon detecting a Strongly Ordered MMIO access, the Control Unit does this IN ORDER:

Step 1: Drain the store buffer completely

- Every pending store is forced to complete.
- The device must acknowledge each one.
- **This ensures the STORE to `0x8000` is FULLY COMMITTED to hardware.**
- Not sitting half-finished in a buffer.

Step 2: Disable data forwarding for this instruction

- The result of the STORE is NOT forwarded to any subsequent instruction.
- Each instruction must read the ACTUAL value from the physical bus.

Step 3: Insert a memory fence (memory barrier)

- An implicit synchronization point.
- Enforces ordering: STORE must complete FULLY before LOAD is issued.
- The software equivalent is `MFENCE` (x86) or `DSB` (ARM).

Mechanism 3 — Pipeline Stall Until Bus Acknowledgment The pipeline does NOT proceed past the STORE's MEM stage until:

- It receives a **completion signal** from the bus.
- In PCIe terms: a **completion TLP** (Transaction Layer Packet).
- This packet confirms: "The write was received and committed."

Only after this completion arrives:

- STORE leaves MEM stage.
- LOAD enters MEM stage.
- LOAD performs a FRESH bus transaction.
- A real physical read over PCIe to the device at `0x8000`.
- Returns whatever the device's hardware state CURRENTLY is.

Result in R2:

- The ACTUAL hardware state after the write.
- May or may not equal what was written to R1.

- **Whatever the physical hardware reported.**
- **This is correct behavior.**

B.7 The Pipeline Timeline With Mitigation

Assume the STORE takes 50 cycles to complete (including PCIe round-trip):

```
Cycle:   1    2    3    4    5..53   54   55   56   57   58
─────────────────────────────────────────────────────────────

STORE:   IF   ID   EX   MEM  MEM(wait states x50)  WB
LOAD:         IF   ID   EX   stall(x50)...          MEM  WB
Instr 3:           IF   ID   stall(x50)...          EX   MEM
Instr 4:                IF   stall(x50)...          ID   EX
Instr 5:                     stall(x50)...          IF   ID
```

What happens:

Cycles 5–53: STORE occupies MEM, waiting for PCIe completion TLP.

- LOAD is stalled in EX.
- All instructions behind LOAD are stalled in IF and ID.
- **50 pipeline bubbles inserted.**

Cycle 54: Completion TLP arrives.

- STORE moves to WB.
- LOAD moves to MEM.
- LOAD begins its own PCIe read transaction.

Cycles 55+: LOAD waits for its own completion with read data.

- Eventually gets the actual hardware state.

The result in R2 is the ACTUAL hardware state after the write.

- Not the forwarded value.
- Not the store buffer value.
- The REAL value from the physical device.

B.8 The Broader Lesson — Why This Matters for System Software

This problem is NOT hypothetical. This is what device driver writers face EVERY TIME they access hardware registers.

Two patterns driver writers use:

Pattern 1: Kernel-provided MMIO functions Linux:

```
u32 ioread32(void __iomem *addr);
void iowrite32(u32 value, void __iomem *addr);
```

Windows:

```
ULONG READ_REGISTER_ULONG(volatile ULONG *Register);
void WRITE_REGISTER_ULONG(volatile ULONG *Register, ULONG Value);
```

What these functions do internally:

- Include necessary memory barriers automatically.
- Ensure Uncacheable memory attributes are set.
- Handle architecture-specific ordering requirements.

Why they exist:

- A raw pointer dereference — even `volatile` — is NOT sufficient.
- `volatile` prevents compiler optimization but does NOT insert hardware barriers.
- These functions exist because the hardware needs MORE than just `volatile`.

Pattern 2: Explicit memory barrier instructions On ARM:

```
DSB     ; Data Synchronization Barrier
        ; Stalls CPU until ALL outstanding memory transactions complete
        ; Including MMIO writes
```

On x86:

```
MFENCE  ; Memory Fence – full ordering
SFENCE  ; Store Fence – write ordering only
LFENCE  ; Load Fence – read ordering only
```

When needed:

- Software needs explicit control over ordering.
- Hardware would not automatically enforce it.
- Between MMIO writes that must happen in sequence.

B.9 The Infamous Driver Bug

One of the most common and most difficult bug classes in driver development:

> **"Intermittent device misbehavior that depends on timing, CPU speed, and compiler optimization level, and that DISAPPEARS when you add a print statement."**

Why does adding a print statement "fix" the bug?

- The print statement itself adds a timing delay.
- `printk()` or `printf()` is a slow function.
- It takes thousands of cycles.
- During those cycles, the MMIO write in the store buffer has time to complete.
- The device has time to process it.
- By the time the next MMIO read happens, the hardware state is stable.
- **The print statement accidentally acts as a memory barrier!**

When you remove the print statement:

- The code runs faster.
- The MMIO read happens BEFORE the write has propagated.
- You get stale data from the store buffer.
- The device appears to misbehave.

Understanding the hardware mechanism:

- Separates a driver writer who FINDS these bugs.
- From one who spends WEEKS puzzling over them.

Part C: Key Points to Remember

- **Address decoding is the foundation of all MMIO.** Every peripheral reduces to checking address bits with gates.
- **Power-of-two ranges are required** for simple decoder circuits. Non-power-of-two ranges need comparators (more transistors, more delay).
- **MMIO pipelining has THREE hazards:**
  1. **Structural/timing hazard:** MMIO takes 50+ cycles, but pipeline assumes 1 cycle. Bus conflict causes stalls.
  2. **Memory-ordering hazard:** Forwarding returns stale values because hardware state may differ from what was written.
  3. **Store buffer hazard:** Store buffer may return buffered value instead of actual hardware state.
- **Three mitigation mechanisms work together:**
  1. **Strongly ordered memory type:** UC/WC attributes in page tables. No caching, no forwarding, no reordering.
  2. **Store buffer flush + forwarding disable:** Drain all pending stores before MMIO access. Disable forwarding for this instruction. Insert implicit memory fence.
  3. **Pipeline stall until bus acknowledgment:** Wait for completion TLP before allowing next instruction.
- **Driver writers use `ioread32`/`iowrite32` (Linux) or `READ/WRITE_REGISTER` (Windows).** These include barriers automatically. Raw pointer dereference is NOT sufficient.
- **Memory barriers:** `DSB` (ARM), `MFENCE`/`SFENCE`/`LFENCE` (x86). Explicit ordering control when hardware does not enforce it automatically.
- **The "print statement fixes the bug" phenomenon:** The print adds delay, accidentally allowing MMIO writes to complete. The real fix is proper memory barriers.

Part D: One-Page Visual Summary

```
+-----------------------------------------------------------------+
|         MMIO PIPELINING HAZARDS: COMPLETE PICTURE               |
+-----------------------------------------------------------------+
|                                                                  |
|   THE THREE HAZARDS:                                             |
|                                                                  |
```

```
|   1. STRUCTURAL/TIMING HAZARD                                    |
|      • MMIO takes 50+ cycles, pipeline assumes 1 cycle           |
|      • STORE holds bus, LOAD conflicts one cycle later            |
|      • Result: 50 pipeline bubbles, CPU frozen                   |
|                                                                  |
|   2. MEMORY-ORDERING HAZARD                                      |
|      • Forwarding assumes write = read-back value                |
|      • Hardware registers can be self-clearing, write-only, etc.|
|      • Forwarded value may be WRONG                              |
|      • Example: Write 1 → triggers reset → reads back 0         |
|                                                                  |
|   3. STORE BUFFER HAZARD                                         |
|      • Store buffer returns buffered value, not hardware state   |
|      • Even if forwarding registers are disabled                 |
|      • Same correctness problem, one level lower                 |
|                                                                  |
+------------------------------------------------------------------+
|                                                                  |
|   THE THREE MITIGATIONS:                                         |
|                                                                  |
|   1. STRONGLY ORDERED MEMORY TYPE                                |
|      • Page table: UC (x86) or device memory (ARM)               |
|      • No caching, no forwarding, no reordering                  |
|      • Every access goes to physical device                      |
|                                                                  |
|   2. STORE BUFFER FLUSH + FORWARDING DISABLE                     |
|      • Drain all pending stores before MMIO access               |
|      • Disable forwarding for this instruction                   |
|      • Insert implicit memory fence                              |
|                                                                  |
|   3. PIPELINE STALL UNTIL COMPLETION                             |
|      • Wait for PCIe completion TLP                              |
|      • STORE leaves MEM → LOAD enters MEM → fresh bus read       |
|      • Returns ACTUAL hardware state                             |
|                                                                  |
+------------------------------------------------------------------+
|                                                                  |
|   SOFTWARE PATTERNS:                                             |
|   • Linux: ioread32(), iowrite32() — include barriers            |
|   • Windows: READ/WRITE_REGISTER — include barriers              |
|   • ARM: DSB instruction for explicit sync                       |
|   • x86: MFENCE/SFENCE/LFENCE for explicit ordering              |
|                                                                  |
|   THE FAMOUS BUG:                                                |
|   • "Adding printk() fixes it"                                   |
|   • Print adds delay → MMIO write completes → bug disappears      |
|   • Real fix: proper memory barriers, not print statements        |
|                                                                  |
+------------------------------------------------------------------+
```

This completes the Section 7 research package on Deep Practice Problems and Solutions. Every problem is built from absolute zero, every solution is solved with complete step-by-step working, every hazard is explained with real hardware behavior, and nothing is left for you to "figure out."